

# Курс-квест:



## 8 врат продвинутого PostgreSQL

ChatGPT

### Квест-курс PostgreSQL для продвинутых

#### Модуль 1 - Типология расширенные типы и JSON

##### Урок 1 - Пользовательские типы (ENUM, composite types)

###### Шаг 01:

###### Миссия 1

Разработчик Олег хочет хранить статусы заказов в PostgreSQL. Он думает использовать ENUM. Какой вариант наиболее подходит?

Подсказка: ENUM хорош, когда набор значений фиксирован и редко меняется.

**Ответ:**

- **CREATE TYPE order\_status AS ENUM ('new','paid','shipped','delivered')**
- CREATE TYPE order\_status AS TEXT
- CREATE TYPE ENUM AS ('a','b','c')
- CREATE TYPE order\_status ENUM ('new','paid')

###### Шаг 02:

###### Миссия 2

Программистка Анна создает ENUM для дней недели. Какой вариант будет корректным?

Подсказка: ENUM создается через CREATE TYPE ... AS ENUM(...).

**Ответ:**

- CREATE ENUM days ('Mon','Tue')
- **CREATE TYPE days AS ENUM ('Mon','Tue','Wed','Thu','Fri','Sat','Sun')**

- ALTER TYPE days ENUM ('Mon','Tue')
- ENUM days = ('Mon','Tue')

### 🔗 Шаг 03:

#### Миссия 3

Сергей использует ENUM для статусов задач. Какое преимущество ENUM по сравнению с TEXT? Подсказка: Вспомните про гарантированную ограниченность набора значений.

**Ответ:**

- **Гарантия фиксированного набора значений и упрощение валидации**
- Более быстрая работа любых запросов без индексации
- Автоматическое создание индексов без указания
- Можно хранить любые данные без ограничений

### 🔗 Шаг 04:

#### Миссия 4

Маша создаёт таблицу с ENUM-типом gender. Как правильно его использовать в столбце? Подсказка: После объявления типа его можно применять как обычный тип данных.

**Ответ:**

- **CREATE TABLE users (id serial, gender gender)**
- CREATE TABLE users (id serial, gender ENUM)
- CREATE TABLE users (id serial, gender VARCHAR)
- CREATE TABLE users (id serial, ENUM gender)

### 🔗 Шаг 05:

#### Миссия 5

Андрей захотел добавить новое значение в ENUM order\_status. Как это сделать? Подсказка: ALTER TYPE — ключ к изменению ENUM.

**Ответ:**

- **ALTER TYPE order\_status ADD VALUE 'canceled'**
- UPDATE order\_status SET value='canceled'
- INSERT INTO order\_status VALUES ('canceled')
- ALTER TABLE order\_status ADD ENUM 'canceled'

### 🔗 Шаг 06:

#### Миссия 6

Разработчица Катя решила использовать composite type для координат (x,y). Как правильно его создать? Подсказка: Composite type объявляется через CREATE TYPE ... AS (...).

**Ответ:**

- **CREATE TYPE point2d AS (x int, y int)**
- CREATE TYPE (x int, y int) AS point2d
- CREATE COMPOSITE point2d (x int, y int)
- CREATE TYPE point2d ENUM (x,y)

## Шаг 07:

### Миссия 7

Фирма использует composite type address (street, city). Как применить его в таблице?

Подсказка: Composite type можно использовать как обычный тип в столбце.

**Ответ:**

- **CREATE TABLE company (id serial, addr address)**
- CREATE TABLE company (id serial, address ENUM)
- CREATE TABLE company (id serial, street text, city text)
- CREATE TABLE company (id serial, addr VARCHAR)

## Шаг 08:

### Миссия 8

Иван хочет получить поле из composite type. Как правильно обратиться?

Подсказка: Для обращения к полю composite используется точка.

**Ответ:**

- **SELECT (addr).city FROM company**
- SELECT addr->'city' FROM company
- SELECT city FROM addr
- SELECT addr.city FROM company

## Шаг 09:

### Миссия 9

Разработчик Дима хочет хранить телефонные номера и коды стран вместе. Composite type подходит. Почему?

Подсказка: Composite type объединяет несколько полей в одно значение.

**Ответ:**

- **Он объединяет связанные значения в единый объект**
- Он быстрее обрабатывается, чем обычные таблицы
- Он автоматически индексируется
- Он заменяет ENUM

## Шаг 10:

### Миссия 10

 Задача: Сопоставьте типы с их особенностями.

Подсказка: Вспомните, ENUM фиксирует набор, а composite объединяет поля.

**Ответ:**

- **ENUM** — Фиксированный список значений
- **Composite type** — Набор полей, объединённых в один объект
- **TEXT** — Произвольные строки без ограничений
- **JSON** — Хранение структурированных данных в текстовом формате

## Шаг 11:

### Миссия 11

Сопоставьте команды с их назначением.

Подсказка: CREATE TYPE определяет, ALTER TYPE изменяет.

**Ответ:**

- **CREATE TYPE** — Создать новый тип
- **ALTER TYPE** — Изменить существующий тип
- **DROP TYPE** — Удалить тип
- **SELECT (addr).city** — Обратиться к полю composite

## 🔮 Шаг 12:

### Миссия 12

Свяжите ситуации с подходящим типом.

Подсказка: Подумайте: фиксированный список — ENUM, несколько полей — composite.

**Ответ:**

- **Статусы заказов** — ENUM
- **Координаты точки** — Composite type
- **Описание профиля в свободной форме** — TEXT
- **Сложная вложенная структура настроек** — JSON

## 🔮 Шаг 13:

### Миссия 13

Соотнесите ошибки и их причины.

Подсказка: Важно помнить синтаксис и ограничения типов.

**Ответ:**

- **Попытка вставить значение 'done' в ENUM без 'done'** — Значение не существует в списке ENUM
- **SELECT addr.city вместо (addr).city** — Неверный синтаксис composite
- **ALTER TYPE ... DROP VALUE** — В PostgreSQL нельзя удалять значения из ENUM
- **CREATE TYPE days ENUM('Mon')** — Неверный синтаксис CREATE TYPE

## 🔮 Шаг 14:

### Миссия 14

Сопоставьте поле и правильный способ его использования.

Подсказка: Composite раскрывается через скобки, ENUM просто хранит значение.

**Ответ:**

- **ENUM status** — Хранение фиксированного набора статусов
- **Composite address** — Объединение улицы и города
- **(addr).city** — Доступ к полю composite
- **ALTER TYPE status ADD VALUE** — Расширение ENUM новым значением

## 🔮 Шаг 15:

### Миссия 15

Расположите шаги создания ENUM и использования в таблице в правильном порядке.

Подсказка: Сначала создаём тип, потом таблицу, затем вставляем данные.

**Ответ:**

- CREATE TYPE status AS ENUM('new','done')

- CREATE TABLE tasks (id serial, st status)
- INSERT INTO tasks (st) VALUES ('new')
- SELECT \* FROM tasks

#### 💡 Шаг 16:

##### Миссия 16

Расположите шаги создания composite type и использования его в таблице.

Подсказка: Сначала объявляем тип, потом применяем в таблице.

**Ответ:**

- CREATE TYPE point3d AS (x int, y int, z int)
- CREATE TABLE coords (id serial, pt point3d)
- INSERT INTO coords (pt) VALUES ((1,2,3))
- SELECT (pt).z FROM coords

## 📖 Урок 2 - JSON и JSONB структуры, индексирование

#### 💡 Шаг 01:

##### Миссия 1

Разработчик Дима сохраняет данные о заказах в формате JSON. Какой тип в PostgreSQL предпочтительнее использовать, если ему нужно часто выполнять поиск по ключам?

Подсказка: Подумай, какой тип поддерживает индексирование и более эффективен при фильтрации.

**Ответ:**

- **JSONB**
- JSON
- TEXT
- BYTEA

#### 💡 Шаг 02:

##### Миссия 2

Марина хранит настройки пользователей в JSONB. Она хочет убедиться, что структура не изменяется случайно. Какой оператор позволит проверить наличие ключа?

Подсказка: Оператор похож на стрелку, но короче.

**Ответ:**

- **?**
- ->
- ->>
- @>

#### 💡 Шаг 03:

##### Миссия 3

Разработчик Саша анализирует логи в формате JSONB. Какой индекс ускорит поиск по конкретному ключу в JSONB?

Подсказка: Индекс поддерживает быстрые операции вхождения и проверки ключей.

**Ответ:**

- GIN
- BTREE
- HASH
- SP-GiST

🔮 **Шаг 04:**

#### **Миссия 4**

Василий работает с заказами в JSONB и хочет извлечь текстовое значение поля `customer_name`. Какой оператор ему поможет?

Подсказка: Один оператор возвращает JSON, а другой — строку.

**Ответ:**

- `->>`
- `->`
- `#>`
- `@>`

🔮 **Шаг 05:**

#### **Миссия 5**

Оля проектирует сервис на PostgreSQL и выбирает между JSON и JSONB. В чём главное отличие JSONB?

Подсказка: Вспомни про бинарное хранение и возможности индексации.

**Ответ:**

- **Хранение в бинарном виде с поддержкой индексов**
- Хранение только строковых значений
- Поддержка исключительно массивов
- Работа только с функцией `to_json`

🔮 **Шаг 06:**

#### **Миссия 6**

Инженер Андрей работает с массивами в JSONB. Какой оператор проверит, что массив содержит определённое значение?

Подсказка: Этот оператор используется и для ключей, и для значений.

**Ответ:**

- `@>`
- `->>`
- `?&`
- `||`

🔮 **Шаг 07:**

#### **Миссия 7**

Компания хранит JSONB с большим количеством данных. Какой индекс использовать для поиска по условию `@> ?`

Подсказка: Этот индекс работает лучше всего с операторами вхождения.

**Ответ:**

- **GIN**
- BRIN
- HASH
- BTREE

 **Шаг 08:**

### Миссия 8

 **Задача:** Сопоставьте операторы JSONB с их назначением.

Подсказка: Вспомни, какой оператор проверяет ключ, а какой возвращает текст.

**Ответ:**

- **->** — Извлекает значение как JSON
- **->>** — Извлекает значение как текст
- **?** — Проверяет наличие ключа
- **@>** — Проверяет, содержит ли JSONB указанный фрагмент

 **Шаг 09:**

### Миссия 9

 **Задача:** Сопоставьте типы индексов с их применением к JSONB.

Подсказка: Один тип универсален, другой — оптимален для вхождения.

**Ответ:**

- **GIN** — Оптимален для поиска по ключам и вхождениям
- **BTREE** — Работает с простыми сравнениями, но неэффективен для JSONB
- **HASH** — Подходит для равенства, но редко используется с JSONB
- **BRIN** — Эффективен для больших последовательных данных, но не для JSONB

 **Шаг 10:**

### Миссия 10

 **Задача:** Сопоставьте функции PostgreSQL с их описанием.

Подсказка: Подумай, какая функция превращает строку в JSON, а какая — наоборот.

**Ответ:**

- **to\_json** — Преобразует значение в JSON
- **jsonb\_each** — Разворачивает объект JSONB в набор ключ-значение
- **jsonb\_array\_elements** — Разворачивает массив JSONB в строки
- **jsonb\_object\_agg** — Агрегирует пары ключ-значение в объект JSONB

 **Шаг 11:**

### Миссия 11

 **Задача:** Сопоставьте реальные задачи программистов с подходящими JSONB функциями.

Подсказка: Смотри, какая функция работает с массивами, а какая с объектами.

**Ответ:**

- **Извлечение каждого элемента массива** — `jsonb_array_elements`
- **Получение всех ключей объекта** — `jsonb_object_keys`
- **Агрегация данных в JSONB** — `jsonb_agg`

- Преобразование значения в JSON — to\_json

## Шаг 12:

### Миссия 12

Расположите шаги по созданию индекса GIN для JSONB в правильном порядке:

Подсказка: Сначала идёт определение таблицы, потом индекс.

**Ответ:**

- Создание таблицы с колонкой JSONB
- Наполнение таблицы данными
- Создание индекса GIN
- Запрос с использованием оператора @>

## Шаг 13:

### Миссия 13

Расположите шаги при извлечении данных из JSONB в правильном порядке:

Подсказка: Сначала выбирается таблица, потом оператор, потом фильтрация.

**Ответ:**

- Выбор таблицы
- Использование оператора ->>
- Указание фильтрации по значению
- Возврат результата пользователю

## Шаг 14:

### Миссия 14

Расположите шаги оптимизации запроса с JSONB:

Подсказка: Сначала изучи план, потом подумай об индексе.

**Ответ:**

- Выполнить EXPLAIN для анализа плана
- Создать индекс GIN
- Повторить запрос с индексом
- Сравнить производительность

## Шаг 15:

### Миссия 15

Расположите шаги при работе с массивом JSONB:

Подсказка: Сначала разбиваем массив, потом фильтруем.

**Ответ:**

- Использование функции jsonb\_array\_elements
- Фильтрация по условию
- Агрегация результатов
- Возврат данных пользователю

## Шаг 16:

### Миссия 16

Расположите шаги при обновлении значения в JSONB:

Подсказка: Подумай, как идёт логика: выбор таблицы → оператор обновления → проверка.

**Ответ:**

- Выбрать строку с JSONB
- Использовать функцию jsonb\_set
- Выполнить UPDATE
- Проверить новое значение

## Урок 3 - Функции работы с JSON (jsonb\_extract\_path, →, →>)

### Шаг 01:

#### Миссия 1

Разработчик хочет получить из JSONB значение поля "name". Какой оператор он должен использовать?

Подсказка: Один оператор возвращает JSON-объект, а другой — текстовое значение.

**Ответ:**

- **Оператор →>**
- Оператор →
- Функция jsonb\_array\_length()
- Оператор #>

### Шаг 02:

#### Миссия 2

Аналитик вытягивает поле "age" из JSONB. Ему нужно именно число, а не JSON. Что выбрать?

Подсказка: Разница между → и →> важна.

**Ответ:**

- Оператор →
- **Оператор →>**
- Функция jsonb\_each\_text()
- Оператор @@

### Шаг 03:

#### Миссия 3

В JSON хранится массив заказов. Нужно получить первый элемент массива. Что использовать?

Подсказка: Индексация начинается с 0.

**Ответ:**

- **orders → 0**
- orders →> 1
- jsonb\_extract\_path\_text(orders, '0')
- orders #> '{1}'

### Шаг 04:

#### Миссия 4

У вас JSONB с вложенным объектом: {"user":{"profile":{"city":"Berlin"}}}. Как получить город?

Подсказка: jsonb\_extract\_path может пройти по цепочке ключей.

**Ответ:**

- **jsonb\_extract\_path(data, 'user', 'profile', 'city')**
- data → 'user.profile.city'
- data →> 'profile.city'
- data @@ 'city = Berlin'

 **Шаг 05:**

### Миссия 5

Разработчик хочет разницу: → или →> ? Он заметил, что одно возвращает JSON, другое текст. Где вернётся JSON?

Подсказка: Вспомните: текст — это финальный результат, JSON можно дальше обрабатывать.

**Ответ:**

- →
- →>
- Оба возвращают только текст
- Ни один не возвращает JSON

 **Шаг 06:**

### Миссия 6

Сопоставьте функцию или оператор с их результатом при работе с JSONB.

Подсказка: Внимательно сравнивайте, где результат текст, а где — JSON.

**Ответ:**

- → — Возвращает JSON-объект или массив
- →> — Возвращает текстовое значение
- **jsonb\_extract\_path** — Достаёт значение по цепочке ключей
- **jsonb\_each\_text** — Разворачивает объект в набор ключ-значение с текстом

 **Шаг 07:**

### Миссия 7

Сопоставьте примеры запросов с ожидаемыми результатами.

Подсказка: Думайте, где вернётся массив, а где строка.

**Ответ:**

- **orders → 0** — Первый элемент массива в виде JSON
- **orders →> 0** — Первый элемент массива как текст
- **data → 'user'** — JSON-подобный объект user
- **data →> 'user'** — Строковое представление объекта user

 **Шаг 08:**

### Миссия 8

Сопоставьте функции с их практическим применением.

Подсказка: Ориентируйтесь на тип возвращаемого значения.

**Ответ:**

- **jsonb\_each** — Возврат ключей и значений как JSON
- **jsonb\_each\_text** — Возврат ключей и значений как текст

- **jsonb\_extract\_path\_text** — Получение текстового значения по ключам
- **jsonb\_array\_elements** — Разворачивание массива JSON в набор элементов

#### 🔮 Шаг 09:

##### Миссия 9

Сопоставьте операторы с их назначением.

Подсказка: Разные операторы работают с вложенностью и массивами.

**Ответ:**

- **#>** — Извлекает JSON по указанному пути
- **#>>** — Извлекает текст по пути
- **||** — Объединение JSONB объектов
- **-** — Удаление ключа из JSONB

#### 🔮 Шаг 10:

##### Миссия 10

Сопоставьте SQL-примеры с результатами.

Подсказка: Заметьте разницу между jsonb\_array\_elements и оператором →.

**Ответ:**

- **jsonb\_array\_elements('[1,2,3]')** — Вернёт три строки: 1,2,3
- **'[1,2,3]'::jsonb → 1** — Вернёт JSON 2
- **'[1,2,3]'::jsonb →> 2** — Вернёт текст '3'
- **jsonb\_extract\_path\_text('{ "a": { "b": 5 } }', 'a', 'b')** — Вернёт '5'

#### 🔮 Шаг 11:

##### Миссия 11

Сопоставьте разработчиков с их задачами по JSON.

Подсказка: Каждая задача связана с конкретной функцией.

**Ответ:**

- **Анна** — Хочет развернуть массив JSON в строки
- **Игорь** — Хочет получить текстовое значение по ключу
- **Марина** — Нужно удалить ключ из JSON
- **Павел** — Хочет объединить два JSONB

#### 🔮 Шаг 12:

##### Миссия 12

Расположите шаги получения значения поля "city" из вложенного JSON: {"user":{"profile":{"city":"Paris"}}}.

Подсказка: Сначала идём к user, потом profile, потом city.

**Ответ:**

- jsonb\_extract\_path(data,'user')
- jsonb\_extract\_path(data,'user','profile')
- jsonb\_extract\_path(data,'user','profile','city')
- Вернуть текстовое значение города

## Шаг 13:

### Миссия 13

Расположите шаги для извлечения первого элемента массива JSONB.

Подсказка: Индексация массива начинается с 0.

#### Ответ:

- Создать JSONB массив
- Применить оператор  $\rightarrow$  с индексом 0
- Получить первый элемент как JSON
- При необходимости преобразовать в текст

## Шаг 14:

### Миссия 14

Упорядочите действия программиста, который хочет сравнить значение внутри JSONB.

Подсказка: Последовательно выделите путь, извлеките текст, затем сравните.

#### Ответ:

- Выбрать JSONB столбец
- Указать путь с помощью  $\#>>$
- Получить текстовое значение
- Сравнить его с искомым значением

## Шаг 15:

### Миссия 15

Расположите шаги для объединения двух JSONB объектов.

Подсказка: Используется оператор объединения.

#### Ответ:

- Взять первый JSONB
- Взять второй JSONB
- Применить оператор  $||$
- Получить объединённый JSONB

## Шаг 16:

### Миссия 16

Расположите шаги программиста при удалении ключа из JSONB.

Подсказка: Используется оператор  $-$ .

#### Ответ:

- Выбрать JSONB объект
- Указать ключ для удаления
- Применить оператор  $-$
- Получить JSONB без ключа

## Шаг 17:

### Миссия 17

Расположите шаги для получения значения по вложенному пути в JSONB.

Подсказка: `jsonb_extract_path` принимает несколько аргументов — от внешнего к внутреннему ключу.

**Ответ:**

- Определить JSONB объект
- Выбрать цепочку ключей
- Применить `jsonb_extract_path`
- Вернуть значение

## Урок 4 - Хранение и запросы по semi-structured данным

### Шаг 01:

#### Миссия 1

Аня решила хранить заказы в JSONB-колонке. Она хочет быстро проверить, какой тип данных лучше подходит для semi-structured информации.

Подсказка: Вспомни различия между JSON и JSONB в PostgreSQL.

**Ответ:**

- **JSONB** — так как поддерживает индексацию и более гибкие запросы
- **JSON** — потому что он хранит текст как есть, быстрее работает
- **JSONB** — потому что он вообще не занимает места
- **JSON** — так как у него есть встроенные B-Tree индексы

### Шаг 02:

#### Миссия 2

Игорь пишет SQL-запрос для поиска по полю `details→'status'` внутри JSONB. Какой оператор ему вернёт текстовое значение?

Подсказка: Один оператор возвращает JSON-объект, другой строку.

**Ответ:**

- `→`
- `→>`
- `#>`
- `@>`

### Шаг 03:

#### Миссия 3

В JSONB-колонке `data` лежат сведения о пользователях. Как извлечь `email` пользователя?

Подсказка: Обрати внимание на разницу между ключом и путём.

**Ответ:**

- **`SELECT data→>'email' FROM users;`**
- `SELECT data→'email' FROM users;`
- `SELECT jsonb_extract_path(data,'email')::int FROM users;`
- `SELECT email FROM data;`

### Шаг 04:

#### Миссия 4

Павел строит индекс для ускорения запросов по JSONB. Какой тип индекса он выберет?

Подсказка: Для JSONB оптимален индекс, работающий с множеством ключей и значений.

**Ответ:**

- B-Tree
- **GIN**
- BRIN
- HASH

## 🔮 Шаг 05:

### Миссия 6

В команде обсуждают, как лучше хранить semi-structured данные. Сопоставьте формат с его особенностью.

Подсказка: JSONB часто предпочтительнее из-за индексов.

**Ответ:**

- **JSON** — Хранит текст как есть, без оптимизации
- **JSONB** — Хранит бинарно, поддерживает индексацию
- **XML** — Подходит для данных с жёсткой структурой
- **HSTORE** — Простой key-value тип в PostgreSQL

## 🔮 Шаг 06:

### Миссия 7

Сопоставьте задачу с правильной функцией.

Подсказка: Вспомни, когда нужен текст, а когда JSON.

**Ответ:**

- **Извлечь email строкой** — `→>`
- **Получить объект address** — `→`
- **Извлечь вложенный путь {info,phone}** — `#>`
- **Извлечь JSON значение по ключу** — `jsonb_extract_path`

## 🔮 Шаг 07:

### Миссия 8

Сопоставьте ситуацию с оптимальным решением.

Подсказка: Индексация ускоряет поиск по ключам.

**Ответ:**

- **Поиск по статусу в JSONB** — GIN-индекс
- **Поиск по диапазону чисел** — B-Tree
- **Хранение логов с полу-структурой** — JSONB
- **Хранение фиксированной таблицы товаров** — Обычные столбцы

## 🔮 Шаг 08:

### Миссия 9

Сопоставьте пример кода с ожидаемым результатом.

Подсказка: Внимательно смотри, возвращается ли JSON или текст.

**Ответ:**

- **SELECT data→'profile';** — Возвращает JSON-объект

- **SELECT data→>'name';** — Возвращает строку
- **SELECT data#>'{'contact,phone}';** — Возвращает вложенный JSON
- **SELECT jsonb\_extract\_path(data,'email');** — Возвращает JSON по ключу

#### 🔧 Шаг 09:

##### Миссия 10

Расположи шаги при работе с semi-structured данными в правильном порядке.

Подсказка: Сначала нужно спроектировать схему, а индексация идёт позже.

##### Ответ:

- Определение ключей и структуры
- Создание JSONB-колонки
- Заполнение данными
- Создание индекса
- Выполнение запросов

#### 🔧 Шаг 10:

##### Миссия 11

Расположи запросы по сложности от простого к сложному.

Подсказка: Сначала простое извлечение, потом вложенные структуры.

##### Ответ:

- **SELECT data→>'name' FROM users;**
- **SELECT data→'profile' FROM users;**
- **SELECT data#>'{'address,city}'' FROM users;**
- **SELECT jsonb\_extract\_path(data,'orders','0','price') FROM users;**

#### 🔧 Шаг 11:

##### Миссия 12

Расположи действия аналитика, работающего с логами в JSONB.

Подсказка: Сначала он проверяет структуру, потом строит запросы.

##### Ответ:

- Изучение структуры JSON
- Написание простых запросов
- Создание индекса для ускорения
- Сложные запросы с условиями
- Анализ результата

#### 🔧 Шаг 12:

##### Миссия 13

Расположи шаги оптимизации запроса по JSONB.

Подсказка: Индексация и EXPLAIN — важные этапы.

##### Ответ:

- Написание базового запроса
- Проверка плана через EXPLAIN
- Создание GIN-индекса

- Повторный анализ плана
- Финальный запуск запроса

### 🔮 Шаг 13:

#### Миссия 14

Расположи типы данных по степени структурированности.

Подсказка: От key-value до реляционных таблиц.

**Ответ:**

- HSTORE
- JSON
- JSONB
- Обычные таблицы

### 🔮 Шаг 14:

#### Миссия 15

Расположи этапы внедрения semi-structured данных в проект.

Подсказка: От эксперимента до интеграции в продакшн.

**Ответ:**

- Прототипирование с JSONB
- Анализ производительности
- Добавление индексов
- Интеграция в продакшн
- Мониторинг и оптимизация

## 📖 Урок 5 - Мини-босс создать систему хранения настроек пользователя

### 🔮 Шаг 01:

#### Миссия 1

Разработчик решил хранить настройки пользователей в PostgreSQL. Какой тип данных лучше всего подойдёт для хранения semi-structured данных с разными ключами у разных пользователей?

Подсказка: Подумайте о типе, который позволяет хранить пары ключ-значение и быстро искать по ключу.

**Ответ:**

- jsonb
- text
- integer
- uuid

### 🔮 Шаг 02:

#### Миссия 2

Алиса хранит настройки профиля в колонке settings (тип jsonb). Какой оператор поможет извлечь под-объект по ключу?

Подсказка: Один оператор возвращает JSON-объект, другой — текстовое значение.

**Ответ:**

- →
- →>
- #>>
- @@

### 🔮 Шаг 03:

#### Миссия 3

У Карла настройки занимают много места. Какое преимущество у jsonb перед json в PostgreSQL?

Подсказка: Подумайте о хранении и индексах.

**Ответ:**

- **jsonb поддерживает индексацию и хранится в бинарном виде**
- jsonb хранит только строки
- json быстрее чем jsonb
- jsonb запрещает вложенные объекты

### 🔮 Шаг 04:

#### Миссия 4

Сопоставьте оператор работы с jsonb с его назначением.

Подсказка: Вспомните разницу между получением объекта, текста и проверкой ключа.

**Ответ:**

- → — Извлечение JSON-объекта по ключу
- →> — Извлечение текстового значения по ключу
- ? — Проверка наличия ключа
- @> — Проверка вхождения подструктуры

### 🔮 Шаг 05:

#### Миссия 5

Сопоставьте инструмент PostgreSQL с его назначением при работе с настройками.

Подсказка: Думайте про индексы, операторы и функции.

**Ответ:**

- **GIN индекс** — Ускоряет поиск по ключам jsonb
- **Функция jsonb\_extract\_path** — Извлекает значение по пути
- **Оператор @>** — Проверяет вхождение поддокумента
- **Оператор ?|** — Проверяет наличие хотя бы одного ключа из списка

### 🔮 Шаг 06:

#### Миссия 6

Сопоставьте программиста с его задачей при проектировании системы настроек.

Подсказка: Каждому персонажу нужна своя операция.

**Ответ:**

- **Алиса** — Хочет быстро находить пользователей с включённым dark mode
- **Боб** — Хочет проверить наличие ключа timezone
- **Карл** — Хочет извлечь объект notifications

- **Дана** — Хочет хранить подструктуру language с кодом языка

## Шаг 07:

### Миссия 7

Расставьте шаги проектирования системы хранения пользовательских настроек в правильном порядке.

Подсказка: Сначала выбор типа, потом структура, затем индексация и тестирование.

#### Ответ:

- Выбор формата хранения (json или jsonb)
- Определение ключей настроек
- Создание индексов для ускорения запросов
- Тестирование выборок на реальных данных

## Шаг 08:

### Миссия 8

Упорядочьте действия программиста, который хочет извлечь текстовое значение из jsonb.

Подсказка: Вспомните про → и →>.

#### Ответ:

- Определить нужный ключ
- Выбрать оператор →> для извлечения текста
- Написать SQL-запрос
- Выполнить запрос и проверить результат

## Шаг 09:

### Миссия 9

Расставьте этапы настройки GIN индекса для колонки settings.

Подсказка: Индекс создаётся отдельной командой и проверяется запросом.

#### Ответ:

- Определить колонку settings
- Выбрать тип индекса GIN
- Создать индекс CREATE INDEX ... USING gin
- Выполнить тестовый запрос с условием @>

## Шаг 10:

### Миссия 10

Поставьте в правильный порядок действия по обновлению JSON-настроек.

Подсказка: Используется оператор || для слияния объектов.

#### Ответ:

- Определить новое значение для ключа
- Написать оператор обновления с ||
- Выполнить команду UPDATE
- Проверить, что настройки обновились

## Шаг 11:

### Миссия 11

Расставьте шаги анализа производительности запросов к jsonb-колонке.

Подсказка: Подумайте про EXPLAIN.

#### Ответ:

- Написать запрос с фильтром по ключу
- Запустить EXPLAIN ANALYZE
- Сравнить план с индексом и без индекса
- Сделать вывод о необходимости индекса

## Шаг 12:

### Миссия 12

Расставьте шаги интеграции пользовательских настроек в веб-приложение.

Подсказка: Настройки сначала сохраняются, потом читаются при логине.

#### Ответ:

- Формирование JSON с настройками на клиенте
- Сохранение JSON в PostgreSQL
- Чтение JSON при логине
- Применение настроек к интерфейсу

## Шаг 13:

### Миссия 13

Упорядочьте шаги финальной оптимизации системы хранения настроек.

Подсказка: Индексация и нормализация — последние шаги.

#### Ответ:

- Анализ реальных запросов
- Выбор часто используемых ключей
- Создание частичных индексов
- Нормализация редко используемых настроек

## Модуль 2 - Гибкие структуры массивы, HSTORE, UUID

### Урок 1 - Массивы и работа с ними (unnest, array\_agg)

## Шаг 01:

### Миссия 1

Антон работает над сервисом, где пользователи могут хранить список любимых фильмов. Он решил использовать массивы в PostgreSQL. Какой тип данных лучше выбрать для хранения массива строк?

Подсказка: В PostgreSQL можно напрямую создавать массивы стандартных типов.

#### Ответ:

- **text[]**
- varchar
- json

- `uuid[]`

## 🔮 Шаг 02:

### Миссия 2

Мария создала таблицу `users` с колонкой `interests` типа `text[]`. Какой оператор позволит проверить, содержит ли массив конкретное значение?

Подсказка: В PostgreSQL для массивов есть специальные операторы сравнения.

**Ответ:**

- **`= ANY(array)`**
- `IN (array)`
- `HAS(array)`
- `LIKE array`

## 🔮 Шаг 03:

### Миссия 3

Денису нужно объединить все email-адреса сотрудников в одну строку. Какую функцию он может использовать для агрегации массива в строку?

Подсказка: Эта функция умеет соединять элементы массива через разделитель.

**Ответ:**

- **`array_to_string`**
- `string_agg`
- `concat_ws`
- `text_agg`

## 🔮 Шаг 04:

### Миссия 4

Андрей хранит список оценок студентов в массиве `integer[]`. Как получить максимальное значение из массива?

Подсказка: Иногда нужно развернуть массив, прежде чем агрегировать его значения.

**Ответ:**

- **`unnest()` + `MAX()`**
- `array_max()`
- `aggregate(array)`
- `first_value(array)`

## 🔮 Шаг 05:

### Миссия 5

Катя хочет создать массив чисел от 1 до 10 прямо в запросе. Какую функцию PostgreSQL она использует?

Подсказка: Эта функция часто применяется для генерации последовательностей.

**Ответ:**

- **`generate_series()`**
- `array_series()`
- `seq_array()`
- `make_series()`

## 🔗 Шаг 06:

### Миссия 6

Иван хранит массив логинов пользователей. Какая функция вернёт количество элементов в массиве?

Подсказка: Она работает не только для массивов, но и для строк.

**Ответ:**

- **cardinality()**
- `count_elements()`
- `array_length()` без индекса
- `len()`

## 🔗 Шаг 07:

### Миссия 7

Саша решил объединить все значения поля `phone` в массив. Какая функция выполняет агрегацию в массив?

Подсказка: Она часто используется с `GROUP BY`.

**Ответ:**

- **array\_agg()**
- `string_agg()`
- `agg_array()`
- `collect()`

## 🔗 Шаг 08:

### Миссия 8

Елена заметила, что в массиве хранятся повторяющиеся элементы. Какой способ проще всего убрать дубликаты?

Подсказка: Иногда помогает перевод массива во временную таблицу и обратно.

**Ответ:**

- **unnest → DISTINCT → array\_agg**
- `array_unique()`
- `filter(array)`
- `dedup(array)`

## 🔗 Шаг 09:

### Миссия 9

📄 Сопоставьте функции работы с массивами с их назначением:

Подсказка: Вспомните, какие функции отвечают за разворачивание массива, а какие — за агрегацию.

**Ответ:**

- **unnest()** — Разворачивает массив в строки
- **array\_agg()** — Объединяет строки в массив
- **cardinality()** — Возвращает количество элементов
- **array\_to\_string()** — Преобразует массив в строку

## 🔮 Шаг 10:

### Миссия 10

📖 Сопоставьте выражение и результат:

Подсказка: Присмотритесь к тому, как работает генерация и агрегация.

**Ответ:**

- **SELECT array[1,2,3];** — {1,2,3}
- **SELECT array\_length(ARRAY[10,20,30],1);** — 3
- **SELECT unnest(ARRAY['a','b']);** — a, b
- **SELECT array\_agg(id) FROM users;** — Массив всех id пользователей

## 🔮 Шаг 11:

### Миссия 11

📖 Сопоставьте ситуацию и SQL-решение:

Подсказка: Иногда нужно комбинировать unnest с агрегатами.

**Ответ:**

- **Найти все уникальные элементы массива** — unnest → DISTINCT → array\_agg
- **Подсчитать количество элементов** — cardinality()
- **Создать массив всех email из таблицы** — array\_agg(email)
- **Развернуть массив чисел для сортировки** — unnest() + ORDER BY

## 🔮 Шаг 12:

### Миссия 12

📖 Сопоставьте оператор и его смысл:

Подсказка: Массивы имеют свои собственные операторы сравнения.

**Ответ:**

- **= ANY(array)** — Проверяет, есть ли значение в массиве
- **= ALL(array)** — Проверяет, равно ли значение всем элементам массива
- **array[1,2] || array[3]** — Конкатенация массивов
- **array[2,3] @> array[2]** — Проверка: содержит ли массив подмассив

## 🔮 Шаг 13:

### Миссия 13

📖 Сопоставьте функцию с примером результата:

Подсказка: Внимательно смотрите на формат вывода.

**Ответ:**

- **array\_position(ARRAY['x','y','z'],'y')** — 2
- **array\_length(ARRAY[10,20,30],1)** — 3
- **array\_to\_string(ARRAY['pg','sql'],'-')** — pg-sql
- **unnest(ARRAY[5,6])** — 5, 6

## 🔮 Шаг 14:

### Миссия 14

Расположите шаги для получения уникальных значений из массива:

Подсказка: Сначала массив разворачивается, затем применяются фильтры, и только потом он снова агрегируется.

**Ответ:**

- `unnest()`
- `DISTINCT`
- `array_agg()`
- Получение уникального массива

#### Шаг 15:

##### **Миссия 15**

Расположите шаги для нахождения максимального числа в массиве:

Подсказка: Нужно развернуть массив, применить агрегат, получить результат.

**Ответ:**

- `unnest()`
- `MAX()`
- Возврат одного числа
- **Результат** — максимум

#### Шаг 16:

##### **Миссия 16**

Расположите шаги для объединения значений в массив строк:

Подсказка: Нужно использовать агрегацию и финальное преобразование.

**Ответ:**

- `array_agg()`
- `array_to_string()`
- Добавление разделителя
- Строка-результат

#### Шаг 17:

##### **Миссия 17**

Расположите шаги для генерации массива чисел и сортировки по убыванию:

Подсказка: Вспомните про `generate_series` и преобразования.

**Ответ:**

- `generate_series()`
- `array_agg()`
- `ORDER BY DESC`
- Получение массива

## Урок 2 - HSTORE — key-value внутри строки

#### Шаг 01:

##### **Миссия 1**

Алексей проектирует систему профилей пользователей. Ему нужно хранить произвольные пары ключ-значение (например, `theme=dark, language=ru`). Какой тип PostgreSQL лучше всего использовать?

Подсказка: Вспомни тип, позволяющий хранить key-value в одной строке.

**Ответ:**

- **HSTORE**
- ARRAY
- UUID
- JSONB

## Шаг 02:

### Миссия 2

Мария хранит в HSTORE данные вида: 'theme' => 'light', 'lang' => 'en' . Как выбрать значение по ключу 'theme' ?

Подсказка: Используется оператор извлечения по ключу.

**Ответ:**

- **settings->'theme'**
- settings['theme']
- get(settings,'theme')
- settings.theme

## Шаг 03:

### Миссия 3

Иван решил проверить, есть ли в HSTORE ключ 'lang' . Какой оператор ему нужен?

Подсказка: Вспомни оператор для проверки существования ключа.

**Ответ:**

- **?**
- ??
- EXISTS
- IN

## Шаг 04:

### Миссия 4

Светлана хочет объединить два HSTORE: 'theme' => 'dark' и 'lang' => 'ru' . Какой оператор применить?

Подсказка: Подумай о символе объединения множеств.

**Ответ:**

- **||**
- &&
- ++
- CONCAT()

## Шаг 05:

### Миссия 5

Антон загружает HSTORE с настройками, но хочет удалить ключ 'debug' . Как это сделать?

Подсказка: Вспомни оператор вычитания.

**Ответ:**

- **settings - 'debug'**
- `delete(settings,'debug')`
- `settings.exclude('debug')`
- `settings.remove('debug')`

#### Шаг 06:

##### **Миссия 6**

Никита пишет API для хранения сессий. Нужно добавить новую пару ключ-значение в HSTORE. Какой оператор использовать?

Подсказка: Подумай о механике объединения с существующими данными.

**Ответ:**

- `||`
- `+=`
- `INSERT`
- `APPEND`

#### Шаг 07:

##### **Миссия 7**

 Сопоставьте операции с HSTORE с их описанием.

Подсказка: Подумай о CRUD-операциях: получить, проверить, удалить, объединить.

**Ответ:**

- **hstore->'key'** — Получение значения по ключу
- **hstore ? 'key'** — Проверка существования ключа
- **hstore - 'key'** — Удаление ключа
- **h1 || h2** — Объединение двух HSTORE

#### Шаг 08:

##### **Миссия 8**

 Сопоставьте встроенные функции HSTORE с их назначением.

Подсказка: Вспомни функции для ключей и значений.

**Ответ:**

- **akeys(h)** — Получить массив всех ключей
- **avals(h)** — Получить массив всех значений
- **hstore\_to\_json(h)** — Преобразовать HSTORE в JSON
- **skeys(h)** — Получить набор ключей в виде множества

#### Шаг 09:

##### **Миссия 9**

 Сопоставьте результат запроса с его оператором HSTORE.

Подсказка: Обрати внимание: результат может быть текст, bool или новый HSTORE.

**Ответ:**

- **hstore->'theme'** — Текстовое значение
- **hstore ? 'theme'** — Булево значение

- **h1 || h2** — Новый HSTORE
- **hstore - 'key'** — HSTORE без указанного ключа

## 🔧 Шаг 10:

### Миссия 10

📖 Сопоставьте выражения с реальной задачей.

Подсказка: Представь, как программист работает с настройками приложения.

**Ответ:**

- **settings ? 'lang'** — Проверить, есть ли язык в настройках
- **settings->'theme'** — Вывести текущую тему интерфейса
- **settings - 'debug'** — Удалить параметр отладки
- **settings || 'currency=>USD'** — Добавить валюту по умолчанию

## 🔧 Шаг 11:

### Миссия 11

Расположите шаги использования HSTORE в проекте:

1. Выборка данных из таблицы с HSTORE
2. Создание таблицы с полем типа HSTORE
3. Установка расширения hstore
4. Добавление и изменение ключей в HSTORE

Подсказка: Сначала нужно включить расширение, потом создать таблицу, затем изменять и читать данные.

**Ответ:**

- Установка расширения hstore
- Создание таблицы с полем типа HSTORE
- Добавление и изменение ключей в HSTORE
- Выборка данных из таблицы с HSTORE

## 🔧 Шаг 12:

### Миссия 12

Расположите действия программиста, который хочет проверить наличие ключа и при отсутствии — добавить его:

1. Добавить недостающий ключ
2. Проверить наличие ключа оператором ?
3. Считать HSTORE из таблицы
4. Сохранить обновленный HSTORE

Подсказка: Представь рабочий цикл: получить данные → проверить → изменить → сохранить.

**Ответ:**

- Считать HSTORE из таблицы
- Проверить наличие ключа оператором ?
- Добавить недостающий ключ
- Сохранить обновленный HSTORE

### Шаг 13:

#### Миссия 13

Упорядочите шаги для конвертации HSTORE в JSON:

1. Вызвать функцию `hstore_to_json`
2. Создать HSTORE
3. Передать HSTORE в функцию
4. Получить JSON-результат

Подсказка: Подумай: сначала создать данные, затем преобразовать.

**Ответ:**

- Создать HSTORE
- Передать HSTORE в функцию
- Вызвать функцию `hstore_to_json`
- Получить JSON-результат

### Шаг 14:

#### Миссия 14

Расположите шаги для удаления ключа и проверки результата:

1. Сохранить измененные данные
2. Удалить ключ оператором -
3. Получить HSTORE из таблицы
4. Вывести HSTORE для проверки

Подсказка: Вспомни: получить → изменить → проверить → сохранить.

**Ответ:**

- Получить HSTORE из таблицы
- Удалить ключ оператором -
- Вывести HSTORE для проверки
- Сохранить измененные данные

## Урок 3 - UUID, генерация и индексирование

### Шаг 01:

#### Миссия 1

Вася пишет сервис учёта заказов. Он думает, как сгенерировать уникальные идентификаторы для записей. Какой тип данных в PostgreSQL лучше всего подходит для глобально уникальных идентификаторов?

Подсказка: Этот тип не зависит от последовательности и может использоваться даже в распределённых системах.

**Ответ:**

- **UUID**
- BIGSERIAL
- TEXT
- INT

## Шаг 02:

### Миссия 2

Аня проектирует распределённое приложение. Ей нужно, чтобы идентификаторы не пересекались между разными серверами. Какой тип данных решает эту проблему без ручного контроля последовательностей?

Подсказка: Подумайте о генераторе, который создаёт уникальные значения независимо от базы данных.

**Ответ:**

- **UUID**
- SERIAL
- NUMERIC
- CHAR(36)

## Шаг 03:

### Миссия 3

Сергей использует функцию `uuid_generate_v4()`. Что именно она создаёт?

Подсказка: Номер версии здесь важен — в четвёртой версии генерация идёт случайным образом.

**Ответ:**

- **Случайный UUID (версия 4)**
- UUID на основе MAC-адреса (версия 1)
- Последовательное число
- Хэш строки

## Шаг 04:

### Миссия 4

Оля слышала, что UUID хранятся как строки. Но как PostgreSQL реально хранит UUID внутри?

Подсказка: Визуально они похожи на текст, но в базе это совсем другой формат.

**Ответ:**

- **Как бинарное значение длиной 16 байт**
- Как строка длиной 36 символов
- Как массив чисел
- Как JSON

## Шаг 05:

### Миссия 5

Дмитрий проектирует таблицу с UUID-ключами. Какой индекс лучше использовать, чтобы ускорить поиск по UUID?

Подсказка: Этот индекс хорошо работает для уникальных значений.

**Ответ:**

- **B-Tree индекс**
- GIN индекс
- GiST индекс
- BRIN индекс

## Шаг 06:

### Миссия 6

Катя хочет быстро вставлять новые UUID. Какой тип генерации UUID минимизирует вероятность коллизий?

Подсказка: Подумайте о различии между случайной и последовательной генерацией.

**Ответ:**

- **UUID v4 (случайный)**
- UUID v1 (MAC + время)
- SERIAL
- BIGINT

## Шаг 07:

### Миссия 7

 Задача: Сопоставьте версию UUID с её описанием.

Подсказка: Внимательно посмотрите на источники данных для разных версий UUID.

**Ответ:**

- **UUID v1** — Основан на времени и MAC-адресе
- **UUID v3** — Генерируется на основе MD5-хэша и пространства имён
- **UUID v4** — Генерируется случайным образом
- **UUID v5** — Генерируется на основе SHA-1 и пространства имён

## Шаг 08:

### Миссия 8

Сопоставьте команду PostgreSQL с её назначением при работе с UUID.

Подсказка: Обратите внимание на функции `uuid_generate` .

**Ответ:**

- **uuid\_generate\_v1()** — Создание UUID на основе времени и MAC
- **uuid\_generate\_v4()** — Создание случайного UUID
- **uuid\_osp** — Расширение для работы с UUID
- **::uuid** — Преобразование строки в UUID

## Шаг 09:

### Миссия 9

 Задача: Сопоставьте тип индекса с его эффективным применением при UUID.

Подсказка: Не все индексы одинаково полезны для поиска по UUID.

**Ответ:**

- **B-Tree** — Поиск по равенству и сортировка
- **GIN** — Полнотекстовый поиск, JSONB
- **GiST** — Поиск по геометрическим данным
- **BRIN** — Работа с данными, где порядок важен

## Шаг 10:

### Миссия 10

Сопоставьте преимущества и недостатки использования UUID.

Подсказка: Подумайте о размере, читаемости и уникальности.

**Ответ:**

- **Преимущество: уникальность** — UUID гарантирует уникальность в распределённых системах
- **Преимущество: универсальность** — Можно использовать в разных базах и системах
- **Недостаток: размер** — 16 байт больше, чем 8 байт у BIGINT
- **Недостаток: читаемость** — UUID труднее воспринимать человеку

## Шаг 11:

### Миссия 11

Сопоставьте расширение PostgreSQL с его назначением.

Подсказка: UUID требует специального расширения для генерации некоторых версий.

**Ответ:**

- **uuid-oss** — Расширение для генерации UUID разных версий
- **pgcrypto** — Позволяет генерировать криптографически стойкие UUID
- **citext** — Работа с нечувствительным к регистру текстом
- **hstore** — Хранение пар ключ-значение

## Шаг 12:

### Миссия 12

Расположите этапы создания таблицы с UUID ключом в правильном порядке.

Подсказка: Вспомните: сначала создание таблицы, потом вставка и индекс.

**Ответ:**

- CREATE TABLE users (id UUID PRIMARY KEY, name TEXT);
- INSERT INTO users VALUES (uuid\_generate\_v4(), 'Alex');
- Создание индекса по UUID (если нужно)

## Шаг 13:

### Миссия 13

Упорядочьте версии UUID по сложности вычисления (от самой простой к самой сложной).

Подсказка: Самая простая — случайная, самая сложная — с криптографическим хэшем.

**Ответ:**

- UUID v4 (random)
- UUID v1 (time + MAC)
- UUID v3 (MD5 + namespace)
- UUID v5 (SHA-1 + namespace)

## Шаг 14:

### Миссия 14

Расположите действия программиста, который настраивает уникальный ключ на UUID-колонку.

Подсказка: Сначала колонка, потом ограничение.

**Ответ:**

- Добавление колонки id UUID
- Генерация UUID через uuid\_generate\_v4()

- Добавление PRIMARY KEY
- Создание индекса (опционально)

### 🔧 Шаг 15:

#### Миссия 15

Расставьте ситуации по уместности использования UUID (от менее подходящей к более подходящей).

Подсказка: Локальная база с одним сервером — это не про UUID, а распределённые системы наоборот выигрывают.

#### Ответ:

- Маленький проект с одной базой
- Внутренняя корпоративная база
- Распределённое веб-приложение
- Глобальный сервис с миллионами пользователей

## 📖 Урок 4 - Мини-босс спроектировать БД для системы тегов и фильтров

### 🔧 Шаг 01:

#### Миссия 1

Команда стартапа хочет добавить теги к статьям в блоге. Какой подход в PostgreSQL будет самым простым для хранения списка тегов?

Подсказка: Подумайте о типах данных, которые позволяют хранить массив строк.

#### Ответ:

- **Использовать массивы (ARRAY text[])**
- Хранить все теги в одной строке через запятую
- Создать отдельную таблицу для каждой статьи
- Использовать числовой тип данных

### 🔧 Шаг 02:

#### Миссия 2

Разработчику нужно добавить возможность быстрого поиска по тегам. Какой тип индекса подойдёт лучше всего для массива строк?

Подсказка: Индексы бывают разных видов: btree, gin, gist. Подумайте, какой из них оптимизирован для работы с массивами.

#### Ответ:

- **GIN-индекс**
- B-Tree индекс
- BRIN-индекс
- Hash-индекс

### 🔧 Шаг 03:

#### Миссия 3

Команда обсуждает, стоит ли использовать HSTORE для хранения тегов в формате ключ-значение. Для каких случаев это уместно?

Подсказка: Вспомните ситуации, когда теги могут нести дополнительные параметры.

**Ответ:**

- **Когда у тегов есть значения (например, цвет:красный)**
- Когда нужны только простые строки
- Когда нужен контроль внешних ключей
- Когда данные всегда числовые

#### **Шаг 04:**

##### **Миссия 4**

Разработчик хочет хранить фильтры по товарам (например, размер=XL, цвет=синий). Что лучше использовать?

Подсказка: Если нужно хранить пары ключ-значение, подумайте про подходящие структуры в PostgreSQL.

**Ответ:**

- **HSTORE или JSONB**
- Простые текстовые поля
- Только массив чисел
- Отдельная таблица на каждый фильтр

#### **Шаг 05:**

##### **Миссия 5**

При проектировании базы данных для тегов обсуждается вариант использовать отдельную таблицу для тегов и связь many-to-many. Какой плюс у такого подхода?

Подсказка: Подумайте про нормализацию и гибкость.

**Ответ:**

- **Можно переиспользовать один тег для многих сущностей**
- Теги всегда хранятся быстрее
- Не нужны индексы
- Гибкость хуже, чем у массива

#### **Шаг 06:**

##### **Миссия 6**

Какой минус у хранения тегов в массиве?

Подсказка: Вспомните, когда может быть сложно поддерживать уникальность и связи.

**Ответ:**

- **Сложно обеспечить уникальность и связи между тегами**
- Массивы нельзя индексировать
- Массивы не поддерживаются в PostgreSQL
- Массивы автоматически теряют данные

#### **Шаг 07:**

##### **Миссия 7**

Компания решила использовать JSONB для хранения тегов и фильтров. Какой плюс у JSONB перед HSTORE?

Подсказка: Подумайте о вложенных структурах.

**Ответ:**

- **Поддержка вложенных объектов и массивов**
- Меньший размер хранения всегда
- JSONB работает только с числами
- JSONB всегда быстрее GIN

## 🔮 Шаг 08:

### Миссия 8

📖 Задача: Сопоставьте тип данных с его характеристикой при хранении тегов.

Подсказка: Вспомните разницу между массивом, HSTORE и JSONB.

**Ответ:**

- **ARRAY** — Хранение простого списка строк
- **HSTORE** — Ключ-значение в одной колонке
- **JSONB** — Хранение вложенных структур
- **Отдельная таблица** — Поддержка связей many-to-many

## 🔮 Шаг 09:

### Миссия 9

📖 Сопоставьте задачу бизнеса с подходящей структурой хранения.

Подсказка: Внимательно читайте примеры: простые, ключ-значение, сложные фильтры.

**Ответ:**

- **Быстрое хранение простых тегов** — ARRAY
- **Гибкие фильтры товара** — HSTORE
- **Сложные настройки с вложенностью** — JSONB
- **Общий список тегов на всю систему** — Отдельная таблица

## 🔮 Шаг 10:

### Миссия 10

📖 Сопоставьте тип индекса с его назначением для тегов и фильтров.

Подсказка: GIN и GiST подходят для одних случаев, B-Tree — для других.

**Ответ:**

- **B-Tree** — Лучше для точных значений и сортировки
- **GIN** — Быстрый поиск по массивам и JSONB
- **GiST** — Гибкие индексы, подходят для пространственных данных
- **Hash** — Редко используется, равенство по хешу

## 🔮 Шаг 11:

### Миссия 11

📖 Сопоставьте задачу программиста с проблемой, которая может возникнуть.

Подсказка: У каждой структуры есть слабые места.

**Ответ:**

- **Использовать массивы** — Сложность поддержания уникальности
- **Использовать HSTORE** — Нет вложенных структур
- **Использовать JSONB** — Потенциально большее потребление памяти

- **Использовать отдельные таблицы** — Усложнение схемы и запросов

## Шаг 12:

### Миссия 12

 Сопоставьте команду разработчиков с их решением по хранению тегов.

Подсказка: Вспомните: стартапы любят быстро, корпорации — гибко.

**Ответ:**

- **Небольшой блог** — ARRAY
- **Магазин одежды** — HSTORE
- **Маркетплейс с категориями** — JSONB
- **Большая соцсеть** — Отдельная таблица

## Шаг 13:

### Миссия 13

Расположите шаги проектирования системы тегов от простого к сложному.

Подсказка: Сначала минимальное решение, потом усложняем.

**Ответ:**

- Использовать массивы для простых тегов
- Добавить HSTORE для фильтров
- Перейти на JSONB для вложенных данных
- Спроектировать отдельные таблицы для масштабируемости

## Шаг 14:

### Миссия 14

Расположите действия разработчика при оптимизации запросов по тегам.

Подсказка: Индексация и анализ производительности идут после внедрения.

**Ответ:**

- Реализовать хранение тегов
- Добавить индексы (GIN/GiST)
- Запустить нагрузочное тестирование
- Оптимизировать запросы

## Шаг 15:

### Миссия 15

Упорядочьте шаги по миграции от массивов к отдельной таблице тегов.

Подсказка: Важно сначала спроектировать таблицу, а потом переносить данные.

**Ответ:**

- Создать новую таблицу tags
- Создать промежуточную таблицу связи
- Перенести данные из массивов
- Удалить старую колонку массива

## Шаг 16:

### Миссия 16

Расположите действия по настройке фильтров в магазине.

Подсказка: Сначала проектируем, потом индексируем и тестируем.

**Ответ:**

- Спроектировать структуру хранения фильтров
- Добавить индексы для поиска
- Протестировать запросы
- Внедрить в продакшн

 **Шаг 17:**

**Миссия 17**

Расположите этапы работы над мини-боссом: системой тегов и фильтров.

Подсказка: От идеи до оптимизации.

**Ответ:**

- Выбрать подходящую структуру хранения
- Реализовать базовый функционал
- Добавить индексы
- Провести нагрузочное тестирование и оптимизацию

## **Модуль 3 - Оптимизация запросов**

### **Урок 1 - План выполнения (EXPLAIN, ANALYZE)**

 **Шаг 01:**

**Миссия 1**

Ты изучаешь план выполнения с помощью EXPLAIN. Что он показывает?

Подсказка: Это не результат запроса, а его схема действий.

**Ответ:**

- **Последовательность шагов, которые выполняет оптимизатор**
- Фактические строки результата
- Ошибки синтаксиса SQL
- Состояние транзакции

 **Шаг 02:**

**Миссия 2**

Программистка Лена запустила EXPLAIN ANALYZE. Чем он отличается от обычного EXPLAIN?

Подсказка: Один только прогнозирует, другой проверяет в бою.

**Ответ:**

- **EXPLAIN ANALYZE выполняет запрос и показывает фактические показатели**
- EXPLAIN ANALYZE работает быстрее, чем EXPLAIN
- EXPLAIN ANALYZE показывает только ошибки
- EXPLAIN ANALYZE доступен только админам

 **Шаг 03:**

**Миссия 3**

Разработчик Дима заметил в плане Seq Scan. Что это означает?

Подсказка: Иногда это дешевле индекса, особенно для маленьких таблиц.

**Ответ:**

- **Таблица читается полностью построчно**
- Используется B-tree индекс
- Применяется параллельный join
- Запрос не оптимизирован

#### Шаг 04:

##### **Миссия 4**

Тимур удивился строке Index Scan. Что это значит?

Подсказка: Это похоже на чтение по оглавлению книги, а не по страницам подряд.

**Ответ:**

- **Поиск строк осуществляется через индекс**
- Все данные сортируются
- Запрос выполнен в кэше
- Таблица была пересоздана

#### Шаг 05:

##### **Миссия 5**

Маша увидела в плане Nested Loop. Как лучше всего описать это действие?

Подсказка: Вложенный цикл — это как двойная проверка списка гостей.

**Ответ:**

- **Для каждой строки из первой таблицы ищутся совпадения во второй**
- Сначала сортируются обе таблицы
- Выполняется поблочный merge
- Применяется только к индексам

#### Шаг 06:

##### **Миссия 6**

Олег заметил в плане Hash Join. Что это значит?

Подсказка: Тут строится временная структура для ускорения поиска.

**Ответ:**

- **Одна таблица хэшируется для быстрого соединения с другой**
- Данные соединяются без фильтрации
- Все строки сортируются перед соединением
- Выполняется параллельное чтение

#### Шаг 07:

##### **Миссия 7**

 Сопоставь элементы плана с их описанием.

Подсказка: Внимательно смотри на ключевые слова: Seq, Index, Hash.

**Ответ:**

- **Seq Scan** — Полный проход по таблице
- **Index Scan** — Поиск строк через индекс

- **Hash Join** — Соединение с помощью хэш-таблицы
- **Nested Loop** — Соединение построчно с вложенными циклами

## 🔮 Шаг 08:

### Миссия 8

🧑💻 Программист проверяет разницу между планами. Сопоставь ключевые команды с их назначением.

Подсказка: ANALYZE — это практика, EXPLAIN — теория.

**Ответ:**

- **EXPLAIN** — Показывает план без выполнения
- **EXPLAIN ANALYZE** — Выполняет и показывает реальные результаты
- **COST** — Оценка затраченных ресурсов
- **ROWS** — Оценка количества строк

## 🔮 Шаг 09:

### Миссия 9

✳ Сопоставь термины с описаниями из плана выполнения.

Подсказка: Фокусируйся на времени и количестве строк.

**Ответ:**

- **Actual Time** — Фактическое время выполнения операции
- **Rows Removed by Filter** — Количество строк, отфильтрованных условием
- **Loops** — Сколько раз операция повторялась
- **Startup Cost** — Цена начала операции

## 🔮 Шаг 10:

### Миссия 10

🔗 Сопоставь методы соединения таблиц с их кратким описанием.

Подсказка: Вспомни Nested Loop, Hash и Merge.

**Ответ:**

- **Nested Loop** — Для каждой строки из первой ищутся совпадения во второй
- **Hash Join** — Создаётся хэш-таблица для соединения
- **Merge Join** — Соединение двух отсортированных наборов
- **Seq Scan** — Полный просмотр таблицы

## 🔮 Шаг 11:

### Миссия 11

📊 Сопоставь параметры из EXPLAIN с их значением.

Подсказка: Rows и Width часто идут вместе.

**Ответ:**

- **Rows** — Оценка количества возвращаемых строк
- **Width** — Средний размер строки в байтах
- **Total Cost** — Общие предполагаемые затраты
- **Plan Rows** — Оценка строк для конкретного узла

## 🔮 Шаг 12:

### Миссия 12

📌 Расположи шаги анализа плана выполнения в правильном порядке.

Подсказка: Сначала читаем структуру, потом цифры, потом ищем узкие места.

#### Ответ:

- Понять структуру плана
- Сравнить прогнозные и фактические строки
- Посмотреть на стоимость операций
- Найти самую дорогую часть

## 🔮 Шаг 13:

### Миссия 13

🔍 Упорядочи действия программиста при оптимизации запроса.

Подсказка: Ошибки сначала ищут глазами, потом проверяют индексами.

#### Ответ:

- Запустить EXPLAIN ANALYZE
- Определить, есть ли Seq Scan
- Рассмотреть возможность индексации
- Сравнить время выполнения после оптимизации

## 🔮 Шаг 14:

### Миссия 14

⚙️ Расположи по порядку шаги работы оптимизатора PostgreSQL.

Подсказка: Сначала анализ запроса, потом варианты планов, потом выбор лучшего.

#### Ответ:

- Парсинг SQL-запроса
- Генерация возможных планов
- Расчёт стоимости планов
- Выбор оптимального плана

## 🔮 Шаг 15:

### Миссия 15

🔑 Расставь шаги работы с EXPLAIN от простого к продвинутому.

Подсказка: Сначала учимся читать план, потом работать с анализом.

#### Ответ:

- EXPLAIN
- EXPLAIN (ANALYZE)
- EXPLAIN (ANALYZE, BUFFERS)
- EXPLAIN (ANALYZE, VERBOSE)

## 🔮 Шаг 16:

### Миссия 16

🔧 Упорядочи действия при устранении медленного запроса.

Подсказка: В конце проверка результата — обязательна.

**Ответ:**

- Изучить план с EXPLAIN
- Добавить недостающий индекс
- Сравнить до и после с ANALYZE
- Проверить результат на реальных данных

## **Урок 2 - Индексы когда они реально помогают, а когда — мешают**

### **Шаг 01:**

#### **Миссия 1**

Когда индекс реально помогает ускорить запрос?

Подсказка: Смотрите на условия WHERE и порядок сортировки.

**Ответ:**

- **Когда используется фильтр по колонке, по которой есть индекс**
- Когда таблица пустая
- Когда SELECT \* возвращает все колонки
- Когда транзакция откатывается

### **Шаг 02:**

#### **Миссия 2**

Программист Артем заметил медленный запрос с фильтром по текстовой колонке. Что стоит проверить?

Подсказка: Иногда индекс не используется, если колонка не поддерживает быстрый поиск.

**Ответ:**

- **Есть ли подходящий индекс для этой колонки**
- Сколько строк в другой таблице
- Используется ли триггер на колонке
- Есть ли внешние ключи в этой таблице

### **Шаг 03:**

#### **Миссия 3**

Маша создала индекс на колонку, по которой почти все строки уникальны. Что может произойти?

Подсказка: Индекс работает эффективно, когда выборка маленькая.

**Ответ:**

- **Индекс ускорит запросы с фильтром**
- Индекс замедлит чтение всех строк
- Индекс удалит дубликаты
- Индекс уменьшит размер таблицы

### **Шаг 04:**

#### **Миссия 4**

Дима заметил медленный INSERT. Что может быть причиной?

Подсказка: Индексы ускоряют SELECT, но влияют на запись.

**Ответ:**

- **Слишком много индексов замедляет вставку**
- Индекс ускоряет INSERT всегда
- Индекс меняет тип колонки
- Индекс блокирует транзакцию

#### Шаг 05:

##### **Миссия 5**

Тимур думает, что индекс на колонку с часто изменяемыми значениями поможет ускорить запрос. Что скажешь?

Подсказка: Подумай о затратах на поддержание индекса.

**Ответ:**

- **Индекс может замедлить вставки и обновления**
- Индекс всегда ускоряет выборку
- Индекс увеличивает RAM только при SELECT
- Индекс удаляет дубликаты автоматически

#### Шаг 06:

##### **Миссия 6**

Юля заметила, что индекс не используется при LIKE '%text%'. Почему?

Подсказка: Смотрите на начало строки при использовании индекса B-tree.

**Ответ:**

- **B-tree индекс не может ускорить поиск с ведущим wildcard**
- Индекс повреждён
- LIKE всегда использует Seq Scan
- PostgreSQL игнорирует индексы по умолчанию

#### Шаг 07:

##### **Миссия 7**

Маша хочет ускорить JOIN двух больших таблиц. Что стоит проверить?

Подсказка: Индексы помогают соединять, если они есть на ключах.

**Ответ:**

- **Есть ли индексы на колонках, участвующих в JOIN**
- Все строки обеих таблиц равны
- Таблицы имеют одинаковое количество строк
- JOIN используется только в подзапросах

#### Шаг 08:

##### **Миссия 8**

 Сопоставьте тип индекса с его преимуществом.

Подсказка: Подумайте о разных структурах: B-tree, Hash, GiST.

**Ответ:**

- **B-tree** — Хорош для точного поиска и диапазонных запросов
- **Hash** — Быстрый точный поиск по равенству

- **GiST** — Поддержка полнотекстового поиска и пространственных данных
- **GIN** — Эффективен для массивов и JSONB

## 🔧 Шаг 09:

### Миссия 9

Программист Никита заметил, что запрос с фильтром по NULL медленный. Как индекс может помочь?

Подсказка: Некоторые типы индексов игнорируют NULL по умолчанию.

**Ответ:**

- **Создать частичный индекс, исключаящий NULL**
- Создать уникальный индекс
- Удалить все NULL из таблицы
- Перенести колонку в отдельную таблицу

## 🔧 Шаг 10:

### Миссия 10

📄 Сопоставьте индексы с ситуациями, где они будут полезны.

Подсказка: Подумайте, где SELECT выигрывает от структуры индекса.

**Ответ:**

- **B-tree на колонке с диапазоном дат** — Ускоряет фильтрацию по интервалам
- **GIN на массиве тегов** — Быстро ищет элементы массива
- **Hash на колонке с равенством** — Эффективен для точного поиска
- **GiST на геоданных** — Поддерживает пространственные запросы

## 🔧 Шаг 11:

### Миссия 11

🧠 Сопоставьте проблему с её возможным решением через индексы.

Подсказка: Иногда правильный тип индекса решает проблему с производительностью.

**Ответ:**

- **Медленный поиск по JSONB** — Использовать GIN индекс
- **Медленный диапазон дат** — Использовать B-tree индекс
- **Частые равенства по числовой колонке** — Использовать Hash индекс
- **Поиск по геопозиции** — Использовать GiST индекс

## 🔧 Шаг 12:

### Миссия 12

Расположите шаги для оптимизации запроса с медленным JOIN с использованием индексов:

1. Проверка плана выполнения через EXPLAIN
2. Добавление индекса на колонку для JOIN
3. Перезапуск запроса и проверка времени выполнения
4. Анализ результатов и возможная корректировка индекса

Подсказка: Сначала изучаем план, потом добавляем индекс, проверяем результат, при необходимости улучшаем.

**Ответ:**

- Проверка плана выполнения через EXPLAIN
- Добавление индекса на колонку для JOIN
- Перезапуск запроса и проверка времени выполнения
- Анализ результатов и возможная корректировка индекса

 **Шаг 13:**

**Миссия 13**

Расположите этапы создания частичного индекса для ускорения выборки без NULL:

1. Выбор колонок и условия WHERE для индекса
2. Создание индекса с USING и WHERE условием
3. Тестирование SELECT с условием, включенным в индекс
4. Анализ EXPLAIN, убедиться, что индекс используется

Подсказка: Частичный индекс создается только для строк, подходящих под условие.

**Ответ:**

- Выбор колонок и условия WHERE для индекса
- Создание индекса с USING и WHERE условием
- Тестирование SELECT с условием, включенным в индекс
- Анализ EXPLAIN, убедиться, что индекс используется

 **Шаг 14:**

**Миссия 14**

Расположите действия DBA при проблемах с производительностью после добавления индекса:

1. Проверка размера и структуры индекса
2. Сравнение времени выполнения запросов с EXPLAIN ANALYZE
3. Оптимизация или удаление неэффективного индекса
4. Повторное тестирование запросов

Подсказка: Индекс может помочь, но иногда его нужно настроить или удалить.

**Ответ:**

- Проверка размера и структуры индекса
- Сравнение времени выполнения запросов с EXPLAIN ANALYZE
- Оптимизация или удаление неэффективного индекса
- Повторное тестирование запросов

 **Шаг 15:**

**Миссия 15**

Расположите шаги при выборе подходящего типа индекса для разных запросов:

1. Анализ типов запросов: диапазоны, равенство, полнотекст
2. Сопоставление типа индекса с типом запроса
3. Создание индекса соответствующего типа
4. Тестирование запросов и корректировка при необходимости

Подсказка: B-tree для диапазонов, Hash для равенства, GIN/GiST для сложных данных.

**Ответ:**

- Анализ типов запросов: диапазоны, равенство, полнотекст
- Сопоставление типа индекса с типом запроса
- Создание индекса соответствующего типа
- Тестирование запросов и корректировка при необходимости

 **Шаг 16:**

**Миссия 16**

Расположите шаги полного анализа производительности запросов с индексами:

1. Запуск EXPLAIN ANALYZE на проблемных запросах
2. Определение медленных операций и возможных узких мест
3. Добавление/удаление/изменение индексов
4. Повторное тестирование и финальная проверка

Подсказка: Анализ должен быть цикличным: планирование, изменение, проверка.

**Ответ:**

- Запуск EXPLAIN ANALYZE на проблемных запросах
- Определение медленных операций и возможных узких мест
- Добавление/удаление/изменение индексов
- Повторное тестирование и финальная проверка

 **Урок 3 - Мультиколоночные, частичные и выраженные индексы**

 **Шаг 01:**

**Миссия 1**

Разработчик Оля хочет ускорить поиск по двум колонкам одновременно. Какой индекс ей стоит использовать?

Подсказка: Для одновременного фильтра по нескольким колонкам существует специальный тип индекса.

**Ответ:**

- **Мультиколоночный B-tree индекс**
- Hash индекс на первой колонке
- Частичный индекс на одной колонке
- GIN индекс на текстовом поле

 **Шаг 02:**

**Миссия 2**

В базе есть колонка status с большим количеством NULL. Как индекс поможет ускорить выборку только по непустым значениям?

Подсказка: Иногда имеет смысл индексировать только часть таблицы.

**Ответ:**

- **Создать частичный индекс с условием WHERE status IS NOT NULL**
- Создать стандартный B-tree на всей колонке
- Удалить NULL из таблицы
- Использовать текстовый GIN индекс

## Шаг 03:

### Миссия 3

Программист Кирилл хочет индексировать вычисляемое выражение `price * quantity` для ускорения аналитических запросов. Какой тип индекса подойдет?

Подсказка: PostgreSQL позволяет индексировать результат выражения, а не только колонки.

**Ответ:**

- **Выраженный (expression) индекс на `price * quantity`**
- Hash индекс на `price`
- B-tree индекс на `quantity`
- GIN индекс на `price` и `quantity`

## Шаг 04:

### Миссия 4

 Сопоставьте тип индекса с его подходящей ситуацией использования.

Подсказка: Подумайте, где выраженный, мультиколоночный или частичный индекс может реально ускорить запросы.

**Ответ:**

- **Мультиколоночный индекс** — Фильтрация и сортировка по двум колонкам одновременно
- **Частичный индекс** — Выборка только по строкам, удовлетворяющим условию WHERE
- **Выраженный индекс** — Индекс на вычисляемое выражение, например `price*quantity`
- **GIN индекс** — Поиск по массивам или JSONB

## Шаг 05:

### Миссия 5

 Сопоставьте преимущества частичных индексов с описанием сценария.

Подсказка: Частичный индекс не индексирует всю таблицу.

**Ответ:**

- **Снижение размера индекса** — Индекс создается только для строк, подходящих под условие WHERE
- **Ускорение выборки** — Запросы с условием совпадают с условием частичного индекса
- **Снижение затрат на обновление** — Неиндексированные строки не требуют поддержания индекса

## Шаг 06:

### Миссия 6

 Сопоставьте сценарий с типом индекса.

Подсказка: Смотрите на структуру данных и условия запроса.

**Ответ:**

- **Поиск по массиву тегов** — GIN индекс
- **Выборка по диапазону дат и статусу** — Мультиколоночный B-tree
- **Анализ сумм заказов** — Выраженный индекс на выражение
- **Выборка только активных клиентов** — Частичный индекс с `WHERE active = true`

## Шаг 07:

### Миссия 7

Расположите шаги создания частичного индекса:

1. Выбор колонок и условия WHERE
2. Создание индекса с USING и условием WHERE
3. Запуск запроса и проверка ускорения
4. Использование EXPLAIN, чтобы убедиться в применении индекса

Подсказка: Частичный индекс создается только для строк, удовлетворяющих условию.

**Ответ:**

- Выбор колонок и условия WHERE
- Создание индекса с USING и условием WHERE
- Запуск запроса и проверка ускорения
- Использование EXPLAIN, чтобы убедиться в применении индекса

## Шаг 08:

### Миссия 8

Разработчик Аня создает индекс на колонку email, чтобы ускорить поиск пользователей. Но запрос часто фильтрует только по домену (например, '@gmail.com'). Как лучше сделать индекс?

Подсказка: Иногда выгоднее индексировать не всю колонку, а выражение.

**Ответ:**

- **Выраженный индекс на выражение SUBSTRING(email FROM POSITION('@' IN email))**
- Обычный B-tree индекс на email
- Hash индекс на email
- GIN индекс на email

## Шаг 09:

### Миссия 9

Саша создает мультиколоночный индекс на (category, price), но заметил, что запрос SELECT \* FROM products WHERE price > 100 не использует индекс. Почему?

Подсказка: Порядок колонок в индексе влияет на его применение.

**Ответ:**

- **Индекс начинается с category, поэтому условия только на price не используют его полностью**
- Индекс слишком большой для таблицы
- PostgreSQL не поддерживает мультиколоночные индексы
- Нужно пересоздать индекс как GIN

## Шаг 10:

### Миссия 10

 Сопоставьте ситуацию с подходящим типом индекса.

Подсказка: Подумайте, что лучше всего ускорит конкретный сценарий.

**Ответ:**

- **Поиск заказов за последний месяц** — Частичный индекс с WHERE order\_date > current\_date - interval '1 month'

- Поиск по JSONB колонке **tags** — GIN индекс
- Сортировка товаров по (**category, price**) — Мультиколоночный B-tree индекс
- Поиск по вычисляемой сумме **total = price\*quantity** — Выраженный индекс

## 🔧 Шаг 11:

### Миссия 11

📖 Сопоставьте проблему с оптимальным решением с помощью индекса.

Подсказка: Иногда частичный или выраженный индекс решает проблему лучше, чем обычный B-tree.

**Ответ:**

- Слишком много **NULL** в колонке **status** — Частичный индекс WHERE status IS NOT NULL
- Фильтрация по комбинации двух колонок — Мультиколоночный индекс
- Поиск по вычисляемой скидке **price\*0.9** — Выраженный индекс
- Поиск по массиву тегов — GIN индекс

## 🔧 Шаг 12:

### Миссия 12

Расположите шаги для создания мультиколоночного индекса:

1. Определить колонки, используемые вместе в WHERE и ORDER BY
2. Создать индекс с использованием этих колонок
3. Протестировать запрос с EXPLAIN
4. При необходимости перестроить индекс с правильным порядком колонок

Подсказка: Порядок колонок влияет на то, какие запросы смогут использовать индекс.

**Ответ:**

- Определить колонки, используемые вместе в WHERE и ORDER BY
- Создать индекс с использованием этих колонок
- Протестировать запрос с EXPLAIN
- При необходимости перестроить индекс с правильным порядком колонок

## 🔧 Шаг 13:

### Миссия 13

Расположите шаги для использования выраженного индекса:

1. Определить выражение для ускорения запросов
2. Создать индекс на это выражение
3. Переписать запрос, чтобы использовать выражение
4. Проверить через EXPLAIN, что индекс применяется

Подсказка: Сначала создаем индекс, потом проверяем его работу через EXPLAIN.

**Ответ:**

- Определить выражение для ускорения запросов
- Создать индекс на это выражение
- Переписать запрос, чтобы использовать выражение
- Проверить через EXPLAIN, что индекс применяется

## Шаг 14:

### Миссия 14

Инженер Максим хочет ускорить выборку только для активных пользователей в большой таблице. Что ему выбрать?

Подсказка: Частичный индекс идеально подходит для фильтрации небольшой части таблицы.

**Ответ:**

- **Частичный индекс с WHERE active = true**
- Обычный B-tree индекс на всех пользователях
- GIN индекс на колонку active
- Hash индекс на колонку active

## Шаг 15:

### Миссия 15

 Сопоставьте вид индекса с характерной особенностью.

Подсказка: Каждому индексу соответствует уникальная задача ускорения запросов.

**Ответ:**

- **Мультиколоночный индекс** — Оптимизация фильтрации и сортировки по нескольким колонкам
- **Выраженный индекс** — Индекс на вычисляемое выражение
- **Частичный индекс** — Индекс только для части таблицы, соответствующей условию WHERE
- **GIN индекс** — Поиск по массивам, JSONB или текстовым данным с полнотекстовым поиском

## Шаг 16:

### Миссия 16

Расположите шаги оптимизации запросов с использованием различных индексов:

1. Проанализировать запрос с EXPLAIN
2. Выбрать подходящий тип индекса (мультиколоночный, частичный, выраженный, GIN)
3. Создать индекс в базе данных
4. Повторно проверить запрос с EXPLAIN, убедившись в использовании индекса

Подсказка: Сначала анализируем, потом создаем индекс, затем проверяем результат.

**Ответ:**

- Проанализировать запрос с EXPLAIN
- Выбрать подходящий тип индекса (мультиколоночный, частичный, выраженный, GIN)
- Создать индекс в базе данных
- Повторно проверить запрос с EXPLAIN, убедившись в использовании индекса

## Урок 4 - Оценка стоимости запроса и рефакторинг

## Шаг 01:

### Миссия 1

Разработчик Анна хочет понять, какой запрос выполняется быстрее. Какой инструмент PostgreSQL поможет ей оценить стоимость запроса?

Подсказка: Подумайте, какой встроенный инструмент показывает план выполнения и стоимость.

**Ответ:**

- **EXPLAIN**
- ANALYZE COST
- SHOW PLAN
- SELECT COST

🔮 **Шаг 02:**

### **Миссия 2**

Игорь замечает, что его запрос с JOIN выполняется долго. Какое действие может помочь снизить стоимость запроса?

Подсказка: Иногда помогает изменение структуры запроса или индекса.

**Ответ:**

- **Создать индекс на колонки, участвующие в JOIN**
- Добавить больше столбцов в SELECT \*
- Удалить WHERE
- Использовать подзапрос вместо JOIN без индекса

🔮 **Шаг 03:**

### **Миссия 3**

Мария хочет сравнить два плана выполнения одного запроса: с индексом и без. Какой режим EXPLAIN лучше использовать?

Подсказка: Есть режим, который выполняет запрос и показывает реальные времена.

**Ответ:**

- **EXPLAIN ANALYZE**
- EXPLAIN COST
- EXPLAIN VERBOSE
- EXPLAIN TIME

🔮 **Шаг 04:**

### **Миссия 4**

Даниил пишет SELECT с множеством OR условий и видит, что стоимость растет. Что можно сделать для снижения стоимости?

Подсказка: Иногда разбиение на несколько запросов или использование индекса помогает.

**Ответ:**

- **Разбить запрос на UNION или добавить индекс на колонки фильтрации**
- Добавить ORDER BY
- Использовать SELECT \*
- Удалить WHERE

🔮 **Шаг 05:**

### **Миссия 5**

📖 Сопоставьте ситуацию с оптимальным способом снижения стоимости запроса.

Подсказка: Каждая ситуация требует своей техники оптимизации.

**Ответ:**

- **Большая таблица без индексов** — Добавить индекс на колонку, участвующую в фильтре
- **JOIN на колонках с низкой селективностью** — Использовать HASH JOIN
- **Запрос с множеством OR** — Разбить на UNION или добавить индексы
- **Агрегация по небольшой группе значений** — Создать частичный индекс на ключевые значения

## Шаг 06:

### Миссия 6

 Сопоставьте понятие с его объяснением.

Подсказка: Различайте план выполнения и реальный результат запроса.

**Ответ:**

- **EXPLAIN** — Показывает план выполнения запроса и оценочную стоимость
- **EXPLAIN ANALYZE** — Выполняет запрос и показывает реальные времена и использование индексов
- **SEQ SCAN** — Полное сканирование таблицы
- **INDEX SCAN** — Использование индекса для выборки данных

## Шаг 07:

### Миссия 7

 Сопоставьте действие с эффектом на стоимость запроса.

Подсказка: Оптимизация может снизить или увеличить стоимость, если сделать неправильно.

**Ответ:**

- **Добавление индекса на часто используемую колонку** — Снижение стоимости и ускорение выборки
- **Добавление индекса на редко используемую колонку** — Минимальное влияние, иногда рост стоимости
- **Удаление ненужных JOIN** — Снижение стоимости
- **Использование подзапроса вместо JOIN** — Может увеличить стоимость при больших данных

## Шаг 08:

### Миссия 8

 Сопоставьте вид сканирования с описанием.

Подсказка: Разные сканирования влияют на стоимость запроса по-разному.

**Ответ:**

- **SEQ SCAN** — Сканирование всей таблицы
- **INDEX SCAN** — Использование индекса для выборки нужных строк
- **BITMAP INDEX SCAN** — Комбинирование нескольких индексов для выборки
- **JOIN HASH** — Использование хэш-таблицы для соединения

## Шаг 09:

### Миссия 9

Расположите шаги рефакторинга запроса в правильном порядке для снижения стоимости:

1. Проанализировать EXPLAIN и выявить узкие места
2. Добавить индексы на фильтруемые колонки
3. Переписать JOIN или WHERE для оптимизации
4. Проверить новый план выполнения и время запроса

Подсказка: Сначала анализируем, затем оптимизируем и проверяем результат.

**Ответ:**

- Проанализировать EXPLAIN и выявить узкие места
- Добавить индексы на фильтруемые колонки
- Переписать JOIN или WHERE для оптимизации
- Проверить новый план выполнения и время запроса

## Шаг 10:

### Миссия 10

Расположите шаги для оценки стоимости сложного запроса:

1. Выполнить EXPLAIN ANALYZE
2. Изучить время выполнения и использование индексов
3. Определить проблемные места с высокой стоимостью
4. Применить оптимизации (индексы, рефакторинг запросов)

Подсказка: Результаты EXPLAIN ANALYZE помогают найти узкие места.

**Ответ:**

- Выполнить EXPLAIN ANALYZE
- Изучить время выполнения и использование индексов
- Определить проблемные места с высокой стоимостью
- Применить оптимизации (индексы, рефакторинг запросов)

## Шаг 11:

### Миссия 11

Костя замечает, что запрос с агрегацией выполняется долго. Что лучше сделать для снижения стоимости?

Подсказка: Иногда частичный индекс или материализованная таблица помогают.

**Ответ:**

- **Создать материализованное представление или частичный индекс**
- Добавить больше столбцов в SELECT \*
- Удалить WHERE
- Использовать COUNT(\*) без фильтра

## Шаг 12:

### Миссия 12

Сергей хочет оценить, как изменение JOIN повлияет на стоимость запроса. Что использовать?

Подсказка: Режим EXPLAIN ANALYZE покажет реальное время и использование индексов.

**Ответ:**

- **EXPLAIN ANALYZE перед и после изменений**

- SHOW COST
- EXPLAIN TIME
- SELECT COUNT(\*)

### Шаг 13:

#### Миссия 13

 Сопоставьте тип рефакторинга с примером оптимизации запроса.

Подсказка: Разные подходы снижают стоимость по-разному.

**Ответ:**

- **Добавление индексов** — Снижение времени выборки по WHERE и JOIN
- **Рефакторинг JOIN** — Уменьшение числа соединений и упрощение условий
- **Материализованное представление** — Снижение стоимости сложных агрегаций
- **Использование частичного индекса** — Оптимизация выборки для небольшой части таблицы

### Шаг 14:

#### Миссия 14

Расположите шаги при рефакторинге запроса для снижения стоимости:

1. Проанализировать план с EXPLAIN ANALYZE
2. Определить узкие места (сканирование таблиц, JOIN, агрегации)
3. Применить индексы или материализованные представления
4. Переписать запрос для более эффективного выполнения
5. Проверить новый план и реальные времена выполнения

Подсказка: Оптимизация — это итеративный процесс: сначала анализ, затем улучшения и проверка.

**Ответ:**

- Проанализировать план с EXPLAIN ANALYZE
- Определить узкие места (сканирование таблиц, JOIN, агрегации)
- Применить индексы или материализованные представления
- Переписать запрос для более эффективного выполнения
- Проверить новый план и реальные времена выполнения

### Шаг 15:

#### Миссия 15

Расположите шаги оптимизации запроса с агрегацией:

1. Определить, какие колонки участвуют в группировке и фильтрах
2. Создать частичный индекс или материализованное представление
3. Переписать запрос, используя предварительно агрегированные данные
4. Проверить EXPLAIN ANALYZE для оценки стоимости

Подсказка: Иногда лучше готовить данные заранее, чтобы запрос выполнялся быстрее.

**Ответ:**

- Определить, какие колонки участвуют в группировке и фильтрах
- Создать частичный индекс или материализованное представление
- Переписать запрос, используя предварительно агрегированные данные

- Проверить EXPLAIN ANALYZE для оценки стоимости

#### Шаг 16:

##### Миссия 16

Анна видит, что индекс не используется при выполнении запроса. Что может быть причиной?

Подсказка: PostgreSQL выбирает план выполнения автоматически, не всегда используя индекс.

**Ответ:**

- **Планировщик считает, что последовательное сканирование быстрее**
- Индекс автоматически не создается
- EXPLAIN запрещает использование индекса
- Индекс удален автоматически

#### Шаг 17:

##### Миссия 17

Игорь хочет понять, какой тип индекса лучше использовать для текстовой колонки с LIKE '%текст%'. Что выбрать?

Подсказка: Стандартный B-tree неэффективен для поиска с wildcard в начале строки.

**Ответ:**

- **GIN или GiST индекс**
- B-tree индекс
- HASH индекс
- Нет индекса

### Урок 5 - Мини-босс ускорить медленные запросы из реального проекта

#### Шаг 01:

##### Миссия 1

Какая из стратегий является первой при диагностике медленного запроса?

Подсказка: Сначала нужно понять, что именно тормозит запрос.

**Ответ:**

- **Использовать EXPLAIN или EXPLAIN ANALYZE**
- Сразу создавать новые индексы
- Переписывать весь запрос с нуля
- Удалять все JOIN без анализа

#### Шаг 02:

##### Миссия 2

Когда стоит использовать частичный индекс?

Подсказка: Частичный индекс помогает не всем строкам, а только тем, что соответствуют условию.

**Ответ:**

- **Когда часто фильтруют подмножество строк по условию**
- Когда таблица маленькая
- Когда все строки одинаково часто используются
- Когда нужен полный текстовый поиск

## 🔮 Шаг 03:

### Миссия 3

Алексей заметил медленный JOIN. Что поможет ускорить выполнение?

Подсказка: Обратите внимание на колонки, по которым соединяются таблицы.

**Ответ:**

- **Создать индекс на колонках JOIN**
- Увеличить RAM на сервере без анализа запроса
- Удалить все условия WHERE
- Переписать запрос на другой SQL-диалект

## 🔮 Шаг 04:

### Миссия 4

Сопоставьте тип индекса и его использование в ускорении запросов.

Понятия:

**Ответ:**

- B-tree индекс
- GIN индекс
- GiST индекс
- Частичный индекс

Определения:

1. Оптимален для поиска по точным значениям и диапазонам
2. Лучше всего подходит для полнотекстового поиска
3. Используется для геометрических и комплексных типов данных
4. Индексируется только часть таблицы по условию

Подсказка: Подумайте, какой тип данных и фильтрация чаще всего встречаются в таблице.

- **B-tree индекс** — Оптимален для поиска по точным значениям и диапазонам
- **GIN индекс** — Лучше всего подходит для полнотекстового поиска
- **GiST индекс** — Используется для геометрических и комплексных типов данных
- **Частичный индекс** — Индексируется только часть таблицы по условию

## 🔮 Шаг 05:

### Миссия 5

Сопоставьте причину медленного запроса и метод оптимизации.

Понятия:

**Ответ:**

- Большая таблица с миллионами строк
- Фильтрация по редко встречающемуся значению
- Частый full scan по текстовому полю
- JOIN без индекса на колонке

Определения:

1. Создание индекса по колонке JOIN
2. Использовать частичный индекс на редкие значения

3. Использовать GIN индекс для текстового поля
4. Разбить таблицу на партиции или использовать LIMIT/OFFSET

Подсказка: Смотрите на тип запроса и распределение данных.

- **Большая таблица с миллионами строк** — Разбить таблицу на партиции или использовать LIMIT/OFFSET
- **Фильтрация по редко встречающемуся значению** — Использовать частичный индекс на редкие значения
- **Частый full scan по текстовому полю** — Использовать GIN индекс для текстового поля
- **JOIN без индекса на колонке** — Создание индекса по колонке JOIN

## Шаг 06:

### Миссия 6

Расположите шаги ускорения медленного запроса с агрегацией:

1. Проанализировать EXPLAIN ANALYZE
2. Определить узкие места (сканирование, JOIN, GROUP BY)
3. Создать индексы или материализованные представления
4. Переписать запрос для лучшей эффективности
5. Проверить результаты на тестовых данных

Подсказка: Оптимизация всегда начинается с анализа и проверки.

**Ответ:**

- Проанализировать EXPLAIN ANALYZE
- Определить узкие места (сканирование, JOIN, GROUP BY)
- Создать индексы или материализованные представления
- Переписать запрос для лучшей эффективности
- Проверить результаты на тестовых данных

## Шаг 07:

### Миссия 7

Когда использование индекса может замедлить запрос?

Подсказка: Не всегда индекс полезен для маленькой таблицы или выборки почти всех строк.

**Ответ:**

- **Когда таблица маленькая или выбираются почти все строки**
- Когда таблица большая
- Когда индекс B-tree
- Когда используется EXPLAIN ANALYZE

## Шаг 08:

### Миссия 8

Какой подход поможет ускорить запрос с большим количеством JOIN?

Подсказка: Смотрите на индексацию колонок, по которым соединяются таблицы.

**Ответ:**

- **Создать индексы на колонках JOIN**
- Удалить все JOIN
- Переписать запрос на другой SQL-диалект

- Добавить случайные фильтры WHERE

## 🔮 Шаг 09:

### Миссия 9

Сопоставьте тип оптимизации и её эффект.

Понятия:

**Ответ:**

- Материализованное представление
- Использование индекса
- Разбиение таблицы на партиции
- Переписывание запроса

Определения:

1. Позволяет хранить предвычисленные результаты для быстрого доступа
2. Ускоряет фильтрацию и поиск по ключевым колонкам
3. Снижает объем данных, которые нужно сканировать
4. Оптимизирует логические операции и уменьшает нагрузку на сервер

Подсказка: Подумайте, какие действия реально уменьшают время выполнения запроса.

- **Материализованное представление** — Позволяет хранить предвычисленные результаты для быстрого доступа
- **Использование индекса** — Ускоряет фильтрацию и поиск по ключевым колонкам
- **Разбиение таблицы на партиции** — Снижает объем данных, которые нужно сканировать
- **Переписывание запроса** — Оптимизирует логические операции и уменьшает нагрузку на сервер

## 🔮 Шаг 10:

### Миссия 10

Какой тип индекса лучше всего использовать для полнотекстового поиска в PostgreSQL?

Подсказка: Этот индекс оптимизирован именно для текстовых данных.

**Ответ:**

- **GIN индекс**
- B-tree индекс
- GiST индекс
- Частичный индекс

## 🔮 Шаг 11:

### Миссия 11

Сопоставьте способ оптимизации и тип запроса.

Понятия:

**Ответ:**

- Партиционирование
- Частичный индекс
- Материализованное представление
- Оптимизация JOIN

Определения:

1. Для больших таблиц с историческими данными
2. Для выборки только подмножества строк по условию
3. Для агрегационных запросов, где часто повторяется группировка
4. Для ускорения соединения нескольких таблиц по ключам

Подсказка: Подумайте, какой метод применим к конкретному сценарию использования данных.

- **Партиционирование** — Для больших таблиц с историческими данными
- **Частичный индекс** — Для выборки только подмножества строк по условию
- **Материализованное представление** — Для агрегационных запросов, где часто повторяется группировка
- **Оптимизация JOIN** — Для ускорения соединения нескольких таблиц по ключам

## 🔮 Шаг 12:

### Миссия 12

Расположите действия для ускорения медленного запроса с WHERE и JOIN:

1. Проанализировать EXPLAIN
2. Добавить индексы на колонки в JOIN и WHERE
3. Переписать запрос для минимизации подзапросов
4. Проверить время выполнения на тестовой среде

Подсказка: Следуйте логике: анализ → оптимизация → проверка.

**Ответ:**

- Проанализировать EXPLAIN
- Добавить индексы на колонки в JOIN и WHERE
- Переписать запрос для минимизации подзапросов
- Проверить время выполнения на тестовой среде

## 🔮 Шаг 13:

### Миссия 13

Что важно учитывать при создании индексированных колонок для часто обновляемой таблицы?

Подсказка: Индексы ускоряют чтение, но могут замедлять запись.

**Ответ:**

- **Баланс между ускорением SELECT и замедлением INSERT/UPDATE**
- Создавать индекс на каждую колонку без анализа
- Игнорировать частоту обновления
- Использовать только B-tree независимо от типа данных

## 🔮 Шаг 14:

### Миссия 14

Сопоставьте тип запроса и подходящую оптимизацию.

Понятия:

**Ответ:**

- Запрос с частой фильтрацией по статусу
- Запрос с агрегацией за последние N дней

- Запрос с геометрическими данными
- Запрос с большим числом соединений

Определения:

1. Частичный индекс по колонке status
2. Материализованное представление с предвычисленными суммами
3. GiST индекс для пространственных данных
4. Индексация колонок JOIN или оптимизация порядка соединений

Подсказка: Разные типы запросов требуют разных подходов.

- **Запрос с частой фильтрацией по статусу** — Частичный индекс по колонке status
- **Запрос с агрегацией за последние N дней** — Материализованное представление с предвычисленными суммами
- **Запрос с геометрическими данными** — GiST индекс для пространственных данных
- **Запрос с большим числом соединений** — Индексация колонок JOIN или оптимизация порядка соединений

## **Модуль 4 - ↻ Реляционное программирование оконные функции**

### **Урок 1 - OVER(), PARTITION BY, ORDER BY**

#### **Шаг 01:**

##### **Миссия 1**

Что делает оконная функция в PostgreSQL?

Подсказка: Она вычисляет значения для строк с учётом соседних строк, но без объединения в группы.

**Ответ:**

- **Выполняет расчёт для каждой строки с учётом окна**
- Объединяет строки в одну группу
- Удаляет дубликаты из таблицы
- Меняет порядок строк в таблице без расчёта

#### **Шаг 02:**

##### **Миссия 2**

Что делает `PARTITION BY` в оконной функции?

Подсказка: Подумайте, как разделяются строки перед вычислением функции.

**Ответ:**

- **Разбивает строки на отдельные группы для вычисления**
- Сортирует строки без группировки
- Создаёт новый столбец с уникальными значениями
- Удаляет все строки без NULL

#### **Шаг 03:**

##### **Миссия 3**

Для чего используется `ORDER BY` внутри `OVER()`?

Подсказка: Она определяет последовательность строк для вычисления функции.

**Ответ:**

- **Устанавливает порядок строк внутри окна для расчёта**
- Сортирует всю таблицу глобально
- Удаляет строки с NULL
- Создаёт индекс на колонку

🔗 **Шаг 04:**

#### **Миссия 4**

Сопоставьте оконную функцию и её назначение.

Понятия:

**Ответ:**

- ROW\_NUMBER()
- RANK()
- DENSE\_RANK()
- SUM()

Определения:

1. Присваивает уникальный номер каждой строке в окне
2. Присваивает ранк с пропуском при равенстве значений
3. Присваивает ранк без пропусков при равенстве
4. Суммирует значения внутри окна

Подсказка: Обратите внимание на различия между уникальным порядком и одинаковыми значениями.

- **ROW\_NUMBER()** — Присваивает уникальный номер каждой строке в окне
- **RANK()** — Присваивает ранк с пропуском при равенстве значений
- **DENSE\_RANK()** — Присваивает ранк без пропусков при равенстве
- **SUM()** — Суммирует значения внутри окна

🔗 **Шаг 05:**

#### **Миссия 5**

Что произойдет, если не использовать `PARTITION BY`?

Подсказка: Подумайте, как функция будет считать значения без деления на группы.

**Ответ:**

- **Функция будет вычисляться на всей таблице как на одном окне**
- Произойдет ошибка синтаксиса
- Будет создана новая таблица
- Все строки будут удалены

🔗 **Шаг 06:**

#### **Миссия 6**

Сопоставьте тип оконной функции и пример использования.

Понятия:

**Ответ:**

- AVG()

- MAX()
- LEAD()
- LAG()

Определения:

1. Среднее значение внутри окна
2. Максимальное значение внутри окна
3. Следующее значение относительно текущей строки
4. Предыдущее значение относительно текущей строки

Подсказка: Подумайте о том, как использовать соседние строки.

- **AVG()** — Среднее значение внутри окна
- **MAX()** — Максимальное значение внутри окна
- **LEAD()** — Следующее значение относительно текущей строки
- **LAG()** — Предыдущее значение относительно текущей строки

## Шаг 07:

### Миссия 7

Что делает `FIRST\_VALUE()` в оконной функции?

Подсказка: Подумайте, какое значение берётся в начале окна.

**Ответ:**

- **Возвращает первое значение в пределах окна**
- Возвращает все значения в столбце
- Возвращает сумму значений окна
- Создаёт индекс на первую строку

## Шаг 08:

### Миссия 8

Как работает `LAST\_VALUE()` с `ORDER BY`?

Подсказка: Последнее значение окна определяется порядком строк.

**Ответ:**

- **Возвращает последнее значение в пределах окна с учётом сортировки**
- Возвращает все значения без сортировки
- Возвращает минимальное значение колонки
- Создаёт новую колонку с последовательностью

## Шаг 09:

### Миссия 9

Сопоставьте оконную функцию с примером применения.

Понятия:

**Ответ:**

- CUME\_DIST()
- PERCENT\_RANK()
- NTILE(n)
- LAG()

Определения:

1. Вычисляет кумулятивное распределение для строки
2. Возвращает относительный ранг в пределах окна от 0 до 1
3. Делит строки на n равных частей
4. Возвращает предыдущее значение относительно текущей строки

Подсказка: Подумайте о том, как функции помогают анализировать позиции и распределение данных.

- **CUME\_DIST()** — Вычисляет кумулятивное распределение для строки
- **PERCENT\_RANK()** — Возвращает относительный ранг в пределах окна от 0 до 1
- **NTILE(n)** — Делит строки на n равных частей
- **LAG()** — Возвращает предыдущее значение относительно текущей строки

## Шаг 10:

### Миссия 10

Сопоставьте оконную функцию и тип аналитики:

Понятия:

**Ответ:**

- SUM() OVER()
- AVG() OVER()
- RANK() OVER()
- LEAD() OVER()

Определения:

1. Накопительные суммы продаж
2. Средняя оценка студентов по группам
3. Определение ранга сотрудника по продажам
4. Следующее значение цены акции

Подсказка: Подумайте, какая функция подходит для числовой агрегатной аналитики, а какая — для сравнения позиций.

- **SUM() OVER()** — Накопительные суммы продаж
- **AVG() OVER()** — Средняя оценка студентов по группам
- **RANK() OVER()** — Определение ранга сотрудника по продажам
- **LEAD() OVER()** — Следующее значение цены акции

## Шаг 11:

### Миссия 11

Расположите шаги использования оконных функций для накопительной суммы:

1. Выберите колонку для суммирования
2. Определите порядок строк через ORDER BY
3. Примените SUM() с OVER()
4. Получите результаты для каждой строки

Подсказка: Следуйте логике: выбор данных → порядок → вычисление → результат.

**Ответ:**

- Выберите колонку для суммирования

- Определите порядок строк через ORDER BY
- Примените SUM() с OVER()
- Получите результаты для каждой строки

## 🔗 Шаг 12:

### Миссия 12

Для чего можно использовать `NTILE(4) OVER (ORDER BY score)``?

Подсказка: Подумайте о разбиении данных на равные части.

**Ответ:**

- **Разбить строки на 4 квантили по значению score**
- Посчитать среднее значение по колонке score
- Вернуть первые 4 строки таблицы
- Удалить строки с низкими значениями score

## 🔗 Шаг 13:

### Миссия 13

Сопоставьте оконную функцию и тип расчёта:

Понятия:

**Ответ:**

- `MAX() OVER(PARTITION BY dept ORDER BY salary)`
- `MIN() OVER(PARTITION BY dept)`
- `LAG(salary) OVER(PARTITION BY dept ORDER BY hire_date)`
- `LEAD(salary) OVER(PARTITION BY dept ORDER BY hire_date)`

Определения:

1. Максимальная зарплата в отделе по возрастанию даты найма
2. Минимальная зарплата в отделе
3. Предыдущая зарплата сотрудника по дате найма
4. Следующая зарплата сотрудника по дате найма

Подсказка: Обратите внимание на PARTITION BY и ORDER BY для разных типов анализа.

- **`MAX() OVER(PARTITION BY dept ORDER BY salary)`** — Максимальная зарплата в отделе по возрастанию даты найма
- **`MIN() OVER(PARTITION BY dept)`** — Минимальная зарплата в отделе
- **`LAG(salary) OVER(PARTITION BY dept ORDER BY hire_date)`** — Предыдущая зарплата сотрудника по дате найма
- **`LEAD(salary) OVER(PARTITION BY dept ORDER BY hire_date)`** — Следующая зарплата сотрудника по дате найма

## 📖 Урок 2 - row\_number, rank, dense\_rank

## 🔗 Шаг 01:

### Миссия 1

Что делает функция ROW\_NUMBER() в PostgreSQL?

Подсказка: Подумайте о нумерации строк в результате запроса.

**Ответ:**

- Присваивает уникальный номер каждой строке в пределах окна
- Возвращает сумму значений столбца
- Определяет ранг с учётом одинаковых значений
- Создаёт новый индекс для таблицы

## Шаг 02:

### Миссия 2

Как функция RANK() обрабатывает одинаковые значения?

Подсказка: Подумайте, какой ранг получают одинаковые строки и что происходит с последующими.

**Ответ:**

- Присваивает одинаковый ранг строкам с одинаковыми значениями, пропуская позиции
- Присваивает уникальный ранг каждой строке независимо от значений
- Возвращает среднее значение рангов для одинаковых строк
- Игнорирует одинаковые значения

## Шаг 03:

### Миссия 3

Чем отличается DENSE\_RANK() от RANK()?

Подсказка: Обратите внимание, как функции ведут себя при одинаковых значениях.

**Ответ:**

- **DENSE\_RANK() не пропускает позиции после одинаковых значений**
- DENSE\_RANK() присваивает случайные номера строкам
- DENSE\_RANK() возвращает только уникальные значения
- DENSE\_RANK() игнорирует сортировку

## Шаг 04:

### Миссия 4

Сопоставьте функцию с её особенностью.

Понятия:

**Ответ:**

- ROW\_NUMBER()
- RANK()
- DENSE\_RANK()
- Пример применения

Определения:

1. Присваивает уникальный номер каждой строке независимо от значений
2. Присваивает одинаковый ранг строкам с одинаковыми значениями, пропуская позиции
3. Присваивает одинаковый ранг строкам с одинаковыми значениями, без пропуска позиций
4. Используется для упорядочивания списка сотрудников по зарплате

Подсказка: Вспомните, как каждая функция работает с одинаковыми значениями.

- **ROW\_NUMBER()** — Присваивает уникальный номер каждой строке независимо от значений
- **RANK()** — Присваивает одинаковый ранг строкам с одинаковыми значениями, пропуская позиции
- **DENSE\_RANK()** — Присваивает одинаковый ранг строкам с одинаковыми значениями, без пропуска позиций
- **Пример применения** — Используется для упорядочивания списка сотрудников по зарплате

## Шаг 05:

### Миссия 5

Что произойдет, если использовать ROW\_NUMBER() с PARTITION BY department?

Подсказка: Подумайте, как нумерация строк будет влиять на отделы.

**Ответ:**

- **Нумерация начнется заново в каждом отделе**
- Все строки будут нумероваться по всей таблице
- Пропустит первый ряд в каждом отделе
- Создаст дубликаты номеров

## Шаг 06:

### Миссия 6

Сопоставьте функцию и её эффект при использовании ORDER BY:

Понятия:

**Ответ:**

- ROW\_NUMBER() OVER(ORDER BY salary DESC)
- RANK() OVER(ORDER BY salary DESC)
- DENSE\_RANK() OVER(ORDER BY salary DESC)
- Применение

Определения:

1. Присваивает уникальные номера строкам, начиная с самой высокой зарплаты
2. Присваивает одинаковый ранг для одинаковых зарплат, с пропуском позиций
3. Присваивает одинаковый ранг для одинаковых зарплат, без пропусков позиций
4. Используется для формирования списка лидеров по зарплате

Подсказка: Обратите внимание на ORDER BY и поведение при одинаковых значениях.

- **ROW\_NUMBER() OVER(ORDER BY salary DESC)** — Присваивает уникальные номера строкам, начиная с самой высокой зарплаты
- **RANK() OVER(ORDER BY salary DESC)** — Присваивает одинаковый ранг для одинаковых зарплат, с пропуском позиций
- **DENSE\_RANK() OVER(ORDER BY salary DESC)** — Присваивает одинаковый ранг для одинаковых зарплат, без пропусков позиций
- **Применение** — Используется для формирования списка лидеров по зарплате

## Шаг 07:

### Миссия 7

Какой вариант лучше использовать для последовательной нумерации без пропусков при одинаковых значениях?

Подсказка: Подумайте, какая функция не пропускает позиции.

**Ответ:**

- **DENSE\_RANK()**
- RANK()
- ROW\_NUMBER()
- COUNT()

## Шаг 08:

### Миссия 8

Сопоставьте функцию и её применение в аналитике продаж:

Понятия:

**Ответ:**

- ROW\_NUMBER()
- RANK()
- DENSE\_RANK()
- Применение

Определения:

1. Нумерация всех заказов по дате без пропусков
2. Присвоение одинакового ранга клиентам с одинаковым объёмом покупок с пропуском
3. Присвоение одинакового ранга клиентам с одинаковым объёмом покупок без пропусков
4. Выявление топ-10 клиентов по продажам

Подсказка: Подумайте, какая функция даёт уникальные номера, а какая — ранги.

- **ROW\_NUMBER()** — Нумерация всех заказов по дате без пропусков
- **RANK()** — Присвоение одинакового ранга клиентам с одинаковым объёмом покупок с пропуском
- **DENSE\_RANK()** — Присвоение одинакового ранга клиентам с одинаковым объёмом покупок без пропусков
- **Применение** — Выявление топ-10 клиентов по продажам

## Шаг 09:

### Миссия 9

Что произойдёт, если использовать RANK() без ORDER BY?

Подсказка: Подумайте, на что влияет сортировка при вычислении рангов.

**Ответ:**

- **Все строки получают одинаковый ранг**
- Будет сгенерирована ошибка
- Присвоение уникальных номеров каждой строке
- Ранги будут случайными

## Шаг 10:

### Миссия 10

Для чего обычно используется PARTITION BY с оконными функциями?

Подсказка: Подумайте о логике разбиения данных на группы.

**Ответ:**

- **Разделяет данные на группы и выполняет нумерацию внутри каждой группы**
- Создаёт новый индекс таблицы
- Сортирует всю таблицу без группировки
- Объединяет значения в одну строку

🔗 **Шаг 11:**

### Миссия 11

Сопоставьте функции и их поведение при PARTITION BY:

Понятия:

**Ответ:**

- ROW\_NUMBER() OVER(PARTITION BY dept)
- RANK() OVER(PARTITION BY dept)
- DENSE\_RANK() OVER(PARTITION BY dept)
- Применение

Определения:

1. Нумерация сотрудников внутри отдела без пропусков
2. Ранги сотрудников с одинаковыми зарплатами с пропуском
3. Ранги сотрудников с одинаковыми зарплатами без пропусков
4. Определение лидеров внутри каждого отдела

Подсказка: PARTITION BY ограничивает действие функции одной группой.

- **ROW\_NUMBER() OVER(PARTITION BY dept)** — Нумерация сотрудников внутри отдела без пропусков
- **RANK() OVER(PARTITION BY dept)** — Ранги сотрудников с одинаковыми зарплатами с пропуском
- **DENSE\_RANK() OVER(PARTITION BY dept)** — Ранги сотрудников с одинаковыми зарплатами без пропусков
- **Применение** — Определение лидеров внутри каждого отдела

🔗 **Шаг 12:**

### Миссия 12

Что делает ORDER BY внутри OVER()?

Подсказка: Подумайте, на что влияет сортировка для оконных функций.

**Ответ:**

- **Определяет порядок строк для нумерации или ранжирования**
- Создаёт индекс таблицы
- Объединяет строки в одну
- Удаляет дубликаты

🔗 **Шаг 13:**

### Миссия 13

Как можно использовать ROW\_NUMBER() для удаления дубликатов?

Подсказка: Подумайте о нумерации внутри групп и фильтрации.

**Ответ:**

- Присвоить номер строкам в группе и оставить только первую
- Использовать ROW\_NUMBER() для суммирования
- Удалить все строки с номером больше 1 в таблице без групп
- Создать индекс для уникальности

🔗 **Шаг 14:**

#### **Миссия 14**

Расположите шаги для удаления дубликатов с помощью ROW\_NUMBER():

1. Присвоить ROW\_NUMBER() в пределах PARTITION BY ключа
2. Отфильтровать строки с ROW\_NUMBER() = 1
3. Сохранить результаты в новую таблицу
4. Удалить старую таблицу или заменить её

Подсказка: Сначала нужно пронумеровать, потом выбрать первую строку и обновить таблицу.

**Ответ:**

- Присвоить ROW\_NUMBER() в пределах PARTITION BY ключа
- Отфильтровать строки с ROW\_NUMBER() = 1
- Сохранить результаты в новую таблицу
- Удалить старую таблицу или заменить её

🔗 **Шаг 15:**

#### **Миссия 15**

Расположите шаги для выявления топ-N сотрудников по зарплате внутри отделов:

1. Использовать DENSE\_RANK() OVER(PARTITION BY dept ORDER BY salary DESC)
2. Отфильтровать строки с рангом меньше или равным N
3. Сформировать итоговую таблицу топ-N сотрудников

Подсказка: Сначала присваиваем ранги, потом фильтруем по рангу.

**Ответ:**

- Использовать DENSE\_RANK() OVER(PARTITION BY dept ORDER BY salary DESC)
- Отфильтровать строки с рангом меньше или равным N
- Сформировать итоговую таблицу топ-N сотрудников

### **Урок 3 - leadlag, first\_value, last\_value**

🔗 **Шаг 01:**

#### **Миссия 1**

Что делает функция LEAD() в PostgreSQL?

Подсказка: Подумайте о следующей строке относительно текущей.

**Ответ:**

- Возвращает значение следующей строки в окне
- Возвращает значение предыдущей строки
- Возвращает первое значение группы
- Возвращает последнее значение группы

## Шаг 02:

### Миссия 2

Что делает функция LAG() в PostgreSQL?

Подсказка: Подумайте о предыдущей строке относительно текущей.

**Ответ:**

- **Возвращает значение предыдущей строки в окне**
- Возвращает значение следующей строки
- Возвращает первое значение группы
- Возвращает последнее значение группы

## Шаг 03:

### Миссия 3

Для чего обычно используют FIRST\_VALUE()?

Подсказка: Подумайте о самой первой записи в окне.

**Ответ:**

- **Получить значение первой строки в окне**
- Получить значение последней строки в окне
- Вернуть значение следующей строки
- Вернуть значение предыдущей строки

## Шаг 04:

### Миссия 4

Для чего обычно используют LAST\_VALUE()?

Подсказка: Подумайте о самой последней записи в окне.

**Ответ:**

- **Получить значение последней строки в окне**
- Получить значение первой строки в окне
- Вернуть значение следующей строки
- Вернуть значение предыдущей строки

## Шаг 05:

### Миссия 5

Сопоставьте функцию с её типичным применением:

Понятия:

**Ответ:**

- LEAD()
- LAG()
- FIRST\_VALUE()
- LAST\_VALUE()

Определения:

1. Получение следующей транзакции пользователя
2. Сравнение текущей продажи с предыдущей
3. Определение первой покупки клиента

#### 4. Определение последней покупки клиента

Подсказка: Подумайте, какая функция смотрит вперед, а какая назад.

- **LEAD()** — Получение следующей транзакции пользователя
- **LAG()** — Сравнение текущей продажи с предыдущей
- **FIRST\_VALUE()** — Определение первой покупки клиента
- **LAST\_VALUE()** — Определение последней покупки клиента

### Шаг 06:

#### Миссия 6

Сопоставьте сценарий с подходящей функцией:

Понятия:

**Ответ:**

- LEAD()
- LAG()
- FIRST\_VALUE()
- LAST\_VALUE()

Определения:

1. Прогноз следующей даты платежа
2. Отслеживание предыдущей оценки студента
3. Начальная дата проекта
4. Конечная дата проекта

Подсказка: LEAD смотрит вперед, LAG — назад, FIRST\_VALUE и LAST\_VALUE — крайние значения.

- **LEAD()** — Прогноз следующей даты платежа
- **LAG()** — Отслеживание предыдущей оценки студента
- **FIRST\_VALUE()** — Начальная дата проекта
- **LAST\_VALUE()** — Конечная дата проекта

### Шаг 07:

#### Миссия 7

Что произойдет, если использовать LEAD() без ORDER BY?

Подсказка: Подумайте, как сортировка влияет на «следующую» строку.

**Ответ:**

- **Следующая строка будет случайной**
- Всегда будет NULL
- Произойдет ошибка
- Вернется значение первой строки

### Шаг 08:

#### Миссия 8

Что произойдет, если использовать LAG() без ORDER BY?

Подсказка: Подумайте, как определяется «предыдущая» строка.

**Ответ:**

- **Предыдущая строка будет случайной**

- Всегда будет NULL
- Произойдет ошибка
- Вернется значение первой строки

## Шаг 09:

### Миссия 9

Расположите шаги для анализа продаж с помощью LEAD():

1. Применить LEAD() к столбцу с продажами
2. Вычислить разницу между текущей и следующей продажей
3. Сгруппировать результаты по регионам
4. Проанализировать пики продаж

Подсказка: Сначала создаем «следующие» значения, затем анализируем различия и группы.

#### Ответ:

- Применить LEAD() к столбцу с продажами
- Вычислить разницу между текущей и следующей продажей
- Сгруппировать результаты по регионам
- Проанализировать пики продаж

## Шаг 10:

### Миссия 10

Сопоставьте функции с типичными примерами:

#### Ответ:

- LEAD()
  - LAG()
  - FIRST\_VALUE()
  - LAST\_VALUE()
1. Следующий заказ пользователя
  2. Предыдущая оценка студента
  3. Дата первой регистрации клиента
  4. Дата последнего входа пользователя

Подсказка: LEAD смотрит вперед, LAG назад, а FIRST\_VALUE и LAST\_VALUE — крайние значения окна.

- **LEAD()** — Следующий заказ пользователя
- **LAG()** — Предыдущая оценка студента
- **FIRST\_VALUE()** — Дата первой регистрации клиента
- **LAST\_VALUE()** — Дата последнего входа пользователя

## Шаг 11:

### Миссия 11

Сопоставьте функцию с её поведением при NULL:

#### Ответ:

- LEAD()
- LAG()
- FIRST\_VALUE()

- **LAST\_VALUE()**

1. Возвращает NULL, если следующей строки нет
2. Возвращает NULL, если предыдущей строки нет
3. Всегда возвращает значение первой строки
4. Всегда возвращает значение последней строки

Подсказка: Подумайте о границах окна.

- **LEAD()** — Возвращает NULL, если следующей строки нет
- **LAG()** — Возвращает NULL, если предыдущей строки нет
- **FIRST\_VALUE()** — Всегда возвращает значение первой строки
- **LAST\_VALUE()** — Всегда возвращает значение последней строки

## Шаг 12:

### Миссия 12

Какой результат даст FIRST\_VALUE() в окне из одной строки?

Подсказка: Окно всего из одной записи.

**Ответ:**

- **Вернет это единственное значение**
- Вернет NULL
- Произойдет ошибка
- Вернет значение следующей строки

## Шаг 13:

### Миссия 13

Что произойдет, если LAST\_VALUE() использовать с ORDER BY DESC?

Подсказка: Подумайте, как сортировка меняет крайние значения.

**Ответ:**

- **LAST\_VALUE вернет значение внизу отсортированного списка**
- LAST\_VALUE вернет значение первой строки по умолчанию
- Произойдет ошибка
- Вернет значение следующей строки

## Шаг 14:

### Миссия 14

Расположите действия для анализа оценок студентов с LAG():

1. Применить LAG() к столбцу оценок
2. Вычислить разницу между текущей и предыдущей оценкой
3. Сгруппировать студентов по классам
4. Выявить улучшения и падения в успеваемости

Подсказка: Сначала создаем предыдущие значения, затем вычисляем разницу, группируем и анализируем.

**Ответ:**

- Применить LAG() к столбцу оценок
- Вычислить разницу между текущей и предыдущей оценкой

- Сгруппировать студентов по классам
- Выявить улучшения и падения в успеваемости

### 🔮 Шаг 15:

#### Миссия 15

Расположите шаги анализа заказов с FIRST\_VALUE и LAST\_VALUE:

1. Применить FIRST\_VALUE к дате первого заказа
2. Применить LAST\_VALUE к дате последнего заказа
3. Сгруппировать заказы по клиентам
4. Сравнить первую и последнюю покупку каждого клиента

Подсказка: Сначала определяем крайние значения, потом группируем и анализируем.

#### Ответ:

- Применить FIRST\_VALUE к дате первого заказа
- Применить LAST\_VALUE к дате последнего заказа
- Сгруппировать заказы по клиентам
- Сравнить первую и последнюю покупку каждого клиента

### 🔮 Шаг 16:

#### Миссия 16

Какой тип оконной функции даст возможность смотреть как вперед, так и назад?

Подсказка: Подумайте о функциях LEAD и LAG.

#### Ответ:

- **LEAD и LAG**
- FIRST\_VALUE и LAST\_VALUE
- ROW\_NUMBER
- RANK

### 🔮 Шаг 17:

#### Миссия 17

Сопоставьте функцию с её типичным сценарием:

#### Ответ:

- LEAD()
  - LAG()
  - FIRST\_VALUE()
  - LAST\_VALUE()
1. Следующая зарплата сотрудника
  2. Предыдущий рейтинг сотрудника
  3. Дата начала работы сотрудника
  4. Дата окончания проекта

Подсказка: LEAD — вперед, LAG — назад, FIRST\_VALUE и LAST\_VALUE — крайние значения.

- **LEAD()** — Следующая зарплата сотрудника
- **LAG()** — Предыдущий рейтинг сотрудника
- **FIRST\_VALUE()** — Дата начала работы сотрудника

- **LAST\_VALUE()** — Дата окончания проекта

## Урок 4 - Сравнение оконных и агрегатных подходов

### Шаг 01:

#### Миссия 1

Что общего у оконных и агрегатных функций в PostgreSQL?

Подсказка: Подумайте о том, как они обрабатывают наборы строк.

**Ответ:**

- **Обе работают с набором строк, но выводят разные результаты**
- Обе всегда возвращают одно значение для всей таблицы
- Обе нельзя использовать с ORDER BY
- Обе автоматически группируют данные по первичному ключу

### Шаг 02:

#### Миссия 2

Какая функция позволит увидеть результат для каждой строки без группировки?

Подсказка: Подумайте, что делает оконная функция.

**Ответ:**

- **Оконная функция**
- Агрегатная функция
- Триггер
- Индекс

### Шаг 03:

#### Миссия 3

Какой результат вернет SUM() как агрегатная функция без GROUP BY?

Подсказка: Подумайте, сколько строк будет объединено.

**Ответ:**

- **Одно значение для всего набора**
- Значение для каждой строки
- Ошибка
- NULL

### Шаг 04:

#### Миссия 4

Сопоставьте функцию с результатом:

**Ответ:**

- SUM() агрегатная
  - SUM() оконная
  - AVG() агрегатная
  - AVG() оконная
1. Одно значение на всю таблицу
  2. Значение для каждой строки с учетом окна

3. Среднее по всему набору
4. Среднее по окну для каждой строки

Подсказка: Подумайте о различиях между агрегатной и оконной функцией.

- **SUM() агрегатная** — Одно значение на всю таблицу
- **SUM() оконная** — Значение для каждой строки с учетом окна
- **AVG() агрегатная** — Среднее по всему набору
- **AVG() оконная** — Среднее по окну для каждой строки

## Шаг 05:

### Миссия 5

Сопоставьте подход с его преимуществом:

**Ответ:**

- Агрегатный
  - Оконный
  - Агрегатный с GROUP BY
  - Оконный с PARTITION BY
1. Суммирует данные по всему набору
  2. Вычисляет результат для каждой строки
  3. Суммирует по группам
  4. Вычисляет результат для каждой строки с разделением на группы

Подсказка: Подумайте о различиях GROUP BY и PARTITION BY.

- **Агрегатный** — Суммирует данные по всему набору
- **Оконный** — Вычисляет результат для каждой строки
- **Агрегатный с GROUP BY** — Суммирует по группам
- **Оконный с PARTITION BY** — Вычисляет результат для каждой строки с разделением на группы

## Шаг 06:

### Миссия 6

Почему оконные функции позволяют анализировать тренды по строкам?

Подсказка: Подумайте о движении по окну строк.

**Ответ:**

- **Они дают значение для каждой строки с учетом соседних**
- Они объединяют все строки в одну
- Они удаляют дубликаты
- Они создают индекс

## Шаг 07:

### Миссия 7

Какая функция покажет, как меняется сумма продаж по дням без группировки?

Подсказка: Подумайте о движущейся сумме.

**Ответ:**

- **SUM() OVER(ORDER BY дата)**

- SUM() без OVER
- AVG() OVER(ORDER BY дата)
- COUNT()

## Шаг 08:

### Миссия 8

Сопоставьте понятие с примером:

**Ответ:**

- Агрегатная функция
- Оконная функция
- GROUP BY
- PARTITION BY

1. SUM(total) по всей таблице
2. SUM(total) для каждой строки с учетом окна
3. Группировка данных по столбцу
4. Разделение данных на окна по столбцу

Подсказка: Подумайте, как группируются и распределяются данные.

- **Агрегатная функция** — SUM(total) по всей таблице
- **Оконная функция** — SUM(total) для каждой строки с учетом окна
- **GROUP BY** — Группировка данных по столбцу
- **PARTITION BY** — Разделение данных на окна по столбцу

## Шаг 09:

### Миссия 9

В чем преимущество оконной функции перед агрегатной при анализе трендов?

Подсказка: Подумайте о видимости каждой строки.

**Ответ:**

- **Можно видеть результат для каждой строки без объединения**
- Можно группировать только по одному столбцу
- Автоматически создаются индексы
- Производительность всегда выше

## Шаг 10:

### Миссия 10

Что будет результатом SUM() OVER(PARTITION BY user\_id) при анализе заказов?

Подсказка: Подумайте о разделении данных на группы.

**Ответ:**

- **Сумма заказов для каждого пользователя**
- Сумма всех заказов
- Среднее значение заказов
- Ошибка синтаксиса

## Шаг 11:

### Миссия 11

Расположите шаги сравнения агрегатного и оконного подходов:

1. Определить цель анализа
2. Применить агрегатную функцию
3. Применить оконную функцию
4. Сравнить результаты и выявить различия

Подсказка: Сначала цель, потом выполнение, потом анализ.

**Ответ:**

- Определить цель анализа
- Применить агрегатную функцию
- Применить оконную функцию
- Сравнить результаты и выявить различия

## Шаг 12:

### **Миссия 12**

Расположите действия для анализа тренда продаж:

1. Сгруппировать данные по дате (день/неделя)
2. Применить SUM() агрегатную
3. Применить SUM() оконную с ORDER BY
4. Сравнить результаты и выявить закономерности

Подсказка: Сначала группировка, затем вычисления, потом сравнение.

**Ответ:**

- Сгруппировать данные по дате (день/неделя)
- Применить SUM() агрегатную
- Применить SUM() оконную с ORDER BY
- Сравнить результаты и выявить закономерности

## Шаг 13:

### **Миссия 13**

Расположите шаги анализа среднего времени отклика клиентов:

1. Использовать AVG() агрегатную для всей таблицы
2. Использовать AVG() оконную с PARTITION BY для каждого клиента
3. Сравнить результаты для выявления отличий
4. Интерпретировать данные для отчета

Подсказка: Сначала выполняем вычисления, потом анализируем различия, затем делаем выводы.

**Ответ:**

- Использовать AVG() агрегатную для всей таблицы
- Использовать AVG() оконную с PARTITION BY для каждого клиента
- Сравнить результаты для выявления отличий
- Интерпретировать данные для отчета

## Шаг 14:

### **Миссия 14**

Какой подход лучше использовать, если нужен результат для каждой строки без потери контекста других данных?

Подсказка: Подумайте, какая функция возвращает значение для каждой строки с учетом соседних.

**Ответ:**

- **Оконная функция**
- Агрегатная функция без GROUP BY
- Триггер
- Индекс

## Шаг 15:

### Миссия 15

Сопоставьте термин с примером использования:

**Ответ:**

- Агрегатная функция
  - Оконная функция
  - GROUP BY
  - PARTITION BY
1. COUNT(order\_id) для всей таблицы
  2. SUM(order\_amount) OVER(ORDER BY date)
  3. Сумма продаж по каждому региону
  4. Сумма продаж по каждому региону для каждой строки

Подсказка: Обратите внимание, где используется окно и где групповая агрегация.

- **Агрегатная функция** — COUNT(order\_id) для всей таблицы
- **Оконная функция** — SUM(order\_amount) OVER(ORDER BY date)
- **GROUP BY** — Сумма продаж по каждому региону
- **PARTITION BY** — Сумма продаж по каждому региону для каждой строки

## Шаг 16:

### Миссия 16

Расположите шаги для анализа cumulative sum с использованием оконной функции:

1. Определить порядок строк (ORDER BY)
2. Применить SUM() OVER() с выбранным окном
3. Визуализировать результаты
4. Сравнить с агрегатным подходом для проверки корректности

Подсказка: Сначала упорядочиваем, потом считаем, потом визуализируем и проверяем.

**Ответ:**

- Определить порядок строк (ORDER BY)
- Применить SUM() OVER() с выбранным окном
- Визуализировать результаты
- Сравнить с агрегатным подходом для проверки корректности

## Урок 5 - Мини-босс аналитика действий пользователей по сессиям

### Шаг 01:

#### Миссия 1

Что позволяет понять функция ROW\_NUMBER() при анализе действий пользователей по сессиям?

Подсказка: Она нумерует строки в пределах группы.

**Ответ:**

- **Порядок действий пользователей внутри сессии**
- Общее количество сессий
- Максимальное время отклика сервера
- Среднее значение по всем пользователям

### Шаг 02:

#### Миссия 2

Какая оконная функция поможет сравнивать действия пользователя с предыдущей записью в сессии?

Подсказка: Она позволяет видеть соседние строки.

**Ответ:**

- **LAG()**
- SUM()
- COUNT()
- MIN()

### Шаг 03:

#### Миссия 3

Какая функция вернёт первое действие пользователя в сессии?

Подсказка: Оконная функция, возвращающая значение первой строки в окне.

**Ответ:**

- **FIRST\_VALUE()**
- LAST\_VALUE()
- ROW\_NUMBER()
- RANK()

### Шаг 04:

#### Миссия 4

Какой подход эффективнее для анализа действий всех пользователей без группировки?

Подсказка: Подумайте, что позволяет получать результат для каждой строки.

**Ответ:**

- **Оконная функция**
- Агрегатная с GROUP BY
- Индексирование
- Триггер

## Шаг 05:

### Миссия 5

Сопоставьте термин с примером из анализа сессий:

**Ответ:**

- ROW\_NUMBER()
- LAG()
- FIRST\_VALUE()
- LAST\_VALUE()

1. Нумерация действий пользователя внутри сессии
2. Сравнение текущего действия с предыдущим
3. Получение первого действия в сессии
4. Получение последнего действия в сессии

Подсказка: Обратите внимание на то, что нужно взять: первый, последний или соседний элемент.

- **ROW\_NUMBER()** — Нумерация действий пользователя внутри сессии
- **LAG()** — Сравнение текущего действия с предыдущим
- **FIRST\_VALUE()** — Получение первого действия в сессии
- **LAST\_VALUE()** — Получение последнего действия в сессии

## Шаг 06:

### Миссия 6

Сопоставьте SQL-конструкцию с её действием в сессиях:

**Ответ:**

- PARTITION BY
- ORDER BY
- OVER()
- FRAME

1. Разделение данных на группы по сессии
2. Определение порядка действий внутри сессии
3. Применение оконной функции к каждой строке
4. Определение диапазона строк для расчёта функции

Подсказка: PARTITION — группы, ORDER — порядок, OVER — применение, FRAME — диапазон.

- **PARTITION BY** — Разделение данных на группы по сессии
- **ORDER BY** — Определение порядка действий внутри сессии
- **OVER()** — Применение оконной функции к каждой строке
- **FRAME** — Определение диапазона строк для расчёта функции

## Шаг 07:

### Миссия 7

Сопоставьте тип аналитики с применением оконной функции:

**Ответ:**

- Последовательность действий
- Разница между событиями

- Первое действие
- Последнее действие

1. ROW\_NUMBER()
2. LAG()
3. FIRST\_VALUE()
4. LAST\_VALUE()

Подсказка: Свяжите функции с тем, что они возвращают в сессиях.

- **Последовательность действий** — ROW\_NUMBER()
- **Разница между событиями** — LAG()
- **Первое действие** — FIRST\_VALUE()
- **Последнее действие** — LAST\_VALUE()

## 🔗 Шаг 08:

### Миссия 8

Расположите шаги для анализа последовательности действий пользователя:

1. Разделение данных на сессии (PARTITION BY user\_id, session\_id)
2. Нумерация действий внутри сессии (ROW\_NUMBER())
3. Сортировка действий по времени (ORDER BY timestamp)
4. Анализ последовательности действий

Подсказка: Сначала формируем группы, затем упорядочиваем, потом нумеруем и анализируем.

**Ответ:**

- Разделение данных на сессии (PARTITION BY user\_id, session\_id)
- Нумерация действий внутри сессии (ROW\_NUMBER())
- Сортировка действий по времени (ORDER BY timestamp)
- Анализ последовательности действий

## 🔗 Шаг 09:

### Миссия 9

Что позволяет функция LEAD() при анализе действий пользователя?

Подсказка: Она показывает следующий элемент в последовательности.

**Ответ:**

- **Сравнивать текущее действие с последующим**
- Суммировать все действия пользователя
- Показывать только первое действие
- Игнорировать порядок действий

## 🔗 Шаг 10:

### Миссия 10

Какая оконная функция полезна для подсчета накопленных действий пользователя в сессии?

Подсказка: Функция вычисляет значение по нарастающей сумме.

**Ответ:**

- **SUM() OVER()**
- COUNT(DISTINCT)

- MAX()
- MIN()

## Шаг 11:

### Миссия 11

Сопоставьте описание с оконной функцией:

**Ответ:**

- ROW\_NUMBER()
  - RANK()
  - DENSE\_RANK()
  - LAG()
1. Порядковый номер строки с пропуском при одинаковых значениях
  2. Порядковый номер строки без пропусков
  3. Нумерация строк внутри группы
  4. Предыдущая запись в последовательности

Подсказка: Различайте порядок с пропусками и без.

- **ROW\_NUMBER()** — Нумерация строк внутри группы
- **RANK()** — Порядковый номер строки с пропуском при одинаковых значениях
- **DENSE\_RANK()** — Порядковый номер строки без пропусков
- **LAG()** — Предыдущая запись в последовательности

## Шаг 12:

### Миссия 12

Сопоставьте аналитический запрос с результатом:

**Ответ:**

- Посчитать первый клик в каждой сессии
  - Сравнить текущее и предыдущее действие
  - Нумерация действий пользователя
  - Определить последнее действие сессии
1. FIRST\_VALUE()
  2. LAG()
  3. ROW\_NUMBER()
  4. LAST\_VALUE()

Подсказка: Подумайте, какая функция возвращает первый и последний элемент, а какая — последовательность.

- **Посчитать первый клик в каждой сессии** — FIRST\_VALUE()
- **Сравнить текущее и предыдущее действие** — LAG()
- **Нумерация действий пользователя** — ROW\_NUMBER()
- **Определить последнее действие сессии** — LAST\_VALUE()

## Шаг 13:

### Миссия 13

Расположите шаги анализа сессий с оконными функциями:

1. Применение оконной функции (OVER())
2. Определение группировки пользователей (PARTITION BY)
3. Сортировка действий по времени (ORDER BY)
4. Анализ результатов функции

Подсказка: Сначала формируем группы, потом порядок, затем применяем функцию и анализируем.

**Ответ:**

- Применение оконной функции (OVER())
- Определение группировки пользователей (PARTITION BY)
- Сортировка действий по времени (ORDER BY)
- Анализ результатов функции

#### Шаг 14:

##### **Миссия 14**

Упорядочите шаги построения отчета по активным пользователям:

1. Сгруппировать по сессиям
2. Посчитать накопленные действия (SUM() OVER())
3. Выбрать ключевые метрики (ROW\_NUMBER(), LAG())
4. Анализировать последовательность действий

Подсказка: Сначала группируем, потом считаем накопленные значения, потом метрики, затем анализ.

**Ответ:**

- Сгруппировать по сессиям
- Посчитать накопленные действия (SUM() OVER())
- Выбрать ключевые метрики (ROW\_NUMBER(), LAG())
- Анализировать последовательность действий

#### Шаг 15:

##### **Миссия 15**

Расположите шаги для вычисления времени между действиями пользователя:

1. Сортировка действий по времени
2. Применение LAG() для предыдущей записи
3. Вычисление разницы между текущей и предыдущей записью
4. Анализ и визуализация полученных временных интервалов

Подсказка: Сначала упорядочите, затем сравните, затем вычислите разницу и анализируйте.

**Ответ:**

- Сортировка действий по времени
- Применение LAG() для предыдущей записи
- Вычисление разницы между текущей и предыдущей записью
- Анализ и визуализация полученных временных интервалов

## Шаг 16:

### Миссия 16

Расположите шаги для выявления первой и последней активности пользователя:

1. Применение FIRST\_VALUE() и LAST\_VALUE()
2. Сортировка действий по времени
3. Формирование оконной функции (OVER(PARTITION BY user\_id ORDER BY timestamp))
4. Анализ результатов

Подсказка: Сначала формируем окно, сортируем, применяем функции, затем анализируем.

#### Ответ:

- Применение FIRST\_VALUE() и LAST\_VALUE()
- Сортировка действий по времени
- Формирование оконной функции (OVER(PARTITION BY user\_id ORDER BY timestamp))
- Анализ результатов

## Шаг 17:

### Миссия 17

Расположите шаги построения комплексного отчета по сессиям пользователей:

1. Определение окон и группировок
2. Применение функций FIRST\_VALUE(), LAST\_VALUE(), LAG(), ROW\_NUMBER()
3. Вычисление накопленных метрик (SUM() OVER())
4. Анализ и визуализация поведения пользователей

Подсказка: Сначала формируем основу (окна), потом применяем функции, считаем метрики и анализируем.

#### Ответ:

- Определение окон и группировок
- Применение функций FIRST\_VALUE(), LAST\_VALUE(), LAG(), ROW\_NUMBER()
- Вычисление накопленных метрик (SUM() OVER())
- Анализ и визуализация поведения пользователей

## Модуль 5 - Функции, процедуры и код на PLpgSQL

### Урок 1 - Что такое PLpgSQL, когда его использовать

## Шаг 01:

### Миссия 1

Что такое PL/pgSQL в PostgreSQL?

Подсказка: Это не отдельная СУБД, а инструмент для расширения возможностей SQL.

#### Ответ:

- **Язык процедурного программирования для PostgreSQL**
- Отдельная СУБД
- Графический интерфейс PostgreSQL
- Набор SQL-команд только для SELECT

## Шаг 02:

### Миссия 2

Когда имеет смысл использовать PL/pgSQL?

Подсказка: Подумайте, когда одного SQL недостаточно для бизнес-логики.

**Ответ:**

- **Для сложной бизнес-логики с условными операторами и циклами**
- Для простой выборки из одной таблицы
- Для форматирования текста на клиенте
- Для экспорта данных в Excel

## Шаг 03:

### Миссия 3

Какая особенность PL/pgSQL помогает обрабатывать ошибки внутри функции?

Подсказка: Это стандартная конструкция для перехвата исключений.

**Ответ:**

- **Блок EXCEPTION**
- Блок TRY-CATCH (Python)
- Функция CHECK()
- Событие ON ERROR

## Шаг 04:

### Миссия 4

Что позволяет использовать курсору в PL/pgSQL?

Подсказка: Подумайте о работе с несколькими строками результата по одной.

**Ответ:**

- **Обход строк результата SELECT по одной**
- Сортировка таблицы
- Создание индексов
- Создание триггеров

## Шаг 05:

### Миссия 5

Какая конструкция позволяет выполнять условные действия в функции PL/pgSQL?

Подсказка: Подумайте о привычном IF...ELSE.

**Ответ:**

- **IF...THEN...ELSE**
- SWITCH...CASE
- FOR EACH
- WHILE SELECT

## Шаг 06:

### Миссия 6

Сопоставьте термин PL/pgSQL с его объяснением:

**Ответ:**

- Функция
  - Процедура
  - Курсор
  - EXCEPTION
1. Блок кода, который можно вызвать и вернуть значение
  2. Блок кода без возврата значения
  3. Обход строк результата SELECT по одной
  4. Перехват ошибок и обработка исключений

Подсказка: Различайте функции и процедуры, а также вспомните конструкции обработки ошибок и курсоры.

- **Функция** — Блок кода, который можно вызвать и вернуть значение
- **Процедура** — Блок кода без возврата значения
- **Курсор** — Обход строк результата SELECT по одной
- **EXCEPTION** — Перехват ошибок и обработка исключений

 **Шаг 07:**

**Миссия 7**

Сопоставьте тип данных PL/pgSQL с их использованием:

**Ответ:**

- INTEGER
  - TEXT
  - BOOLEAN
  - RECORD
1. Хранение целых чисел
  2. Хранение строкового текста
  3. Истина или ложь
  4. Объединение произвольных колонок одной строки

Подсказка: Обратите внимание на стандартные типы и специальные RECORD.

- **INTEGER** — Хранение целых чисел
- **TEXT** — Хранение строкового текста
- **BOOLEAN** — Истина или ложь
- **RECORD** — Объединение произвольных колонок одной строки

 **Шаг 08:**

**Миссия 8**

Сопоставьте PL/pgSQL-конструкции с их назначением:

**Ответ:**

- LOOP
  - FOR
  - WHILE
  - PERFORM
1. Бесконечный цикл с ручным выходом

2. Цикл по диапазону значений
3. Цикл с условием проверки до входа
4. Вызов функции без возврата значения

Подсказка: Различайте типы циклов и вызов функций без возврата.

- **LOOP** — Бесконечный цикл с ручным выходом
- **FOR** — Цикл по диапазону значений
- **WHILE** — Цикл с условием проверки до входа
- **PERFORM** — Вызов функции без возврата значения

## Шаг 09:

### Миссия 9

Расположите шаги создания функции на PL/pgSQL:

1. Определение входных параметров и типа возвращаемого значения
2. Начало блока функции (DECLARE при необходимости)
3. Основная логика функции (BEGIN...END)
4. Возврат результата

Подсказка: Сначала определяем параметры, затем блок переменных, потом логику, в конце возвращаем значение.

#### Ответ:

- Определение входных параметров и типа возвращаемого значения
- Начало блока функции (DECLARE при необходимости)
- Основная логика функции (BEGIN...END)
- Возврат результата

## Шаг 10:

### Миссия 10

Расположите шаги создания функции на PL/pgSQL:

1. Определение входных параметров и типа возвращаемого значения
2. Начало блока функции (DECLARE при необходимости)
3. Основная логика функции (BEGIN...END)
4. Возврат результата

Подсказка: Сначала определяем параметры, затем блок переменных, потом логику, в конце возвращаем значение.

#### Ответ:

- Определение входных параметров и типа возвращаемого значения
- Начало блока функции (DECLARE при необходимости)
- Основная логика функции (BEGIN...END)
- Возврат результата

## Шаг 11:

### Миссия 11

Расположите шаги обработки ошибок в PL/pgSQL:

1. Определение блока BEGIN...END функции

2. Выполнение основной логики
3. Использование блока EXCEPTION для перехвата ошибок
4. Возврат значения или логирование ошибки

Подсказка: Сначала основной код, затем блок обработки ошибок, в конце завершаем функцию корректно.

**Ответ:**

- Определение блока BEGIN...END функции
- Выполнение основной логики
- Использование блока EXCEPTION для перехвата ошибок
- Возврат значения или логирование ошибки

## Шаг 12:

### Миссия 12

Расположите этапы работы с курсором в PL/pgSQL:

1. Открытие курсора
2. Извлечение строк с помощью FETCH
3. Обработка данных в цикле
4. Закрытие курсора

Подсказка: Курсор сначала открываем, затем извлекаем и обрабатываем, в конце закрываем.

**Ответ:**

- Открытие курсора
- Извлечение строк с помощью FETCH
- Обработка данных в цикле
- Закрытие курсора

## Шаг 13:

### Миссия 13

Расположите шаги создания процедуры в PL/pgSQL:

1. Определение процедуры с входными параметрами
2. Начало блока тела процедуры
3. Выполнение операций (INSERT, UPDATE, DELETE и др.)
4. Завершение процедуры

Подсказка: Процедура похожа на функцию, только без возврата значения.

**Ответ:**

- Определение процедуры с входными параметрами
- Начало блока тела процедуры
- Выполнение операций (INSERT, UPDATE, DELETE и др.)
- Завершение процедуры

## Шаг 14:

### Миссия 14

Расположите шаги вызова функции с использованием PERFORM:

1. Написание запроса PERFORM function\_name(params)

2. Передача параметров функции
3. Выполнение функции без возврата результата
4. Продолжение работы функции или процедуры

Подсказка: PERFORM используется для вызова функции без возврата значения, порядок важен.

**Ответ:**

- Написание запроса PERFORM function\_name(params)
- Передача параметров функции
- Выполнение функции без возврата результата
- Продолжение работы функции или процедуры

## Шаг 15:

### Миссия 15

Расположите шаги выполнения условного цикла WHILE:

1. Определение условия цикла
2. Выполнение блока кода, если условие TRUE
3. Проверка условия перед каждой итерацией
4. Выход из цикла при FALSE

Подсказка: WHILE проверяет условие до каждой итерации, затем выполняет блок кода.

**Ответ:**

- Определение условия цикла
- Выполнение блока кода, если условие TRUE
- Проверка условия перед каждой итерацией
- Выход из цикла при FALSE

## Шаг 16:

### Миссия 16

Расположите этапы комбинированного использования функций и курсоров:

1. Создание функции с возвращаемым типом RECORD
2. Определение курсора внутри функции
3. Цикл обработки каждой строки курсора
4. Возврат результирующего набора данных

Подсказка: Сначала функция с RECORD, затем курсор, потом обработка, в конце возврат результата.

**Ответ:**

- Создание функции с возвращаемым типом RECORD
- Определение курсора внутри функции
- Цикл обработки каждой строки курсора
- Возврат результирующего набора данных

## Урок 2 - Создание и вызов функций

## Шаг 01:

### Миссия 1

Вы пишете функцию для подсчета среднего балла студентов. Какой тип функции в PL/pgSQL лучше использовать?

Подсказка: Подумайте, нужна ли функция для вычисления или просто для изменения данных.

**Ответ:**

- **Функция, которая возвращает значение**
- Функция, которая изменяет таблицу без возврата
- Триггерная функция
- Процедура для обновления записей

## Шаг 02:

### Миссия 2

Вы хотите вызвать функцию подсчета среднего балла для всех студентов в группе. Какой синтаксис правильный?

Подсказка: Функции, возвращающие значение, можно использовать в SELECT.

**Ответ:**

- **SELECT avg\_score('group1');**
- CALL avg\_score('group1');
- EXEC avg\_score('group1');
- RUN avg\_score('group1');

## Шаг 03:

### Миссия 3

Вы создаете функцию для увеличения зарплаты сотрудников на 10%. Какой тип функции использовать?

Подсказка: Функция должна менять данные в таблице, но не обязательно возвращать значение.

**Ответ:**

- **Процедура**
- Функция, возвращающая значение
- VIEW
- Триггерная функция

## Шаг 04:

### Миссия 4

Вы вызываете функцию, но получаете ошибку. Что чаще всего является причиной?

Подсказка: Проверьте правильность имени функции и количество аргументов.

**Ответ:**

- **Несоответствие количества или типов аргументов**
- Отсутствие индекса на таблице
- Неправильный SELECT без JOIN
- Ошибка в формате даты

## Шаг 05:

### Миссия 5

 Сопоставьте термины PL/pgSQL с их назначением

Термины:

**Ответ:**

- Функция
- Процедура
- RETURN
- CALL

Назначение:

1. Возвращает значение из функции
2. Выполняет действия без возврата значения
3. Вызывает процедуру
4. Определяет блок кода, который возвращает результат

 **Подсказка:**

Функции могут возвращать значения, процедуры — выполнять действия. RETURN используется для результата.

- **Функция** — Определяет блок кода, который возвращает результат
- **Процедура** — Выполняет действия без возврата значения
- **RETURN** — Возвращает значение из функции
- **CALL** — Вызывает процедуру

 **Шаг 06:**

**Миссия 6**

 Сопоставьте ошибки PL/pgSQL с их описанием

Ошибки:

**Ответ:**

- EXCEPTION
- NULL
- RAISE NOTICE
- PERFORM

Описание:

1. Используется для вызова сообщения в консоль
2. Используется для обработки исключений
3. Позволяет игнорировать возвращаемое значение функции
4. Отсутствие значения

 **Подсказка:**

EXCEPTION — обработка ошибок, NULL — пустое значение, RAISE NOTICE — сообщение.

- **EXCEPTION** — Используется для обработки исключений
- **NULL** — Отсутствие значения
- **RAISE NOTICE** — Используется для вызова сообщения в консоль
- **PERFORM** — Позволяет игнорировать возвращаемое значение функции

 **Шаг 07:**

**Миссия 7**

 Сопоставьте параметры функции с их ролью

Параметры:

**Ответ:**

- IN
- OUT
- INOUT
- VARIADIC

Роль:

1. Входной параметр функции
2. Выходной параметр функции
3. Параметр и для входа, и для выхода
4. Параметр, принимающий любое количество значений

 **Подсказка:**

IN — вход, OUT — выход, INOUT сочетает оба, VARIADIC — массив значений.

- **IN** — Входной параметр функции
- **OUT** — Выходной параметр функции
- **INOUT** — Параметр и для входа, и для выхода
- **VARIADIC** — Параметр, принимающий любое количество значений

 **Шаг 08:**

**Миссия 8**

 Сопоставьте типы функций с примерами использования

Типы:

**Ответ:**

- IMMUTABLE
- STABLE
- VOLATILE

Использование:

1. Функция всегда возвращает одинаковый результат для одних и тех же входных данных
2. Функция возвращает результат, который может зависеть от состояния базы, но не изменяет данные
3. Функция может изменять данные или состояние базы

 **Подсказка:**

IMMUTABLE — полностью предсказуемая, STABLE — консистентная в рамках запроса, VOLATILE — может менять данные.

- **IMMUTABLE** — Функция всегда возвращает одинаковый результат для одних и тех же входных данных
- **STABLE** — Функция возвращает результат, который может зависеть от состояния базы, но не изменяет данные
- **VOLATILE** — Функция может изменять данные или состояние базы

 **Шаг 09:**

**Миссия 9**

Расположите шаги создания функции в правильном порядке:

1. Написание тела функции

2. Определение типа и параметров функции
3. Вызов функции для тестирования
4. Сохранение функции в базе данных

 **Подсказка:**

Сначала описываем интерфейс, потом пишем тело, сохраняем и тестируем.

**Ответ:**

- Определение типа и параметров функции
- Написание тела функции
- Сохранение функции в базе данных
- Вызов функции для тестирования

 **Шаг 10:**

**Миссия 10**

Расположите шаги обработки ошибок в функции:

1. Добавление блока EXCEPTION
2. Определение кода, который может вызвать ошибку
3. Обработка ошибки и вывод сообщения
4. Возврат результата функции

 **Подсказка:**

Сначала идентифицируем потенциальные ошибки, потом добавляем обработку, выводим сообщение и завершаем работу.

**Ответ:**

- Определение кода, который может вызвать ошибку
- Добавление блока EXCEPTION
- Обработка ошибки и вывод сообщения
- Возврат результата функции

 **Шаг 11:**

**Миссия 11**

Расположите шаги оптимизации функции:

1. Профилирование функции
2. Выявление узких мест
3. Изменение кода для повышения эффективности
4. Повторное тестирование функции

 **Подсказка:**

Сначала измеряем, потом улучшаем, затем проверяем результат.

**Ответ:**

- Профилирование функции
- Выявление узких мест
- Изменение кода для повышения эффективности
- Повторное тестирование функции

## Шаг 12:

### Миссия 12

Расположите шаги работы с параметрами функции:

1. Определение параметров IN, OUT или INOUT
2. Использование параметров в теле функции
3. Вызов функции с аргументами
4. Проверка возвращаемых значений

### Подсказка:

Сначала определяем, потом используем, вызываем и проверяем результат.

### Ответ:

- Определение параметров IN, OUT или INOUT
- Использование параметров в теле функции
- Вызов функции с аргументами
- Проверка возвращаемых значений

## Шаг 13:

### Миссия 13

Расположите шаги вызова процедуры с несколькими операциями:

1. Определение процедуры
2. Передача параметров
3. Выполнение действий внутри процедуры
4. Проверка результатов в таблицах

### Подсказка:

Процедура определяет действия, потом мы передаем параметры, выполняем и проверяем результат.

### Ответ:

- Определение процедуры
- Передача параметров
- Выполнение действий внутри процедуры
- Проверка результатов в таблицах

## Шаг 14:

### Миссия 14

Расположите шаги создания функции с обработкой ошибок и логированием:

1. Определение параметров функции
2. Написание тела функции с основным кодом
3. Добавление блока EXCEPTION для обработки ошибок
4. Использование RAISE NOTICE для логирования
5. Тестирование функции

### Подсказка:

Сначала создаём функцию, затем добавляем обработку ошибок и логирование, в конце тестируем.

**Ответ:**

- Определение параметров функции
- Написание тела функции с основным кодом
- Добавление блока EXCEPTION для обработки ошибок
- Использование RAISE NOTICE для логирования
- Тестирование функции

 **Шаг 15:**

**Миссия 15**

Расположите шаги при создании функции с несколькими параметрами и проверкой на NULL:

1. Определение входных параметров
2. Добавление проверки на NULL внутри тела функции
3. Обработка ошибок при недопустимых значениях
4. Возврат результата функции
5. Тестирование с разными значениями

 **Подсказка:**

Сначала определяем параметры, затем проверяем значения, обрабатываем ошибки и тестируем результат.

**Ответ:**

- Определение входных параметров
- Добавление проверки на NULL внутри тела функции
- Обработка ошибок при недопустимых значениях
- Возврат результата функции
- Тестирование с разными значениями

 **Шаг 16:**

**Миссия 16**

Расположите шаги при создании функции с циклом и условными операторами:

1. Определение функции и параметров
2. Написание цикла для обработки множества строк
3. Добавление условных операторов внутри цикла
4. Обработка возможных ошибок с EXCEPTION
5. Возврат итогового результата
6. Тестирование функции на разных наборах данных

 **Подсказка:**

Сначала создаём функцию, затем цикл с условиями, добавляем обработку ошибок и проверяем результат.

**Ответ:**

- Определение функции и параметров
- Написание цикла для обработки множества строк
- Добавление условных операторов внутри цикла
- Обработка возможных ошибок с EXCEPTION
- Возврат итогового результата
- Тестирование функции на разных наборах данных

## Урок 3 - Переменные, IF, CASE, циклы

### Шаг 01:

#### Миссия 1

Вы создаёте функцию для подсчёта бонусов сотрудников. Какой тип переменной лучше использовать для хранения промежуточного результата?

Подсказка: Подумайте о типе данных, который подходит для числовых расчётов с дробными значениями.

**Ответ:**

- **NUMERIC или DECIMAL**
- TEXT
- BOOLEAN
- DATE

### Шаг 02:

#### Миссия 2

Вам нужно использовать условие в функции: если зарплата > 5000, начислить бонус 10%, иначе 5%. Какой оператор подходит лучше всего?

Подсказка: Сравниваем значения и выполняем разные действия.

**Ответ:**

- **IF ... THEN ... ELSE ... END IF**
- CASE без WHEN
- LOOP
- RAISE NOTICE

### Шаг 03:

#### Миссия 3

Вы хотите обработать несколько условий одновременно: начислить бонус в зависимости от уровня должности. Какой вариант конструкции проще всего использовать?

Подсказка: Если есть несколько вариантов значений, удобнее использовать выборку по вариантам.

**Ответ:**

- **CASE WHEN ... THEN ... ELSE ... END**
- LOOP
- IF без ELSE
- RAISE EXCEPTION

### Шаг 04:

#### Миссия 4

В функции нужно повторно выполнять начисление бонусов для каждой записи в таблице сотрудников. Какой оператор PL/pgSQL подойдет?

Подсказка: Подумайте о повторении действия для нескольких строк.

**Ответ:**

- **FOR ... LOOP**
- IF ... THEN

- RAISE NOTICE
- PERFORM

## 🔮 Шаг 05:

### Миссия 5

📖 Сопоставьте типы переменных с примером использования в функции

Типы:

**Ответ:**

- INTEGER
- TEXT
- BOOLEAN
- NUMERIC

Пример:

1. Счётчик циклов
2. Флаг для проверки условия
3. Описание должности сотрудника
4. Сумма бонуса с дробными числами

### 🔮 Подсказка:

INTEGER для счётчиков, BOOLEAN для условий, TEXT для текста, NUMERIC для денежных расчётов.

- **INTEGER** — Счётчик циклов
- **BOOLEAN** — Флаг для проверки условия
- **TEXT** — Описание должности сотрудника
- **NUMERIC** — Сумма бонуса с дробными числами

## 🔮 Шаг 06:

### Миссия 6

📖 Сопоставьте конструкции PL/pgSQL с их назначением

Конструкции:

**Ответ:**

- IF
- CASE
- FOR ... LOOP
- WHILE ... LOOP

Назначение:

1. Выбор между несколькими вариантами на основе условий
2. Цикл с заранее известным числом повторений
3. Условная проверка одного условия
4. Цикл, выполняющийся, пока выполняется условие

### 🔮 Подсказка:

IF — для одного условия, CASE — для множества вариантов, FOR и WHILE для циклов.

- **IF** — Условная проверка одного условия
- **CASE** — Выбор между несколькими вариантами на основе условий

- **FOR ... LOOP** — Цикл с заранее известным числом повторений
- **WHILE ... LOOP** — Цикл, выполняющийся, пока выполняется условие

## Шаг 07:

### Миссия 7

 Сопоставьте ключевые слова PL/pgSQL с их функцией

Ключевые слова:

**Ответ:**

- RAISE NOTICE
- PERFORM
- RETURN
- EXCEPTION

Функция:

1. Вывод отладочного сообщения
2. Выполнение запроса без сохранения результата
3. Возврат значения из функции
4. Обработка ошибок

### Подсказка:

RAISE NOTICE — для сообщений, PERFORM — для выполнения запроса без результата, RETURN — возврат, EXCEPTION — ошибки.

- **RAISE NOTICE** — Вывод отладочного сообщения
- **PERFORM** — Выполнение запроса без сохранения результата
- **RETURN** — Возврат значения из функции
- **EXCEPTION** — Обработка ошибок

## Шаг 08:

### Миссия 8

 Сопоставьте виды циклов с примером использования

Циклы:

**Ответ:**

- FOR
- WHILE
- LOOP

Пример:

1. Повторить 10 начислений бонусов
2. Продолжать начислять бонусы пока зарплата < 5000
3. Выполнять цикл без заранее известного числа итераций с ручной проверкой

### Подсказка:

FOR — известное число итераций, WHILE — условие, LOOP — универсальный цикл.

- **FOR** — Повторить 10 начислений бонусов
- **WHILE** — Продолжать начислять бонусы пока зарплата < 5000
- **LOOP** — Выполнять цикл без заранее известного числа итераций с ручной проверкой

## Шаг 09:

### Миссия 9

Расположите шаги для вычисления бонусов с использованием цикла FOR:

1. Объявление переменной-счётчика
2. Запуск цикла FOR по всем сотрудникам
3. Вычисление бонуса для каждой записи
4. Сохранение результата в таблице
5. Выход из цикла

### Подсказка:

Сначала создаём переменную, затем цикл, вычисляем и сохраняем результат, после чего цикл завершён.

### Ответ:

- Объявление переменной-счётчика
- Запуск цикла FOR по всем сотрудникам
- Вычисление бонуса для каждой записи
- Сохранение результата в таблице
- Выход из цикла

## Шаг 10:

### Миссия 10

Расположите шаги для использования конструкции CASE для разных должностей:

1. Определение переменной для хранения бонуса
2. Написание конструкции CASE с проверкой должности
3. Назначение соответствующего бонуса
4. Сохранение значения переменной
5. Вывод или возврат результата

### Подсказка:

Сначала определяем переменную, потом CASE для выбора, присваиваем значение и возвращаем результат.

### Ответ:

- Определение переменной для хранения бонуса
- Написание конструкции CASE с проверкой должности
- Назначение соответствующего бонуса
- Сохранение значения переменной
- Вывод или возврат результата

## Шаг 11:

### Миссия 11

Расположите шаги использования WHILE для начисления бонусов до достижения лимита бюджета:

1. Определение переменной для текущего бюджета
2. Запуск цикла WHILE с условием бюджета
3. Начисление бонуса сотруднику
4. Обновление переменной бюджета

5. Выход из цикла, когда бюджет исчерпан

 **Подсказка:**

Сначала определяем переменную, затем выполняем цикл с проверкой условия и обновляем значение после каждой итерации.

**Ответ:**

- Определение переменной для текущего бюджета
- Запуск цикла WHILE с условием бюджета
- Начисление бонуса сотруднику
- Обновление переменной бюджета
- Выход из цикла, когда бюджет исчерпан

 **Шаг 12:**

**Миссия 12**

Расположите шаги для обработки ошибок с EXCEPTION внутри функции:

1. Начало блока BEGIN
2. Основной код функции
3. Объявление блока EXCEPTION
4. Обработка ошибок и RAISE NOTICE
5. Завершение функции RETURN

 **Подсказка:**

Сначала пишем основной код, затем блок ошибок, в конце возвращаем результат.

**Ответ:**

- Начало блока BEGIN
- Основной код функции
- Объявление блока EXCEPTION
- Обработка ошибок и RAISE NOTICE
- Завершение функции RETURN

 **Шаг 13:**

**Миссия 13**

Расположите шаги для создания функции с циклом FOR и проверкой условия IF:

1. Объявление функции и параметров
2. Запуск цикла FOR по всем строкам таблицы
3. Внутри цикла проверка условия IF
4. Выполнение действий для удовлетворяющих условию записей
5. Завершение функции RETURN

 **Подсказка:**

Сначала определяем функцию, затем цикл, внутри цикла проверяем условия и выполняем действия, потом возвращаем результат.

**Ответ:**

- Объявление функции и параметров
- Запуск цикла FOR по всем строкам таблицы
- Внутри цикла проверка условия IF
- Выполнение действий для удовлетворяющих условию записей

- Завершение функции RETURN

## Шаг 14:

### Миссия 14

Расположите шаги для вычисления среднего бонуса всех сотрудников с циклом LOOP:

1. Объявление переменных для суммы и счётчика
2. Начало цикла LOOP
3. Добавление бонуса текущего сотрудника к сумме
4. Увеличение счётчика
5. Проверка условия выхода из цикла
6. Вычисление среднего и RETURN

### Подсказка:

Сначала объявляем переменные, затем суммируем в цикле, проверяем условие выхода и в конце вычисляем среднее.

### Ответ:

- Объявление переменных для суммы и счётчика
- Начало цикла LOOP
- Добавление бонуса текущего сотрудника к сумме
- Увеличение счётчика
- Проверка условия выхода из цикла
- Вычисление среднего и RETURN

## Шаг 15:

### Миссия 15

Расположите шаги для функции с вложенными IF и CASE:

1. Определение функции и параметров
2. Проверка условия IF на нулевое значение параметра
3. Использование CASE для распределения бонусов по уровням
4. Сохранение результата в переменную
5. Возврат значения с RETURN

### Подсказка:

Сначала проверяем входные данные, затем распределяем бонусы по уровням и возвращаем результат.

### Ответ:

- Определение функции и параметров
- Проверка условия IF на нулевое значение параметра
- Использование CASE для распределения бонусов по уровням
- Сохранение результата в переменную
- Возврат значения с RETURN

## Шаг 16:

### Миссия 16

Расположите шаги для функции с вложенным циклом FOR и WHILE:

1. Определение функции и параметров

2. Внешний цикл FOR по отделам
3. Внутренний цикл WHILE для начисления бонусов сотрудникам отдела
4. Обновление таблицы с результатами
5. Возврат итогового результата

#### Подсказка:

Сначала определяем функцию, затем внешний и внутренний циклы, обновляем таблицу и возвращаем результат.

#### Ответ:

- Определение функции и параметров
- Внешний цикл FOR по отделам
- Внутренний цикл WHILE для начисления бонусов сотрудникам отдела
- Обновление таблицы с результатами
- Возврат итогового результата

#### Шаг 17:

#### Миссия 17

Расположите шаги для комплексной функции: проверка параметров, циклы, IF/CASE и обработка ошибок:

1. Определение функции и параметров
2. Проверка входных параметров IF
3. Цикл FOR для обхода сотрудников
4. Внутри цикла CASE для распределения бонусов по уровням
5. Блок EXCEPTION для обработки ошибок
6. Обновление таблицы и RETURN результата

#### Подсказка:

Функция начинается с проверки данных, затем выполняются циклы и условные операции, ошибки обрабатываются перед возвратом результата.

#### Ответ:

- Определение функции и параметров
- Проверка входных параметров IF
- Цикл FOR для обхода сотрудников
- Внутри цикла CASE для распределения бонусов по уровням
- Блок EXCEPTION для обработки ошибок
- Обновление таблицы и RETURN результата

### Урок 4 - Процедуры, возврат CURSOR

#### Шаг 01:

#### Миссия 1

Какую цель преследует использование процедуры вместо функции в PostgreSQL?

Подсказка: Подумайте, когда важно изменить данные, а не возвращать результат.

#### Ответ:

- **Выполнение операций, которые изменяют данные без возврата значения**
- Создание только вычисляемого результата без изменения данных
- Автоматическое создание индексов

- Настройка соединений с внешними базами данных

## Шаг 02:

### Миссия 2

Что важно помнить при возврате CURSOR из процедуры?

Подсказка: CURSOR не содержит сразу все строки, он даёт доступ по частям.

**Ответ:**

- **Открыть CURSOR перед чтением и закрыть после использования**
- CURSOR сразу возвращает все строки в памяти
- CURSOR может использоваться только в функциях, а не процедурах
- CURSOR автоматически сохраняет изменения в таблицах

## Шаг 03:

### Миссия 3

Когда удобнее использовать параметризованный CURSOR?

Подсказка: Подумайте о фильтрах для выборки данных.

**Ответ:**

- **Чтобы выбирать только нужные строки в зависимости от входного параметра**
- Чтобы CURSOR возвращал структуру таблицы, но без строк
- Чтобы автоматически создавать индексы по параметрам
- Чтобы CURSOR мог работать только с одной таблицей

## Шаг 04:

### Миссия 4

 Сопоставьте понятия с их описанием для работы с CURSOR

**Ответ:**

- OPEN CURSOR
- FETCH
- CLOSE CURSOR
- FOR LOOP по CURSOR

1. Открытие CURSOR для чтения строк
2. Извлечение одной или нескольких строк из CURSOR
3. Закрытие CURSOR после использования
4. Автоматический проход по всем строкам CURSOR

Подсказка: Подумайте, что делается сначала, что после, и что облегчает проход по всем строкам.

- **OPEN CURSOR** — Открытие CURSOR для чтения строк
- **FETCH** — Извлечение одной или нескольких строк из CURSOR
- **CLOSE CURSOR** — Закрытие CURSOR после использования
- **FOR LOOP по CURSOR** — Автоматический проход по всем строкам CURSOR

## Шаг 05:

### Миссия 5

 Сопоставьте типы CURSOR с их особенностями:

**Ответ:**

- STATIC CURSOR
- DYNAMIC CURSOR
- SCROLL CURSOR
- INSENSITIVE CURSOR

1. Не реагирует на изменения данных после открытия
2. Позволяет перемещаться вперед и назад по результатам
3. Обновляется при изменениях таблицы
4. Фиксирует результат на момент открытия

Подсказка: Подумайте, какой курсор даёт фиксированные результаты, а какой реагирует на изменения.

- **STATIC CURSOR** — Фиксирует результат на момент открытия
- **DYNAMIC CURSOR** — Обновляется при изменениях таблицы
- **SCROLL CURSOR** — Позволяет перемещаться вперед и назад по результатам
- **INSENSITIVE CURSOR** — Не реагирует на изменения данных после открытия

 **Шаг 06:**

**Миссия 6**

Расположите шаги для процедуры с CURSOR, которая возвращает список сотрудников отдела:

1. Определить процедуру с параметром отдела
2. Объявить CURSOR для выборки сотрудников
3. Открыть CURSOR
4. Использовать FETCH для чтения строк
5. Закрыть CURSOR
6. RETURN результата (если нужно)

Подсказка: Сначала объявление, потом открытие и чтение, в конце закрытие.

**Ответ:**

- Определить процедуру с параметром отдела
- Объявить CURSOR для выборки сотрудников
- Открыть CURSOR
- Использовать FETCH для чтения строк
- Закрыть CURSOR
- RETURN результата (если нужно)

 **Шаг 07:**

**Миссия 7**

Расположите шаги для процедуры с вложенным циклом FOR и FETCH CURSOR:

1. Объявление CURSOR для заказов
2. Открытие CURSOR
3. Цикл FOR для обработки строк
4. Обработка каждой строки (например, начисление бонуса)
5. Закрытие CURSOR

Подсказка: Помните: объявление, открытие, цикл обработки, закрытие.

**Ответ:**

- Объявление CURSOR для заказов
- Открытие CURSOR
- Цикл FOR для обработки строк
- Обработка каждой строки (например, начисление бонуса)
- Закрытие CURSOR

 **Шаг 08:****Миссия 8**

Расположите шаги использования параметризированного CURSOR в процедуре:

1. Создать процедуру с входным параметром
2. Объявить CURSOR с фильтром по параметру
3. Открыть CURSOR
4. Использовать FETCH или FOR LOOP для получения данных
5. Закрыть CURSOR

Подсказка: Сначала параметр, потом CURSOR с фильтром, затем чтение и закрытие.

**Ответ:**

- Создать процедуру с входным параметром
- Объявить CURSOR с фильтром по параметру
- Открыть CURSOR
- Использовать FETCH или FOR LOOP для получения данных
- Закрыть CURSOR

 **Шаг 09:****Миссия 9**

Расположите шаги обработки ошибок при работе с CURSOR:

1. BEGIN блока процедуры
2. Открытие CURSOR
3. FETCH данных
4. EXCEPTION блок для ловли ошибок
5. Закрытие CURSOR
6. RETURN результата

Подсказка: Сначала блок BEGIN, затем основная логика, потом обработка ошибок и закрытие CURSOR.

**Ответ:**

- BEGIN блока процедуры
- Открытие CURSOR
- FETCH данных
- EXCEPTION блок для ловли ошибок
- Закрытие CURSOR
- RETURN результата

 **Шаг 10:****Миссия 10**

Расположите шаги для процедуры с несколькими CURSOR и вложенными циклами:

1. Объявление всех CURSOR
2. Открытие первого CURSOR
3. Вложенный цикл обработки второго CURSOR
4. Обработка данных
5. Заккрытие всех CURSOR
6. RETURN результата

Подсказка: Сначала объявляем, затем открываем, обрабатываем вложенные CURSOR, закрываем и возвращаем.

**Ответ:**

- Объявление всех CURSOR
- Открытие первого CURSOR
- Вложенный цикл обработки второго CURSOR
- Обработка данных
- Заккрытие всех CURSOR
- RETURN результата

### Шаг 11:

#### **Миссия 11**

Расположите шаги процедуры с динамическим CURSOR:

1. Создать SQL-строку с параметрами
2. Объявить CURSOR FOR EXECUTE SQL-строки
3. Открыть CURSOR
4. FETCH данных
5. Закрыть CURSOR
6. RETURN результата

Подсказка: Сначала динамический SQL, затем CURSOR и чтение.

**Ответ:**

- Создать SQL-строку с параметрами
- Объявить CURSOR FOR EXECUTE SQL-строки
- Открыть CURSOR
- FETCH данных
- Закрыть CURSOR
- RETURN результата

### Шаг 12:

#### **Миссия 12**

Расположите шаги процедуры для обработки большого объёма данных с CURSOR:

1. Объявление CURSOR
2. Открытие CURSOR
3. Цикл FETCH порциями (например, 100 строк за раз)
4. Обработка каждой порции
5. Заккрытие CURSOR
6. RETURN результата

Подсказка: Для больших данных используйте порционное чтение, чтобы не перегружать память.

**Ответ:**

- Объявление CURSOR
- Открытие CURSOR
- Цикл FETCH порциями (например, 100 строк за раз)
- Обработка каждой порции
- Закрытие CURSOR
- RETURN результата

 **Шаг 13:**

**Миссия 13**

Расположите шаги процедуры для объединения данных из двух CURSOR:

1. Объявление CURSOR 1 и CURSOR 2
2. Открытие обоих CURSOR
3. Цикл FETCH по CURSOR 1
4. Внутренний цикл FETCH по CURSOR 2
5. Объединение данных
6. Закрытие обоих CURSOR
7. RETURN результата

Подсказка: Сначала объявление и открытие, затем вложенные циклы, потом закрытие и возврат.

**Ответ:**

- Объявление CURSOR 1 и CURSOR 2
- Открытие обоих CURSOR
- Цикл FETCH по CURSOR 1
- Внутренний цикл FETCH по CURSOR 2
- Объединение данных
- Закрытие обоих CURSOR
- RETURN результата

 **Шаг 14:**

**Миссия 14**

Расположите шаги процедуры для проверки пустого CURSOR перед чтением:

1. Объявление CURSOR
2. Открытие CURSOR
3. Проверка, есть ли строки для FETCH
4. Чтение строк с помощью FETCH
5. Закрытие CURSOR
6. RETURN результата

Подсказка: Сначала CURSOR объявляем и открываем, затем проверяем наличие данных, читаем, закрываем и возвращаем.

**Ответ:**

- Объявление CURSOR
- Открытие CURSOR

- Проверка, есть ли строки для FETCH
- Чтение строк с помощью FETCH
- Закрытие CURSOR
- RETURN результата

### Шаг 15:

#### Миссия 15

Расположите шаги процедуры для параллельного использования нескольких CURSOR:

1. Объявление всех CURSOR
2. Открытие CURSOR 1 и CURSOR 2
3. Чтение и обработка строк из CURSOR 1
4. Чтение и обработка строк из CURSOR 2
5. Закрытие всех CURSOR
6. RETURN результата

Подсказка: Сначала объявление, открытие, обработка данных по отдельности, закрытие и возврат.

#### Ответ:

- Объявление всех CURSOR
- Открытие CURSOR 1 и CURSOR 2
- Чтение и обработка строк из CURSOR 1
- Чтение и обработка строк из CURSOR 2
- Закрытие всех CURSOR
- RETURN результата

### Шаг 16:

#### Миссия 16

Расположите шаги процедуры с CURSOR и обработкой исключений:

1. BEGIN блока процедуры
2. Объявление CURSOR
3. Открытие CURSOR
4. FETCH данных в цикле
5. EXCEPTION блок для обработки ошибок
6. Закрытие CURSOR
7. RETURN результата

Подсказка: Сначала блок BEGIN и CURSOR, затем чтение, ловля ошибок, закрытие и возврат.

#### Ответ:

- BEGIN блока процедуры
- Объявление CURSOR
- Открытие CURSOR
- FETCH данных в цикле
- EXCEPTION блок для обработки ошибок
- Закрытие CURSOR
- RETURN результата

## Шаг 17:

### Миссия 17

Расположите шаги процедуры с вложенными параметризованными CURSOR и динамическим SQL:

1. Создание динамического SQL с параметрами
2. Объявление CURSOR FOR EXECUTE
3. Открытие CURSOR
4. FETCH данных в цикле
5. Обработка полученных данных
6. Закрытие CURSOR
7. RETURN результата

Подсказка: Сначала формируем SQL, потом CURSOR, читаем данные, обрабатываем и закрываем.

#### Ответ:

- Создание динамического SQL с параметрами
- Объявление CURSOR FOR EXECUTE
- Открытие CURSOR
- FETCH данных в цикле
- Обработка полученных данных
- Закрытие CURSOR
- RETURN результата

## Урок 5 - Мини-босс функция для генерации кастомного отчёта

## Шаг 01:

### Миссия 1

Какой тип функции лучше использовать для генерации кастомного отчёта, который возвращает набор строк?

Подсказка: Подумайте, нужен ли одиночный результат или набор строк.

#### Ответ:

- **Функция, возвращающая SETOF RECORD**
- Функция, возвращающая BOOLEAN
- Функция, возвращающая INTEGER
- Функция, возвращающая TEXT

## Шаг 02:

### Миссия 2

Что нужно сделать, чтобы функция могла принимать параметры для фильтрации данных отчёта?

Подсказка: Подумайте, как функции PostgreSQL получают данные извне.

#### Ответ:

- **Добавить входные параметры в объявление функции**
- Использовать глобальные переменные
- Вставлять значения напрямую в SQL внутри функции
- Использовать триггер вместо параметров

### Шаг 03:

#### Миссия 3

 Сопоставьте ключевые элементы функции для генерации отчёта с их описанием:

Элементы:

Ответ:

- RETURN QUERY
- DECLARE
- LOOP
- IF ... THEN

Описание:

1. Объявление переменных для использования внутри функции
2. Условное выполнение блока кода
3. Возврат результата SQL запроса
4. Циклическая обработка данных

Подсказка: Подумайте, какая часть отвечает за возврат данных, а какая за управление потоком.

- **RETURN QUERY** — Возврат результата SQL запроса
- **DECLARE** — Объявление переменных для использования внутри функции
- **LOOP** — Циклическая обработка данных
- **IF ... THEN** — Условное выполнение блока кода

### Шаг 04:

#### Миссия 4

 Сопоставьте типы возвращаемых значений функции с их применением:

Типы:

Ответ:

- SETOF RECORD
- TABLE
- TEXT
- INTEGER

Применение:

1. Возврат одной строки с числовым результатом
2. Возврат строки текста
3. Возврат набора строк без заранее определённой структуры
4. Возврат набора строк с фиксированными колонками

Подсказка: Обратите внимание, нужен ли набор строк или единичное значение.

- **SETOF RECORD** — Возврат набора строк без заранее определённой структуры
- **TABLE** — Возврат набора строк с фиксированными колонками
- **TEXT** — Возврат строки текста
- **INTEGER** — Возврат одной строки с числовым результатом

### Шаг 05:

#### Миссия 5

Какой способ обработки нескольких строк внутри функции PostgreSQL наиболее эффективен?

Подсказка: Подумайте о количестве записей и нагрузке на сервер.

**Ответ:**

- **Использовать CURSOR и цикл FETCH**
- Создавать отдельные функции для каждой строки
- Вставлять данные в массив и возвращать его
- Использовать статический SELECT без цикла

#### Шаг 06:

##### **Миссия 6**

Что помогает сделать функцию отчёта более универсальной для разных пользователей?

Подсказка: Подумайте о гибкости входных данных.

**Ответ:**

- **Добавление параметров фильтрации и сортировки**
- Жёстко зашитые значения в SELECT
- Удаление RETURN QUERY
- Прямое использование глобальных переменных

#### Шаг 07:

##### **Миссия 7**

 Сопоставьте шаги создания кастомного отчёта с их назначением:

Шаги:

**Ответ:**

- Объявление переменных
- Формирование SQL
- RETURN QUERY
- Обработка ошибок

Назначение:

1. Возврат результата пользователю
2. Ловля исключений и предотвращение падения функции
3. Хранение промежуточных данных
4. Создание динамического запроса с параметрами

Подсказка: Каждый шаг имеет своё логическое место в функции.

- **Объявление переменных** — Хранение промежуточных данных
- **Формирование SQL** — Создание динамического запроса с параметрами
- **RETURN QUERY** — Возврат результата пользователю
- **Обработка ошибок** — Ловля исключений и предотвращение падения функции

#### Шаг 08:

##### **Миссия 8**

Расположите этапы выполнения функции отчёта в правильном порядке:

1. Получение параметров
2. Объявление переменных

3. Формирование динамического SQL
4. Цикл обработки CURSOR
5. Возврат результата
6. Закрытие CURSOR

Подсказка: Логика начинается с параметров и заканчивается возвратом результата.

**Ответ:**

- Получение параметров
- Объявление переменных
- Формирование динамического SQL
- Цикл обработки CURSOR
- Возврат результата
- Закрытие CURSOR

#### **Шаг 09:**

##### **Миссия 9**

Какая конструкция позволяет динамически строить SQL внутри функции?

Подсказка: Думайте о тексте запроса, который можно менять в зависимости от параметров.

**Ответ:**

- **EXECUTE**
- PERFORM
- SELECT INTO
- LOOP FETCH

#### **Шаг 10:**

##### **Миссия 10**

Для чего используется CURSOR в функции генерации отчёта?

Подсказка: Подумайте о работе с большим количеством строк.

**Ответ:**

- **Построчная обработка набора результатов**
- Возврат одного значения
- Определение типа функции
- Создание таблицы

#### **Шаг 11:**

##### **Миссия 11**

 Сопоставьте действия с результатами при работе функции отчёта:

Действия:

**Ответ:**

- OPEN CURSOR
- FETCH
- CLOSE CURSOR
- RETURN QUERY

Результаты:

1. Начало обхода набора строк

2. Получение следующей строки
3. Завершение обхода и освобождение ресурсов
4. Возврат всего результата пользователю

Подсказка: Логика CURSOR идёт OPEN → FETCH → CLOSE.

- **OPEN CURSOR** — Начало обхода набора строк
- **FETCH** — Получение следующей строки
- **CLOSE CURSOR** — Завершение обхода и освобождение ресурсов
- **RETURN QUERY** — Возврат всего результата пользователю

## 🔮 Шаг 12:

### Миссия 12

📄 Сопоставьте типы функций с их подходящим использованием для отчёта:

Типы:

**Ответ:**

- IMMUTABLE
- STABLE
- VOLATILE

Использование:

1. Функции, всегда возвращающие одинаковый результат при одинаковых входных данных
2. Функции, зависящие от данных таблицы, но не изменяющие их
3. Функции, которые могут изменять данные или среду выполнения

Подсказка: Обратите внимание на побочные эффекты функций.

- **IMMUTABLE** — Функции, всегда возвращающие одинаковый результат при одинаковых входных данных
- **STABLE** — Функции, зависящие от данных таблицы, но не изменяющие их
- **VOLATILE** — Функции, которые могут изменять данные или среду выполнения

## 🔮 Шаг 13:

### Миссия 13

Какую конструкцию лучше использовать для проверки условий внутри функции отчёта?

Подсказка: Подумайте о логическом ветвлении.

**Ответ:**

- **IF ... THEN ... ELSE**
- SELECT \* FROM
- LOOP ... END LOOP
- DECLARE ...

## 🔮 Шаг 14:

### Миссия 14

Что нужно сделать, чтобы функция отчёта корректно возвращала несколько колонок?

Подсказка: Подумайте о типе возвращаемого значения и структуре SELECT.

**Ответ:**

- **Использовать RETURN QUERY SELECT с нужными колонками**

- Возвращать один текст с разделителями
- Использовать массив из чисел
- Удалить DECLARE блок

## Шаг 15:

### Миссия 15

Расположите шаги от создания функции до получения отчёта в правильной последовательности:

1. Объявление функции и входных параметров
2. Объявление переменных
3. Формирование SQL запроса
4. Использование LOOP или FETCH
5. RETURN QUERY
6. Тестирование функции

Подсказка: Логика всегда идёт от объявления до тестирования.

#### Ответ:

- Объявление функции и входных параметров
- Объявление переменных
- Формирование SQL запроса
- Использование LOOP или FETCH
- RETURN QUERY
- Тестирование функции

## Шаг 16:

### Миссия 16

Расположите этапы обработки CURSOR в функции отчёта по логике выполнения:

1. OPEN CURSOR
2. FETCH INTO переменные
3. Обработка данных внутри LOOP
4. EXIT WHEN NOT FOUND
5. CLOSE CURSOR
6. RETURN QUERY

Подсказка: Этапы следуют порядку обхода набора строк.

#### Ответ:

- OPEN CURSOR
- FETCH INTO переменные
- Обработка данных внутри LOOP
- EXIT WHEN NOT FOUND
- CLOSE CURSOR
- RETURN QUERY

## Шаг 17:

### Миссия 17

Расположите этапы создания полностью динамического отчёта с параметрами пользователя:

1. Приём параметров
2. Создание динамического SQL через EXECUTE
3. Объявление переменных и CURSOR
4. Цикл FETCH и обработка
5. Возврат результата через RETURN QUERY
6. Закрытие CURSOR
7. Тестирование функции

Подсказка: Логика начинается с параметров и завершается тестированием.

**Ответ:**

- Приём параметров
- Создание динамического SQL через EXECUTE
- Объявление переменных и CURSOR
- Цикл FETCH и обработка
- Возврат результата через RETURN QUERY
- Закрытие CURSOR
- Тестирование функции

## **Модуль 6 - Репликация, бэкапы, масштабирование**

### **Урок 1 - Logical vs physical replication**

#### **Шаг 01:**

##### **Миссия 1**

Что из перечисленного является примером физической репликации?

Подсказка: Подумайте о точной копии базы данных на другом сервере.

**Ответ:**

- **Копирование WAL и файлов базы на реплику**
- Отправка изменений через SQL INSERT/UPDATE
- Создание отчётов через функции
- Резервное копирование отдельной таблицы

#### **Шаг 02:**

##### **Миссия 2**

Какое преимущество имеет логическая репликация по сравнению с физической?

Подсказка: Подумайте о гибкости и возможности работы с отдельными таблицами.

**Ответ:**

- **Можно реплицировать отдельные таблицы или схемы**
- Требуется идентичная структура сервера
- Мгновенное восстановление всей базы
- Не поддерживает фильтры и правила трансформации

#### **Шаг 03:**

##### **Миссия 3**

Что необходимо для настройки логической репликации в PostgreSQL?

Подсказка: Подумайте о подписке и публикации изменений.

**Ответ:**

- **Создать публикацию на мастере и подписку на реплике**
- Копировать WAL напрямую
- Создать CRON-скрипт бэкапов
- Использовать `pg_dump` каждый час

#### Шаг 04:

##### Миссия 4

 Сопоставьте тип репликации с его характеристикой:

Типы:

**Ответ:**

- Physical replication
- Logical replication

Характеристики:

1. Копирует все файлы базы данных, полностью идентичная копия
2. Можно реплицировать отдельные таблицы и фильтровать данные

Подсказка: Обратите внимание на масштаб и гибкость.

- **Physical replication** — Копирует все файлы базы данных, полностью идентичная копия
- **Logical replication** — Можно реплицировать отдельные таблицы и фильтровать данные

#### Шаг 05:

##### Миссия 5

 Сопоставьте этапы настройки логической репликации:

Этапы:

**Ответ:**

- Создание публикации на мастере
- Создание подписки на реплике
- Проверка синхронизации

Описание:

1. Отправка изменений с мастера на реплику
2. Настройка, какие таблицы будут реплицироваться
3. Убедиться, что данные на реплике совпадают с мастером

Подсказка: Логика начинается с публикации и заканчивается проверкой.

- **Создание публикации на мастере** — Настройка, какие таблицы будут реплицироваться
- **Создание подписки на реплике** — Отправка изменений с мастера на реплику
- **Проверка синхронизации** — Убедиться, что данные на реплике совпадают с мастером

#### Шаг 06:

##### Миссия 6

Почему физическая репликация быстрее логической при больших объёмах данных?

Подсказка: Думайте о прямом копировании файлов, а не отдельной обработке SQL.

**Ответ:**

- **Копируются файлы WAL без анализа SQL**
- Применяется только один SQL-запрос
- Используется pg\_dump
- Данные пересылаются через CSV

 **Шаг 07:**

### **Миссия 7**

Что нужно учитывать при логической репликации с большим количеством таблиц?

Подсказка: Подумайте о нагрузке и фильтрации данных.

**Ответ:**

- **Фильтровать только нужные таблицы и колонки**
- Реплицировать все таблицы без исключений
- Применять физическую репликацию для каждой таблицы
- Использовать pg\_dump для отдельных таблиц

 **Шаг 08:**

### **Миссия 8**

 Сопоставьте тип репликации с возможностями восстановления после сбоя:

Типы:

**Ответ:**

- Physical
- Logical

Возможности:

1. Мгновенное переключение на реплику с точной копией
2. Возможность восстановить отдельные таблицы или фильтровать изменения

Подсказка: Подумайте о granularности и скорости.

- **Physical** — Мгновенное переключение на реплику с точной копией
- **Logical** — Возможность восстановить отдельные таблицы или фильтровать изменения

 **Шаг 09:**

### **Миссия 9**

Расположите шаги настройки физической репликации:

1. Настройка WAL на мастере
2. Создание слота репликации
3. Запуск реплики
4. Проверка синхронизации

Подсказка: Логика всегда от конфигурации WAL к запуску реплики.

**Ответ:**

- Настройка WAL на мастере
- Создание слота репликации
- Запуск реплики

- Проверка синхронизации

## Шаг 10:

### Миссия 10

Расположите этапы логической репликации с подпиской:

1. Создание публикации
2. Настройка публикации с таблицами
3. Создание подписки
4. Применение изменений на реплике
5. Мониторинг синхронизации

Подсказка: Начните с публикации и завершите мониторингом.

**Ответ:**

- Создание публикации
- Настройка публикации с таблицами
- Создание подписки
- Применение изменений на реплике
- Мониторинг синхронизации

## Шаг 11:

### Миссия 11

Что будет, если репликация остановится на несколько часов?

Подсказка: Подумайте о накоплении WAL и задержках данных.

**Ответ:**

- **На реплике накопятся WAL, и данные будут отставать**
- Данные на реплике удалятся
- Мастер перестанет работать
- Все транзакции откатятся

## Шаг 12:

### Миссия 12

Какой вид репликации позволяет использовать реплику для чтения запросов?

Подсказка: Подумайте о разгрузке мастера.

**Ответ:**

- **Both physical (hot standby) and logical**
- Только физическая
- Только логическая
- Ни один

## Шаг 13:

### Миссия 13

 Сопоставьте тип реплики с её назначением:

Типы:

**Ответ:**

- Hot standby

- Logical subscriber

Назначение:

1. Используется для чтения и резервного переключения, поддерживая физическую идентичность базы
2. Позволяет получать обновления отдельных таблиц и применять фильтры данных

Подсказка: Обратите внимание на различия в granularity данных.

- **Hot standby** — Используется для чтения и резервного переключения, поддерживая физическую идентичность базы
- **Logical subscriber** — Позволяет получать обновления отдельных таблиц и применять фильтры данных

#### Шаг 14:

##### Миссия 14

Что произойдёт, если в логической репликации добавится новая таблица без обновления публикации?

Подсказка: Подумайте, какие объекты реально отслеживаются публикацией.

**Ответ:**

- **Новая таблица не будет реплицироваться**
- Все реплики обновятся автоматически
- Мастер перестанет работать
- Логическая репликация перейдёт на физическую

#### Шаг 15:

##### Миссия 15

Расположите шаги восстановления реплики после сбоя:

1. Проверка WAL на мастере
2. Перезапуск реплики
3. Применение накопленных WAL
4. Мониторинг синхронизации

Подсказка: Сначала проверяем журналы, потом восстанавливаем реплику, и в конце контролируем синхронизацию.

**Ответ:**

- Проверка WAL на мастере
- Перезапуск реплики
- Применение накопленных WAL
- Мониторинг синхронизации

#### Шаг 16:

##### Миссия 16

Расположите действия по настройке логической репликации для фильтрации изменений только одной таблицы:

1. Создать публикацию с указанием конкретной таблицы
2. Создать подписку на реплике
3. Применить изменения на реплике

#### 4. Проверить синхронизацию

Подсказка: Сначала публикуем, потом подписываем, далее применяем изменения и контролируем результат.

**Ответ:**

- Создать публикацию с указанием конкретной таблицы
- Создать подписку на реплике
- Применить изменения на реплике
- Проверить синхронизацию

## Урок 2 - Настройка реплики

### Шаг 01:

#### Миссия 1

Какая команда используется для запуска физической реплики PostgreSQL?

Подсказка: Подумайте о том, как стартует сервис базы данных на вторичном сервере.

**Ответ:**

- **pg\_ctl start -D /var/lib/postgresql/data**
- pg\_restore
- psql -U postgres
- CREATE REPLICA DATABASE

### Шаг 02:

#### Миссия 2

Что обязательно нужно настроить на мастере для работы реплики?

Подсказка: Подумайте, какие параметры отвечают за доступ реплик к WAL.

**Ответ:**

- **wal\_level и max\_wal\_senders**
- shared\_buffers
- work\_mem
- maintenance\_work\_mem

### Шаг 03:

#### Миссия 3

Что из перечисленного является преимуществом физической реплики?

Подсказка: Подумайте о полном копировании базы данных.

**Ответ:**

- **Полная копия базы, синхронная или асинхронная**
- Поддержка только отдельных таблиц
- Не требует настройки wal\_level
- Можно изменять данные напрямую на реплике

### Шаг 04:

#### Миссия 4

 Сопоставьте тип реплики с её характеристикой:

Типы:

**Ответ:**

- Синхронная реплика
- Асинхронная реплика

Характеристики:

1. Подтверждение записи транзакции ждет реплики
2. Мастер продолжает работу, не дожидаясь подтверждения

Подсказка: Подумайте, какой режим обеспечивает строгую консистентность.

- **Синхронная реплика** — Подтверждение записи транзакции ждет реплики
- **Асинхронная реплика** — Мастер продолжает работу, не дожидаясь подтверждения

## Шаг 05:

### Миссия 5

 Сопоставьте команды с их назначением при настройке реплики:

Команды:

**Ответ:**

- pg\_basebackup
- psql CREATE SUBSCRIPTION

Назначение:

1. Создает полную копию базы с мастера на реплику
2. Подписывает реплику на публикацию мастера

Подсказка: Обратите внимание, какая команда отвечает за копирование данных, а какая — за связь реплик.

- **pg\_basebackup** — Создает полную копию базы с мастера на реплику
- **psql CREATE SUBSCRIPTION** — Подписывает реплику на публикацию мастера

## Шаг 06:

### Миссия 6

 Сопоставьте параметры конфигурации с их ролью:

Параметры:

**Ответ:**

- primary\_conninfo
- restore\_command

Роль:

1. Содержит информацию для подключения реплики к мастеру
2. Команда для восстановления WAL файлов при отставании реплики

Подсказка: Один параметр отвечает за соединение, другой — за обработку WAL.

- **primary\_conninfo** — Содержит информацию для подключения реплики к мастеру
- **restore\_command** — Команда для восстановления WAL файлов при отставании реплики

## 🔧 Шаг 07:

### Миссия 7

Что произойдет, если настроить реплику с неправильным `primary_conninfo`?

Подсказка: Подумайте, как реплика соединяется с мастером.

**Ответ:**

- **Реплика не сможет подключиться к мастеру**
- Реплика синхронизируется автоматически
- Мастер перестанет работать
- Реплика станет мастером

## 🔧 Шаг 08:

### Миссия 8

Что важно сделать перед запуском реплики на новом сервере?

Подсказка: Подумайте о подготовке данных и конфигурации.

**Ответ:**

- **Скопировать базу мастера через `pg_basebackup` и настроить конфигурацию**
- Создать пустую базу и подключить реплику
- Просто запустить сервис PostgreSQL
- Изменить все пароли на реплике

## 🔧 Шаг 09:

### Миссия 9

📄 Сопоставьте роли файлов WAL с их использованием:

Файлы:

**Ответ:**

- `pg_wal`
- `archive WAL`

Использование:

1. Хранение текущих WAL файлов для реплики
2. Резервное копирование и восстановление при сбое

Подсказка: Обратите внимание на локальное хранение и архивирование.

- **`pg_wal`** — Хранение текущих WAL файлов для реплики
- **`archive WAL`** — Резервное копирование и восстановление при сбое

## 🔧 Шаг 10:

### Миссия 10

📄 Сопоставьте тип репликации с его возможностью:

Типы:

**Ответ:**

- Физическая реплика
- Логическая реплика

Возможности:

1. Полная копия базы, не поддерживает выборочные таблицы
2. Можно реплицировать отдельные таблицы и фильтровать данные

Подсказка: Один тип строго копирует все данные, другой гибко управляет таблицами.

- **Физическая реплика** — Полная копия базы, не поддерживает выборочные таблицы
- **Логическая реплика** — Можно реплицировать отдельные таблицы и фильтровать данные

## Шаг 11:

### Миссия 11

Как проверить, что реплика синхронизирована с мастером?

Подсказка: Подумайте о системных представлениях для репликации.

**Ответ:**

- **SELECT \* FROM pg\_stat\_replication;**
- SELECT version();
- SHOW wal\_level;
- pg\_ctl status

## Шаг 12:

### Миссия 12

Какая команда позволит проверить статус реплики и задержку по WAL?

Подсказка: Ищите команду в системных представлениях PostgreSQL.

**Ответ:**

- **SELECT \* FROM pg\_stat\_replication;**
- SELECT \* FROM pg\_stat\_activity;
- SHOW max\_wal\_senders;
- pg\_ctl status

## Шаг 13:

### Миссия 13

 Сопоставьте шаги при создании логической реплики:

Шаги:

**Ответ:**

- Создать публикацию на мастере
- Создать подписку на реплике
- Проверить синхронизацию

Описание:

1. Отправка изменений с мастера
2. Подключение реплики к публикации
3. Убедиться, что данные совпадают

Подсказка: Подумайте о порядке создания публикации и подписки.

- **Создать публикацию на мастере** — Отправка изменений с мастера

- **Создать подписку на реплике** — Подключение реплики к публикации
- **Проверить синхронизацию** — Убедиться, что данные совпадают

#### Шаг 14:

##### **Миссия 14**

Что важно учесть при настройке реплики на удаленном сервере с низкой скоростью сети?

Подсказка: Подумайте о задержках и устойчивости соединения.

**Ответ:**

- **Использовать асинхронную репликацию и архивирование WAL**
- Настроить синхронную реплику без WAL
- Запускать реплику только ночью
- Отключить реплику на мастере

#### Шаг 15:

##### **Миссия 15**

Расположите шаги настройки физической реплики в правильном порядке:

1. Остановить реплику и удалить старые данные
2. Скопировать базу мастера через pg\_basebackup
3. Настроить параметры recovery.conf / standby.signal
4. Запустить реплику и проверить подключение

Подсказка: Подумайте о логическом порядке подготовки, копирования и запуска.

**Ответ:**

- Остановить реплику и удалить старые данные
- Скопировать базу мастера через pg\_basebackup
- Настроить параметры recovery.conf / standby.signal
- Запустить реплику и проверить подключение

#### Шаг 16:

##### **Миссия 16**

Расположите шаги устранения проблем с задержкой реплики:

1. Проверка нагрузки на мастер и реплику
2. Анализ состояния WAL и задержки
3. Корректировка max\_wal\_senders и wal\_keep\_size
4. Повторная проверка синхронизации

Подсказка: Сначала диагностируем проблему, потом исправляем конфигурацию, затем проверяем результат.

**Ответ:**

- Проверка нагрузки на мастер и реплику
- Анализ состояния WAL и задержки
- Корректировка max\_wal\_senders и wal\_keep\_size
- Повторная проверка синхронизации

## Урок 3 - pg\_dump, pg\_restore, pgBackRest

### Шаг 01:

#### Миссия 1

Что делает команда pg\_dump в PostgreSQL?

Подсказка: Это инструмент для создания резервной копии базы данных.

**Ответ:**

- **Создаёт резервную копию базы данных**
- Восстанавливает базу данных из архива
- Обновляет версию PostgreSQL
- Создает реплику базы данных

### Шаг 02:

#### Миссия 2

Какой формат резервной копии позволяет выбрать определённые таблицы при восстановлении?

Подсказка: Обратите внимание на форматы tar и custom.

**Ответ:**

- **Custom (c)**
- Plain SQL
- Directory
- CSV

### Шаг 03:

#### Миссия 3

Что делает pg\_restore?

Подсказка: Вспомните, что pg\_dump создаёт резервную копию, а pg\_restore — ?

**Ответ:**

- **Восстанавливает базу данных из резервной копии**
- Создает резервную копию
- Настраивает реплику
- Удаляет данные из базы

### Шаг 04:

#### Миссия 4

 Сопоставьте команды с их назначением.

Команды:

**Ответ:**

- pg\_dump
- pg\_restore
- pgBackRest
- psql

Назначение:

1. Выполняет SQL-команды в базе данных

2. Создаёт резервную копию базы
3. Восстанавливает базу из резервной копии
4. Продвинутое управление резервными копиями с бэкапами и архивами WAL

Подсказка: Подумайте, кто для копирования, кто для восстановления и кто для продвинутого бэкапа.

- **pg\_dump** — Создаёт резервную копию базы
- **pg\_restore** — Восстанавливает базу из резервной копии
- **pgBackRest** — Продвинутое управление резервными копиями с бэкапами и архивами WAL
- **psql** — Выполняет SQL-команды в базе данных

## 🔮 Шаг 05:

### Миссия 5

📖 Сопоставьте типы резервного копирования pgBackRest с их особенностями.

Типы:

Ответ:

- Full
- Incremental
- Differential

Описание:

1. Копирует все данные базы
2. Копирует только изменения с момента последнего полного бэкапа
3. Копирует изменения с последнего бэкапа любого типа

Подсказка: Full — всегда всё, incremental — только новые изменения, differential — изменения с последнего полного.

- **Full** — Копирует все данные базы
- **Incremental** — Копирует только изменения с момента последнего полного бэкапа
- **Differential** — Копирует изменения с последнего бэкапа любого типа

## 🔮 Шаг 06:

### Миссия 6

Какой ключ позволяет pg\_dump создать архив в формате custom?

Подсказка: Формат указывается опцией -F.

Ответ:

- **-F c**
- -f c
- --format=plain
- -C

## 🔮 Шаг 07:

### Миссия 7

Что важно учитывать при восстановлении базы с pg\_restore в другую схему?

Подсказка: Обратите внимание на параметры -n и --schema.

**Ответ:**

- **Указать целевую схему через -n или --schema**
- Обязательно использовать plain SQL
- Не нужно указывать схему, pg\_restore выберет автоматически
- Нельзя восстанавливать в другую схему

## Шаг 08:

### Миссия 8

 Сопоставьте опции pgBackRest с их назначением.

**Ответ:**

- --stanza
  - --type
  - --repo1-path
  - --db-path
1. Имя конфигурации бэкапа
  2. Тип резервного копирования
  3. Путь к хранилищу репозитория
  4. Путь к базе данных PostgreSQL

Подсказка: Каждая опция имеет конкретное назначение для резервирования и восстановления.

- **--stanza** — Имя конфигурации бэкапа
- **--type** — Тип резервного копирования
- **--repo1-path** — Путь к хранилищу репозитория
- **--db-path** — Путь к базе данных PostgreSQL

## Шаг 09:

### Миссия 9

Какой инструмент позволяет автоматизировать резервное копирование с поддержкой WAL и сжатия?

Подсказка: Это не pg\_dump, а более продвинутое решение.

**Ответ:**

- **pgBackRest**
- pg\_restore
- psql
- pg\_dump

## Шаг 10:

### Миссия 10

Что нужно сделать перед восстановлением резервной копии, если база существует?

Подсказка: Подумайте, что будет при восстановлении на существующую базу без подготовки.

**Ответ:**

- **Удалить или переименовать существующую базу**
- Ничего не нужно, pg\_restore перезапишет автоматически
- Просто выполнить pg\_restore без параметров
- Создать дополнительную реплику

## Шаг 11:

### Миссия 11

 Сопоставьте форматы резервных копий `pg_dump` с их характеристиками.

**Ответ:**

- Plain SQL
  - Custom
  - Directory
1. Текстовый файл с SQL-командами
  2. Выборочное восстановление таблиц и схем
  3. Директория с отдельными файлами объектов базы

Подсказка: Подумайте, какой формат удобен для простого чтения и какой для гибкого восстановления.

- **Plain SQL** — Текстовый файл с SQL-командами
- **Custom** — Выборочное восстановление таблиц и схем
- **Directory** — Директория с отдельными файлами объектов базы

## Шаг 12:

### Миссия 12

 Сопоставьте команды `pg_restore` с их функционалом.

**Ответ:**

- -l
  - -L
  - -j
  - -n
1. Выводит список объектов из резервной копии
  2. Выполняет восстановление по списку объектов
  3. Восстановление с параллельными потоками
  4. Указывает схему для восстановления

Подсказка: Каждая опция имеет своё назначение — внимательно читайте документацию.

- **-l** — Выводит список объектов из резервной копии
- **-L** — Выполняет восстановление по списку объектов
- **-j** — Восстановление с параллельными потоками
- **-n** — Указывает схему для восстановления

## Шаг 13:

### Миссия 13

Расположите шаги восстановления базы из резервной копии в правильном порядке:

1. Выполнить `pg_restore` с указанием базы и формата архива
2. Подготовить базу (создать или удалить существующую)
3. Проверить целостность резервной копии
4. При необходимости указать схему и параллельные потоки

Подсказка: Сначала проверяем копию, потом готовим базу, потом настраиваем параметры и выполняем восстановление.

**Ответ:**

- Проверить целостность резервной копии
- Подготовить базу (создать или удалить существующую)
- При необходимости указать схему и параллельные потоки
- Выполнить `pg_restore` с указанием базы и формата архива

 **Шаг 14:**

**Миссия 14**

Расположите шаги создания резервной копии с помощью `pgBackRest`:

1. Настроить stanza и пути к репозиторию
2. Выполнить полный бэкап (full)
3. Выполнять периодические incremental или differential бэкапы
4. Проверить бэкап и журнал WAL

Подсказка: Подумайте о логике: настройка → полный бэкап → регулярные инкременты → проверка.

**Ответ:**

- Настроить stanza и пути к репозиторию
- Выполнить полный бэкап (full)
- Выполнять периодические incremental или differential бэкапы
- Проверить бэкап и журнал WAL

 **Шаг 15:**

**Миссия 15**

Расположите шаги восстановления базы с `pgBackRest`:

1. Выбрать stanza и репозиторий
2. Указать тип восстановления (full, incremental)
3. Выполнить восстановление
4. Проверить целостность данных после восстановления

Подсказка: Восстановление начинается с выбора конфигурации, потом тип бэкапа, затем восстановление и проверка.

**Ответ:**

- Выбрать stanza и репозиторий
- Указать тип восстановления (full, incremental)
- Выполнить восстановление
- Проверить целостность данных после восстановления

 **Урок 4 - Шардирование и масштабирование (вводная)**

 **Шаг 01:**

**Миссия 1**

Андрей запускает свой первый крупный проект на PostgreSQL. Он слышал о шардировании, но пока не понимает, зачем оно нужно. Что является основной целью шардирования?

Подсказка: Подумайте о росте базы и как можно разделить нагрузку.

**Ответ:**

- **Разделение данных на несколько серверов для увеличения производительности**
- Добавление индексов для ускорения запросов
- Создание резервной копии базы
- Использование функций и триггеров для автоматизации задач

## Шаг 02:

### **Миссия 2**

Катя заметила, что один сервер PostgreSQL не справляется с нагрузкой. Какой подход поможет масштабировать систему горизонтально?

Подсказка: Горизонтальное масштабирование — это добавление серверов, а не улучшение одного.

**Ответ:**

- **Добавление новых серверов и распределение данных между ними**
- Увеличение RAM на существующем сервере
- Оптимизация SQL-запросов
- Использование более быстрых дисков для одного сервера

## Шаг 03:

### **Миссия 3**

Иван хочет реализовать шардирование для таблицы пользователей. Какой критерий обычно используют для распределения данных по шардам?

Подсказка: Подумайте, что у всех пользователей есть уникальный идентификатор.

**Ответ:**

- **По хешу уникального идентификатора пользователя**
- По имени пользователя
- По дате регистрации
- По IP-адресу сервера

## Шаг 04:

### **Миссия 4**

Саша создает резервные копии базы PostgreSQL. Что важно помнить при масштабировании и шардировании?

Подсказка: Бэкапы должны работать независимо для каждого шарда.

**Ответ:**

- **Создавать бэкапы каждого шарда отдельно**
- Делать один бэкап всей системы на одном сервере
- Бэкап не нужен, если есть репликация
- Можно просто копировать файлы данных

## Шаг 05:

### **Миссия 5**

 Сопоставьте понятия шардирования с их описанием.

Понятия:

**Ответ:**

- Шард
- Шардирование
- Горизонтальное масштабирование
- Вертикальное масштабирование

Определения:

1. Раздел базы данных на независимые части, каждая из которых хранится на отдельном сервере.
2. Добавление ресурсов (CPU, RAM) на один сервер для увеличения производительности.
3. Отдельная часть базы данных, размещенная на отдельном сервере.
4. Добавление новых серверов для распределения нагрузки.

 **Подсказка:**

Сравните термины: горизонтальное и вертикальное масштабирование решают разные задачи.

- **Шард** — Отдельная часть базы данных, размещенная на отдельном сервере.
- **Шардирование** — Раздел базы данных на независимые части, каждая из которых хранится на отдельном сервере.
- **Горизонтальное масштабирование** — Добавление новых серверов для распределения нагрузки.
- **Вертикальное масштабирование** — Добавление ресурсов (CPU, RAM) на один сервер для увеличения производительности.

 **Шаг 06:**

**Миссия 6**

 Сопоставьте типы репликации с их характеристиками.

**Ответ:**

- Синхронная репликация
  - Асинхронная репликация
  - Физическая репликация
  - Логическая репликация
1. Данные записываются на вторичный сервер одновременно с основным.
  2. Данные на вторичном сервере могут отставать от основного.
  3. Копируются физические файлы базы данных.
  4. Копируются изменения на уровне таблиц или строк.

 **Подсказка:**

Обратите внимание: физическая/логическая репликация решают разные задачи, синхронная/асинхронная — вопрос задержки.

- **Синхронная репликация** — Данные записываются на вторичный сервер одновременно с основным.
- **Асинхронная репликация** — Данные на вторичном сервере могут отставать от основного.
- **Физическая репликация** — Копируются физические файлы базы данных.
- **Логическая репликация** — Копируются изменения на уровне таблиц или строк.

 **Шаг 07:**

**Миссия 7**

 Сопоставьте термины масштабирования с примерами.

**Ответ:**

- Шардирование
  - Репликация
  - Балансировка нагрузки
  - Кэширование
1. Создание нескольких серверов с одинаковыми данными для распределения чтения
  2. Разделение таблицы заказов на 3 сервера по ID заказов
  3. Использование Redis для ускорения повторяющихся запросов
  4. Использование прокси-сервера для распределения запросов между серверами

 **Подсказка:**

Подумайте, какая задача каждого подхода: шардирование — деление данных, кэш — ускорение, репликация — резерв, балансировка — распределение.

- **Шардирование** — Разделение таблицы заказов на 3 сервера по ID заказов
- **Репликация** — Создание нескольких серверов с одинаковыми данными для распределения чтения
- **Балансировка нагрузки** — Использование прокси-сервера для распределения запросов между серверами
- **Кэширование** — Использование Redis для ускорения повторяющихся запросов

 **Шаг 08:**

### Миссия 8

 Сопоставьте термины из практики масштабирования с их функцией.

**Ответ:**

- PgBouncer
  - Patroni
  - pgBackRest
  - HAProxy
1. Балансировка соединений PostgreSQL между серверами
  2. Автоматическое управление высокодоступным кластером PostgreSQL
  3. Инструмент для резервного копирования PostgreSQL
  4. Менеджер пулов соединений PostgreSQL

 **Подсказка:**

Обратите внимание на роли инструментов: репликация, бэкапы, балансировка, пул соединений — все разные задачи.

- **PgBouncer** — Менеджер пулов соединений PostgreSQL
- **Patroni** — Автоматическое управление высокодоступным кластером PostgreSQL
- **pgBackRest** — Инструмент для резервного копирования PostgreSQL
- **HAProxy** — Балансировка соединений PostgreSQL между серверами

 **Шаг 09:**

### Миссия 9

Расположите шаги шардирования базы данных в правильном порядке:

1. Определение критерия разделения данных (например, ID пользователя)

2. Создание отдельных серверов для хранения шардов
3. Распределение данных между шардами
4. Настройка приложения на работу с шардированной базой

 **Подсказка:**

Подумайте, что сначала нужно определить, потом подготовить инфраструктуру, затем распределить данные и только после этого настроить приложение.

**Ответ:**

- Определение критерия разделения данных (например, ID пользователя)
- Создание отдельных серверов для хранения шардов
- Распределение данных между шардами
- Настройка приложения на работу с шардированной базой

 **Шаг 10:**

**Миссия 10**

Расположите шаги создания высокодоступного кластера PostgreSQL:

1. Настройка репликации между мастер- и слейв-серверами
2. Установка и конфигурация Patroni
3. Тестирование отказоустойчивости
4. Настройка мониторинга кластера

 **Подсказка:**

Сначала настраивается база, затем кластер, после — тесты и мониторинг.

**Ответ:**

- Настройка репликации между мастер- и слейв-серверами
- Установка и конфигурация Patroni
- Тестирование отказоустойчивости
- Настройка мониторинга кластера

 **Шаг 11:**

**Миссия 11**

Расположите шаги для безопасного бэкапа шардированной базы PostgreSQL:

1. Определение всех серверов и шардов
2. Создание резервных копий каждого шарда с помощью pgBackRest
3. Проверка целостности и восстановления из бэкапа
4. Документирование процедуры и расписания бэкапов

 **Подсказка:**

Сначала нужно идентифицировать источники, затем выполнить бэкап, проверить и зафиксировать процесс.

**Ответ:**

- Определение всех серверов и шардов
- Создание резервных копий каждого шарда с помощью pgBackRest
- Проверка целостности и восстановления из бэкапа
- Документирование процедуры и расписания бэкапов

## 🔮 Шаг 12:

### Миссия 12

Расположите этапы внедрения кэширования для масштабируемой базы данных:

1. Выбор данных для кэширования
2. Настройка Redis или Memcached
3. Изменение приложения для использования кэша
4. Мониторинг и оптимизация кэш-памяти

### 🔮 Подсказка:

Сначала решаем что кэшировать, затем настраиваем, потом интегрируем и мониторим.

### Ответ:

- Выбор данных для кэширования
- Настройка Redis или Memcached
- Изменение приложения для использования кэша
- Мониторинг и оптимизация кэш-памяти

## 🔮 Шаг 13:

### Миссия 13

Расположите шаги настройки логической репликации PostgreSQL:

1. Создание публикации на основном сервере
2. Создание подписки на реплицирующем сервере
3. Проверка синхронизации таблиц
4. Мониторинг и устранение конфликтов

### 🔮 Подсказка:

Сначала создаем публикацию, потом подписку, проверяем данные и следим за возможными конфликтами.

### Ответ:

- Создание публикации на основном сервере
- Создание подписки на реплицирующем сервере
- Проверка синхронизации таблиц
- Мониторинг и устранение конфликтов

## 🔮 Шаг 14:

### Миссия 14

Расположите шаги для внедрения HAProxy для балансировки нагрузки PostgreSQL:

1. Установка HAProxy на отдельный сервер
2. Настройка backend-серверов PostgreSQL
3. Конфигурация правил распределения запросов
4. Тестирование работы балансировщика

### 🔮 Подсказка:

Важно сначала установить HAProxy, затем подключить серверы и настроить правила, после чего тестировать.

### Ответ:

- Установка HAProxy на отдельный сервер

- Настройка backend-серверов PostgreSQL
- Конфигурация правил распределения запросов
- Тестирование работы балансировщика

## Шаг 15:

### Миссия 15

Расположите шаги масштабирования чтения в PostgreSQL с помощью репликации:

1. Создание слейв-серверов
2. Настройка асинхронной репликации
3. Перенаправление запросов на чтение к слейвам
4. Мониторинг задержки репликации

### Подсказка:

Сначала создаем слейвы, настраиваем репликацию, распределяем чтение и следим за задержкой.

### Ответ:

- Создание слейв-серверов
- Настройка асинхронной репликации
- Перенаправление запросов на чтение к слейвам
- Мониторинг задержки репликации

## Шаг 16:

### Миссия 16

Расположите шаги для безопасного масштабирования базы с помощью шардирования и кэша:

1. Определение горячих и холодных данных
2. Распределение данных по шардированной инфраструктуре
3. Настройка кэша для горячих данных
4. Мониторинг производительности и корректировка стратегии

### Подсказка:

Сначала анализируем данные, затем распределяем по шардам, добавляем кэш и следим за результатом.

### Ответ:

- Определение горячих и холодных данных
- Распределение данных по шардированной инфраструктуре
- Настройка кэша для горячих данных
- Мониторинг производительности и корректировка стратегии

## Шаг 17:

### Миссия 17

Расположите шаги комплексного масштабирования PostgreSQL, объединяющего шардирование, репликацию и кэш:

1. Проектирование архитектуры с учетом шардирования
2. Настройка репликации для каждого шарда
3. Интеграция кэша для горячих данных
4. Тестирование, мониторинг и оптимизация всей системы

### **Подсказка:**

Планируем архитектуру, настраиваем репликацию, добавляем кэш и только после этого проводим комплексное тестирование.

### **Ответ:**

- Проектирование архитектуры с учетом шардирования
- Настройка репликации для каждого шарда
- Интеграция кэша для горячих данных
- Тестирование, мониторинг и оптимизация всей системы

## **Урок 5 - Написать план резервного копирования и восстановления БД**

### **Шаг 01:**

#### **Миссия 1**

Андрей запускает свой первый крупный проект на PostgreSQL. Он слышал о шардировании, но пока не понимает, зачем оно нужно. Что является основной целью шардирования?

Подсказка: Подумайте о росте базы и как можно разделить нагрузку.

### **Ответ:**

- **Разделение данных на несколько серверов для увеличения производительности**
- Добавление индексов для ускорения запросов
- Создание резервной копии базы
- Использование функций и триггеров для автоматизации задач

### **Шаг 02:**

#### **Миссия 2**

Катя заметила, что один сервер PostgreSQL не справляется с нагрузкой. Какой подход поможет масштабировать систему горизонтально?

Подсказка: Горизонтальное масштабирование — это добавление серверов, а не улучшение одного.

### **Ответ:**

- **Добавление новых серверов и распределение данных между ними**
- Увеличение RAM на существующем сервере
- Оптимизация SQL-запросов
- Использование более быстрых дисков для одного сервера

### **Шаг 03:**

#### **Миссия 3**

Иван хочет реализовать шардирование для таблицы пользователей. Какой критерий обычно используют для распределения данных по шардам?

Подсказка: Подумайте, что у всех пользователей есть уникальный идентификатор.

### **Ответ:**

- **По хешу уникального идентификатора пользователя**
- По имени пользователя
- По дате регистрации
- По IP-адресу сервера

## Шаг 04:

### Миссия 4

Саша создает резервные копии базы PostgreSQL. Что важно помнить при масштабировании и шардировании?

Подсказка: Бэкапы должны работать независимо для каждого шарда.

**Ответ:**

- **Создавать бэкапы каждого шарда отдельно**
- Делать один бэкап всей системы на одном сервере
- Бэкап не нужен, если есть репликация
- Можно просто копировать файлы данных

## Шаг 05:

### Миссия 5

 Сопоставьте понятия шардирования с их описанием.

**Ответ:**

- Шард
  - Шардирование
  - Горизонтальное масштабирование
  - Вертикальное масштабирование
1. Раздел базы данных на независимые части, каждая из которых хранится на отдельном сервере.
  2. Добавление ресурсов (CPU, RAM) на один сервер для увеличения производительности.
  3. Отдельная часть базы данных, размещенная на отдельном сервере.
  4. Добавление новых серверов для распределения нагрузки.

 **Подсказка:**

Сравните термины: горизонтальное и вертикальное масштабирование решают разные задачи.

- **Шард** — Отдельная часть базы данных, размещенная на отдельном сервере.
- **Шардирование** — Раздел базы данных на независимые части, каждая из которых хранится на отдельном сервере.
- **Горизонтальное масштабирование** — Добавление новых серверов для распределения нагрузки.
- **Вертикальное масштабирование** — Добавление ресурсов (CPU, RAM) на один сервер для увеличения производительности.

## Шаг 06:

### Миссия 6

 Сопоставьте типы репликации с их характеристиками.

**Ответ:**

- Синхронная репликация
  - Асинхронная репликация
  - Физическая репликация
  - Логическая репликация
1. Данные записываются на вторичный сервер одновременно с основным.
  2. Данные на вторичном сервере могут отставать от основного.

3. Копируются физические файлы базы данных.
4. Копируются изменения на уровне таблиц или строк.

#### **Подсказка:**

Обратите внимание: физическая/логическая репликация решают разные задачи, синхронная/асинхронная — вопрос задержки.

- **Синхронная репликация** — Данные записываются на вторичный сервер одновременно с основным.
- **Асинхронная репликация** — Данные на вторичном сервере могут отставать от основного.
- **Физическая репликация** — Копируются физические файлы базы данных.
- **Логическая репликация** — Копируются изменения на уровне таблиц или строк.

#### **Шаг 07:**

##### **Миссия 7**

 Сопоставьте типы масштабирования с реальными примерами:

##### **Ответ:**

- Шардирование
  - Репликация
  - Балансировка нагрузки
  - Кэширование
1. Разделение таблицы заказов по ID на несколько серверов
  2. Создание копий базы для распределения чтения
  3. Использование прокси для распределения запросов между серверами
  4. Использование Redis для ускорения повторяющихся запросов

#### **Подсказка:**

Подумайте, какая задача каждого подхода: шард — деление данных, кэш — ускорение, репликация — резерв, балансировка — распределение.

- **Шардирование** — Разделение таблицы заказов по ID на несколько серверов
- **Репликация** — Создание копий базы для распределения чтения
- **Балансировка нагрузки** — Использование прокси для распределения запросов между серверами
- **Кэширование** — Использование Redis для ускорения повторяющихся запросов

#### **Шаг 08:**

##### **Миссия 8**

 Сопоставьте этапы внедрения шардирования с их описанием:

##### **Ответ:**

- Определение шард-ключа
  - Создание шардов
  - Распределение данных
  - Тестирование и мониторинг
1. Выбор поля для разделения данных
  2. Создание отдельных баз/серверов для хранения частей данных
  3. Помещение данных в соответствующие шарды

#### 4. Проверка корректности распределения и производительности

##### **Подсказка:**

Сначала нужно выбрать ключ, затем создать шард и распределить данные, после чего проверяем систему.

- **Определение шард-ключа** — Выбор поля для разделения данных
- **Создание шардов** — Создание отдельных баз/серверов для хранения частей данных
- **Распределение данных** — Помещение данных в соответствующие шарды
- **Тестирование и мониторинг** — Проверка корректности распределения и производительности

##### **Шаг 09:**

#### **Миссия 9**

 Сопоставьте подходы к масштабированию с их преимуществами:

##### **Ответ:**

- Шардирование
  - Репликация
  - Кэширование
  - Балансировка нагрузки
1. Снижение нагрузки на основной сервер, ускорение чтения
  2. Позволяет обрабатывать больше данных за счет разделения
  3. Ускоряет повторяющиеся запросы и снижает задержки
  4. Равномерное распределение трафика между серверами

##### **Подсказка:**

Каждый метод решает отдельную проблему производительности и масштабируемости.

- **Шардирование** — Позволяет обрабатывать больше данных за счет разделения
- **Репликация** — Снижение нагрузки на основной сервер, ускорение чтения
- **Кэширование** — Ускоряет повторяющиеся запросы и снижает задержки
- **Балансировка нагрузки** — Равномерное распределение трафика между серверами

##### **Шаг 10:**

#### **Миссия 10**

 Сопоставьте виды кэша с их назначением:

##### **Ответ:**

- Memcached
  - Redis
  - Кэш на уровне базы данных
  - HTTP-кэш
1. Хранение временных объектов в памяти для ускорения запросов
  2. Поддержка структур данных, Pub/Sub и долговременного хранения
  3. Ускорение повторяющихся SQL-запросов
  4. Снижение нагрузки на сервер приложений при выдаче контента

##### **Подсказка:**

Подумайте, какой кэш подходит для чего: данные, SQL-запросы, контент, сложные структуры.

- **Memcached** — Хранение временных объектов в памяти для ускорения запросов
- **Redis** — Поддержка структур данных, Pub/Sub и долговременного хранения
- **Кэш на уровне базы данных** — Ускорение повторяющихся SQL-запросов
- **HTTP-кэш** — Снижение нагрузки на сервер приложений при выдаче контента

## Шаг 11:

### Миссия 11

Расположите шаги настройки репликации PostgreSQL в правильном порядке:

1. Создание публикации на основном сервере
2. Создание подписки на реплицирующем сервере
3. Проверка синхронизации таблиц
4. Мониторинг и устранение конфликтов

### Подсказка:

Сначала создаем публикацию, потом подписку, проверяем данные и следим за возможными конфликтами.

#### Ответ:

- Создание публикации на основном сервере
- Создание подписки на реплицирующем сервере
- Проверка синхронизации таблиц
- Мониторинг и устранение конфликтов

## Шаг 12:

### Миссия 12

Расположите шаги для внедрения HAProxy для балансировки нагрузки PostgreSQL:

1. Установка HAProxy на отдельный сервер
2. Настройка backend-серверов PostgreSQL
3. Конфигурация правил распределения запросов
4. Тестирование работы балансировщика

### Подсказка:

Важно сначала установить HAProxy, затем подключить серверы и настроить правила, после чего тестировать.

#### Ответ:

- Установка HAProxy на отдельный сервер
- Настройка backend-серверов PostgreSQL
- Конфигурация правил распределения запросов
- Тестирование работы балансировщика

## Шаг 13:

### Миссия 13

Расположите шаги масштабирования чтения в PostgreSQL с помощью репликации:

1. Создание слейв-серверов
2. Настройка асинхронной репликации
3. Перенаправление запросов на чтение к слейвам

#### 4. Мониторинг задержки репликации

##### **Подсказка:**

Сначала создаем слейвы, настраиваем репликацию, распределяем чтение и следим за задержкой.

##### **Ответ:**

- Создание слейв-серверов
- Настройка асинхронной репликации
- Перенаправление запросов на чтение к слейвам
- Мониторинг задержки репликации

##### **Шаг 14:**

##### **Миссия 14**

Расположите шаги для безопасного масштабирования базы с помощью шардирования и кэша:

1. Определение горячих и холодных данных
2. Распределение данных по шардированной инфраструктуре
3. Настройка кэша для горячих данных
4. Мониторинг производительности и корректировка стратегии

##### **Подсказка:**

Сначала анализируем данные, затем распределяем по шардам, добавляем кэш и следим за результатом.

##### **Ответ:**

- Определение горячих и холодных данных
- Распределение данных по шардированной инфраструктуре
- Настройка кэша для горячих данных
- Мониторинг производительности и корректировка стратегии

## **Модуль 7 - Безопасность, аудит, расширения**

### **Урок 1 - Управление доступом политики безопасности (RLS)**

##### **Шаг 01:**

##### **Миссия 1**

Что делает политика RLS в PostgreSQL?

Подсказка: Подумайте о том, кто может видеть строки таблицы.

##### **Ответ:**

- **Ограничивает доступ к строкам таблицы на уровне SQL-запросов**
- Блокирует подключение к базе данных
- Шифрует данные в таблице
- Создаёт резервную копию таблицы

##### **Шаг 02:**

##### **Миссия 2**

Когда стоит использовать RLS?

Подсказка: Подумайте о случаях, когда пользователи видят только часть данных.

**Ответ:**

- Когда данные пользователей чувствительны
- Когда нужно разграничивать доступ между отделами
- Когда нужно ускорить SELECT-запросы
- Когда требуется резервное копирование таблиц

### Шаг 03:

#### Миссия 3

 Сопоставьте команды RLS с их назначением:

**Ответ:**

- CREATE POLICY
- ALTER TABLE ... ENABLE ROW LEVEL SECURITY
- GRANT SELECT ON TABLE
- DROP POLICY

1. Создает правило для ограничения доступа к строкам
2. Включает механизм RLS для таблицы
3. Предоставляет разрешения на выборку строк таблицы
4. Удаляет ранее созданную политику

#### Подсказка:

Каждая команда решает отдельную задачу: создание, активация, предоставление прав или удаление.

- **CREATE POLICY** — Создает правило для ограничения доступа к строкам
- **ALTER TABLE ... ENABLE ROW LEVEL SECURITY** — Включает механизм RLS для таблицы
- **GRANT SELECT ON TABLE** — Предоставляет разрешения на выборку строк таблицы
- **DROP POLICY** — Удаляет ранее созданную политику

### Шаг 04:

#### Миссия 4

 Сопоставьте типы условий в RLS с примерами:

**Ответ:**

- USING
- WITH CHECK

1. Определяет строки, которые пользователь может видеть
2. Определяет строки, которые пользователь может вставлять или обновлять

#### Подсказка:

Подумайте о различии между чтением данных и их изменением.

- **USING** — Определяет строки, которые пользователь может видеть
- **WITH CHECK** — Определяет строки, которые пользователь может вставлять или обновлять

### Шаг 05:

#### Миссия 5

Расположите шаги настройки RLS на новой таблице:

1. Создание таблицы
2. Включение RLS
3. Создание политики доступа
4. Тестирование доступа пользователей

 **Подсказка:**

Сначала создаем таблицу, затем включаем RLS, добавляем политику и проверяем, работает ли всё корректно.

**Ответ:**

- Создание таблицы
- Включение RLS
- Создание политики доступа
- Тестирование доступа пользователей

 **Шаг 06:**

**Миссия 6**

Расположите шаги удаления политики RLS с минимальным риском:

1. Проверка пользователей, использующих политику
2. Отключение политики
3. Удаление политики
4. Тестирование доступа после удаления

 **Подсказка:**

Сначала проверяем влияние, затем отключаем, удаляем и проверяем результаты.

**Ответ:**

- Проверка пользователей, использующих политику
- Отключение политики
- Удаление политики
- Тестирование доступа после удаления

 **Шаг 07:**

**Миссия 7**

Как проверить, что политика RLS работает корректно?

Подсказка: Подумайте о тестировании с разными пользователями.

**Ответ:**

- **Выполнить SELECT от имени разных пользователей**
- **Попробовать вставку и обновление с разными правами**
- Сравнить структуру таблицы с другой базой
- Перезапустить сервер PostgreSQL

 **Шаг 08:**

**Миссия 8**

Что произойдет, если пользователь нарушает условие USING?

Подсказка: Подумайте о том, какие строки он сможет увидеть.

**Ответ:**

- **Пользователь не увидит строки, не удовлетворяющие условию**
- Пользователь сможет изменить все строки таблицы
- Запрос завершится с ошибкой сервера
- Все строки станут доступны для чтения

## Шаг 09:

### Миссия 9

 Сопоставьте действия с последствиями нарушения RLS:

**Ответ:**

- Вставка строки без разрешения
- Обновление строки с несоответствующим условием
- Удаление строки, доступ к которой запрещен
- SELECT строк, доступных по USING

1. Строка не добавится, операция заблокирована
2. Обновление не выполнится, операция отклонена
3. Удаление не будет выполнено
4. Строка успешно возвращена в выборке

## Подсказка:

Подумайте, что делает RLS: ограничивает доступ и изменения строк.

- **Вставка строки без разрешения** — Строка не добавится, операция заблокирована
- **Обновление строки с несоответствующим условием** — Обновление не выполнится, операция отклонена
- **Удаление строки, доступ к которой запрещен** — Удаление не будет выполнено
- **SELECT строк, доступных по USING** — Строка успешно возвращена в выборке

## Шаг 10:

### Миссия 10

 Сопоставьте типы пользователей с их правами при включенной RLS:

**Ответ:**

- Администратор
- Менеджер отдела
- Обычный пользователь
- Гость

1. Полный доступ ко всем данным
2. Доступ только к своему отделу
3. Доступ только к своим записям
4. Минимальный доступ или запрет доступа

## Подсказка:

Подумайте, кто должен видеть все данные, а кто только часть.

- **Администратор** — Полный доступ ко всем данным
- **Менеджер отдела** — Доступ только к своему отделу
- **Обычный пользователь** — Доступ только к своим записям

- **Гость** — Минимальный доступ или запрет доступа

## Шаг 11:

### Миссия 11

Расположите действия администратора при настройке RLS в правильном порядке:

1. Создать таблицу
2. Определить роли пользователей
3. Включить RLS на таблице
4. Создать политики доступа
5. Проверить доступ пользователей

### Подсказка:

Сначала создаём таблицу, определяем роли, включаем RLS, задаем политики и тестируем.

### Ответ:

- Создать таблицу
- Определить роли пользователей
- Включить RLS на таблице
- Создать политики доступа
- Проверить доступ пользователей

## Шаг 12:

### Миссия 12

Расположите шаги добавления новой политики RLS без простоев:

1. Создание новой политики
2. Тестирование на тестовом пользователе
3. Включение политики на боевой таблице
4. Проверка доступа всех пользователей

### Подсказка:

Сначала тестируем, затем включаем на боевой системе и проверяем.

### Ответ:

- Создание новой политики
- Тестирование на тестовом пользователе
- Включение политики на боевой таблице
- Проверка доступа всех пользователей

## Шаг 13:

### Миссия 13

Какие ошибки могут возникнуть при неправильной настройке RLS?

Подсказка: Подумайте о доступе и безопасности.

### Ответ:

- **Пользователи могут видеть чужие данные**
- **Невозможность вставки или обновления собственных строк**
- Сервер PostgreSQL перезапустится
- Данные автоматически шифруются неверно

## Шаг 14:

### Миссия 14

 Сопоставьте тип RLS-политики с примером:

**Ответ:**

- PERMISSIVE
- RESTRICTIVE

1. Строка видна, если хотя бы одна политика разрешает
2. Строка видна только если все политики разрешают

 **Подсказка:**

Подумайте, что происходит, если несколько политик конфликтуют.

- **PERMISSIVE** — Строка видна, если хотя бы одна политика разрешает
- **RESTRICTIVE** — Строка видна только если все политики разрешают

## Шаг 15:

### Миссия 15

Какая роль в PostgreSQL может обходить RLS при необходимости?

Подсказка: Подумайте о суперпользователях.

**Ответ:**

- **Роль с правами SUPERUSER**
- Обычный пользователь
- Гость
- Менеджер отдела

## Шаг 16:

### Миссия 16

Расположите шаги создания сложной RLS-политики с несколькими условиями:

1. Определение нужных ролей и прав
2. Создание базовой политики
3. Добавление дополнительных условий USING и WITH CHECK
4. Тестирование политики на разных ролях
5. Включение политики на боевой таблице

 **Подсказка:**

Сначала определяем роли, затем создаем политику, добавляем условия, тестируем и включаем на боевой системе.

**Ответ:**

- Определение нужных ролей и прав
- Создание базовой политики
- Добавление дополнительных условий USING и WITH CHECK
- Тестирование политики на разных ролях
- Включение политики на боевой таблице

## Урок 2 - Аудит изменений через триггеры

### Шаг 01:

#### Миссия 1

Для чего используют триггеры в PostgreSQL для аудита?

Подсказка: Подумайте, как можно автоматически фиксировать изменения.

**Ответ:**

- **Фиксировать изменения данных автоматически**
- **Создавать журналы операций INSERT, UPDATE, DELETE**
- Удалять старые таблицы
- Синхронизировать данные с другой базой без кода

### Шаг 02:

#### Миссия 2

Когда триггер на UPDATE срабатывает для аудита?

Подсказка: Подумайте, какие действия в таблице должны фиксироваться.

**Ответ:**

- **При каждом изменении существующей строки**
- При создании таблицы
- При удалении триггера
- При выполнении SELECT

### Шаг 03:

#### Миссия 3

 Сопоставьте тип триггера с примером аудита:

**Ответ:**

- BEFORE INSERT
  - AFTER INSERT
  - BEFORE UPDATE
  - AFTER DELETE
1. Запись в журнал перед добавлением данных
  2. Запись в журнал после добавления данных
  3. Фиксация старого значения перед изменением
  4. Фиксация удалённой строки после удаления

### Подсказка:

Обратите внимание, когда именно нужны старые значения, а когда новые.

- **BEFORE INSERT** — Запись в журнал перед добавлением данных
- **AFTER INSERT** — Запись в журнал после добавления данных
- **BEFORE UPDATE** — Фиксация старого значения перед изменением
- **AFTER DELETE** — Фиксация удалённой строки после удаления

### Шаг 04:

#### Миссия 4

Какие данные стоит сохранять в журнале аудита?

Подсказка: Подумайте, что поможет восстановить изменения и понять их причину.

**Ответ:**

- **Старые значения строк**
- **Новые значения строк**
- **Время изменения и пользователь**
- Структуру сервера PostgreSQL

## Шаг 05:

### Миссия 5

 Сопоставьте триггер и его роль в аудите:

**Ответ:**

- AFTER INSERT
  - AFTER UPDATE
  - AFTER DELETE
  - BEFORE UPDATE
1. Фиксация добавленных данных
  2. Фиксация изменённых данных
  3. Фиксация удалённых данных
  4. Проверка условий перед обновлением

 **Подсказка:**

Подумайте, когда нужно фиксировать результат, а когда — проверять условия.

- **AFTER INSERT** — Фиксация добавленных данных
- **AFTER UPDATE** — Фиксация изменённых данных
- **AFTER DELETE** — Фиксация удалённых данных
- **BEFORE UPDATE** — Проверка условий перед обновлением

## Шаг 06:

### Миссия 6

Какой язык используют для написания функций триггера в PostgreSQL?

Подсказка: Подумайте о встроенных средствах PostgreSQL.

**Ответ:**

- **PL/pgSQL**
- Python
- C++
- JavaScript

## Шаг 07:

### Миссия 7

Расположите шаги создания триггера аудита:

1. Создать таблицу журнала
2. Написать функцию триггера на PL/pgSQL
3. Создать триггер на основной таблице
4. Тестировать работу триггера

#### **Подсказка:**

Сначала нужна таблица для хранения, потом функция, потом триггер и тестирование.

#### **Ответ:**

- Создать таблицу журнала
- Написать функцию триггера на PL/pgSQL
- Создать триггер на основной таблице
- Тестировать работу триггера

#### **Шаг 08:**

##### **Миссия 8**

Расположите шаги создания триггера для аудита данных в правильном порядке:

1. Создание таблицы журнала изменений
2. Написание функции триггера на PL/pgSQL
3. Создание триггера на основной таблице
4. Проверка работы триггера с тестовыми данными

#### **Подсказка:**

Сначала нужна таблица для хранения, потом функция, потом триггер, и только в конце тестирование.

#### **Ответ:**

- Создание таблицы журнала изменений
- Написание функции триггера на PL/pgSQL
- Создание триггера на основной таблице
- Проверка работы триггера с тестовыми данными

#### **Шаг 09:**

##### **Миссия 9**

Что важно учитывать при создании триггеров аудита в высоконагруженной базе?

Подсказка: Подумайте о производительности и влиянии на работу основной таблицы.

#### **Ответ:**

- **Минимизировать тяжелую логику внутри триггера**
- **Использовать асинхронное логирование при необходимости**
- Вставлять тысячи строк в журнал в одной транзакции без ограничений
- Отключать индексы на основной таблице

#### **Шаг 10:**

##### **Миссия 10**

 Сопоставьте элементы триггера аудита с их назначением:

#### **Ответ:**

- OLD
  - NEW
  - TG\_OP
  - TG\_TABLE\_NAME
1. Старые значения строки перед изменением
  2. Новые значения строки после изменения

3. Тип операции (INSERT, UPDATE, DELETE)
4. Название таблицы, на которой сработал триггер

 **Подсказка:**

Подумайте, какие значения нужны для полной фиксации изменений.

- **OLD** — Старые значения строки перед изменением
- **NEW** — Новые значения строки после изменения
- **TG\_OP** — Тип операции (INSERT, UPDATE, DELETE)
- **TG\_TABLE\_NAME** — Название таблицы, на которой сработал триггер

 **Шаг 11:**

**Миссия 11**

Какие ограничения можно установить для триггеров аудита?

Подсказка: Подумайте о типах операций и строк, на которые триггер срабатывает.

**Ответ:**

- **Срабатывать только на UPDATE или DELETE**
- **Срабатывать только для определённых колонок**
- Срабатывать вне зависимости от таблицы и операции
- Игнорировать транзакции и фиксацию данных

 **Шаг 12:**

**Миссия 12**

Расположите шаги тестирования триггера аудита:

1. Вставка тестовой строки
2. Изменение строки
3. Удаление строки
4. Проверка таблицы журнала на корректность записей

 **Подсказка:**

Сначала создаём событие, потом проверяем, что оно правильно зафиксировано в журнале.

**Ответ:**

- Вставка тестовой строки
- Изменение строки
- Удаление строки
- Проверка таблицы журнала на корректность записей

 **Шаг 13:**

**Миссия 13**

 Сопоставьте тип события и возможную цель аудита:

**Ответ:**

- INSERT
- UPDATE
- DELETE
- TRUNCATE

1. Фиксация новых записей

2. Фиксация изменений существующих записей
3. Фиксация удалённых данных
4. Фиксация массового удаления всех строк

#### 🔔 Подсказка:

Подумайте, как разные операции влияют на данные и зачем их отслеживать.

- **INSERT** — Фиксация новых записей
- **UPDATE** — Фиксация изменений существующих записей
- **DELETE** — Фиксация удалённых данных
- **TRUNCATE** — Фиксация массового удаления всех строк

### 📖 Урок 3 - Расширения PostgreSQL PostGIS, pg\_stat\_statements, pg\_trgm

#### 🔔 Шаг 01:

##### Миссия 1

Вы только что установили PostgreSQL и хотите использовать расширение PostGIS. Что нужно сделать первым шагом?

Подсказка: Подумайте, прежде чем использовать расширение, его нужно подготовить.

**Ответ:**

- **Включить расширение в нужной базе с помощью CREATE EXTENSION**
- Просто начать использовать функции PostGIS без установки
- Перезапустить PostgreSQL
- Скачать сторонний клиент GIS

#### 🔔 Шаг 02:

##### Миссия 2

Разработчик хочет отслеживать производительность всех SQL-запросов. Какое расширение PostgreSQL ему поможет?

Подсказка: Смотрите на расширение, которое собирает статистику по запросам.

**Ответ:**

- **pg\_stat\_statements**
- PostGIS
- pg\_trgm
- hstore

#### 🔔 Шаг 03:

##### Миссия 3

Вы хотите ускорить поиск по текстовым полям с помощью похожих слов. Какое расширение следует подключить?

Подсказка: Расширение для триграмм и текстовой схожести.

**Ответ:**

- **pg\_trgm**
- PostGIS
- pg\_stat\_statements
- uuid-oss

## 🔧 Шаг 04:

### Миссия 4

Команда разработчиков хочет строить географические запросы для карты города. Что им нужно подключить?

Подсказка: Расширение для работы с геометрией и географией.

**Ответ:**

- **PostGIS**
- `pg_trgm`
- `pg_stat_statements`
- `citext`

## 🔧 Шаг 05:

### Миссия 5

Вы установили `pg_stat_statements` и хотите включить сбор статистики для всех запросов. Что нужно сделать?

Подсказка: Это делается через конфигурацию PostgreSQL.

**Ответ:**

- Установить `shared_preload_libraries = 'pg_stat_statements'` и перезапустить сервер
- Просто вызвать `SELECT * FROM pg_stat_statements`
- Создать таблицу `log_statements`
- Выполнить `VACUUM ANALYZE`

## 🔧 Шаг 06:

### Миссия 6

Сопоставьте расширение PostgreSQL с его основным назначением.

Подсказка: Обратите внимание на ключевое слово в названии.

**Ответ:**

- **PostGIS** — Работа с географическими данными и геометрией
- **`pg_stat_statements`** — Сбор статистики по выполненным SQL-запросам
- **`pg_trgm`** — Ускорение поиска по текстовым данным с помощью триграмм
- **`uuid-oss`** — Генерация уникальных идентификаторов

## 🔧 Шаг 07:

### Миссия 7

Сопоставьте функцию расширения с её возможностью.

Подсказка: Подумайте, какие задачи решает каждая функция.

**Ответ:**

- **`ST_Distance (PostGIS)`** — Вычисление расстояния между двумя геометриями
- **`similarity (pg_trgm)`** — Вычисление степени похожести строк
- **`pg_stat_statements_reset()`** — Сброс статистики по SQL-запросам
- **`uuid_generate_v4()`** — Создание случайного уникального идентификатора

## 🔧 Шаг 08:

### Миссия 8

Сопоставьте тип индекса с расширением, которое его использует для ускорения поиска.  
Подсказка: Индексы помогают искать быстрее, особенно при работе с текстом или геоданными.

**Ответ:**

- **GIST** — PostGIS
- **GIN** — pg\_trgm
- **B-Tree** — uuid-oss
- **HASH** — Общие текстовые поля

#### 🔮 Шаг 09:

##### Миссия 9

Сопоставьте SQL-конструкцию с её типичным применением.  
Подсказка: Подумайте, где удобно использовать функции расширений.

**Ответ:**

- **SELECT \* FROM pg\_stat\_statements** — Просмотр статистики запросов
- **SELECT similarity('cat', 'car')** — Определение схожести строк
- **SELECT ST\_Within(geom1, geom2)** — Проверка, входит ли геометрия внутри другой
- **SELECT uuid\_generate\_v4()** — Генерация уникального идентификатора

#### 🔮 Шаг 10:

##### Миссия 10

Сопоставьте проблему с решением с помощью расширения PostgreSQL.  
Подсказка: Каждое расширение решает конкретную задачу.

**Ответ:**

- **Нужен поиск похожих слов** — pg\_trgm
- **Нужно хранить географические объекты** — PostGIS
- **Хотим собирать статистику SQL-запросов** — pg\_stat\_statements
- **Нужен генератор уникальных идентификаторов** — uuid-oss

#### 🔮 Шаг 11:

##### Миссия 11

Сопоставьте расширение PostgreSQL с примером его использования в реальной компании.  
Подсказка: Подумайте о том, какие данные и задачи могут быть в проектах.

**Ответ:**

- **PostGIS** — Геоинформационная система для доставки товаров
- **pg\_stat\_statements** — Мониторинг медленных запросов в онлайн-магазине
- **pg\_trgm** — Поиск похожих названий товаров
- **uuid-oss** — Назначение уникальных ID пользователям

#### 🔮 Шаг 12:

##### Миссия 12

Расположите шаги подключения расширения PostGIS в правильном порядке:

1. Включение расширения в базе с CREATE EXTENSION
2. Установка пакета PostGIS через систему пакетов
3. Проверка функций PostGIS с помощью SELECT

#### 4. Перезапуск PostgreSQL (если требуется)

Подсказка: Сначала устанавливаем пакет, потом включаем расширение, проверяем работу, перезапускаем при необходимости.

##### Ответ:

- Установка пакета PostGIS через систему пакетов
- Включение расширения в базе с CREATE EXTENSION
- Проверка функций PostGIS с помощью SELECT
- Перезапуск PostgreSQL (если требуется)

#### Шаг 13:

##### Миссия 13

Расположите действия для подключения pg\_stat\_statements:

1. Перезапуск сервера после изменения shared\_preload\_libraries
2. Включение расширения с CREATE EXTENSION
3. Проверка данных через SELECT \* FROM pg\_stat\_statements
4. Добавление pg\_stat\_statements в shared\_preload\_libraries

Подсказка: Настройка через конфигурацию, включение в базе, перезапуск и проверка статистики.

##### Ответ:

- Добавление pg\_stat\_statements в shared\_preload\_libraries
- Включение расширения с CREATE EXTENSION
- Перезапуск сервера после изменения shared\_preload\_libraries
- Проверка данных через SELECT \* FROM pg\_stat\_statements

#### Шаг 14:

##### Миссия 14

Расположите этапы использования pg\_trgm для ускорения поиска по тексту:

1. Создание индекса GIN на текстовое поле
2. Использование функции similarity() для поиска похожих строк
3. Подключение расширения pg\_trgm
4. Проверка работы через SELECT

Подсказка: Подключаем расширение, создаем индекс, используем функции, проверяем результаты.

##### Ответ:

- Подключение расширения pg\_trgm
- Создание индекса GIN на текстовое поле
- Использование функции similarity() для поиска похожих строк
- Проверка работы через SELECT

#### Шаг 15:

##### Миссия 15

Расположите шаги для мониторинга производительности с помощью pg\_stat\_statements:

1. Сбросить статистику при необходимости
2. Собрать статистику выполнения запросов

3. Включить расширение и перезапустить сервер
4. Анализировать результаты SELECT

Подсказка: Подключаем расширение, собираем данные, при необходимости сбрасываем, анализируем.

**Ответ:**

- Включить расширение и перезапустить сервер
- Собрать статистику выполнения запросов
- Сбросить статистику при необходимости
- Анализировать результаты SELECT

## Урок 4 - Защитить таблицу с конфиденциальными данными и отслеживать измен

### Шаг 01:

#### Миссия 1

Вы хотите использовать расширение PostGIS для работы с картами в PostgreSQL. Какой ваш первый шаг?

Подсказка: Прежде чем использовать расширение, его нужно подключить.

**Ответ:**

- **Включить расширение в нужной базе с помощью CREATE EXTENSION**
- Начать использовать функции PostGIS без установки
- Перезапустить PostgreSQL
- Скачать сторонний GIS-клиент

### Шаг 02:

#### Миссия 2

Вы хотите собирать статистику по всем SQL-запросам. Какое расширение использовать?

Подсказка: Оно помогает анализировать производительность запросов.

**Ответ:**

- **pg\_stat\_statements**
- PostGIS
- pg\_trgm
- hstore

### Шаг 03:

#### Миссия 3

Вы хотите ускорить поиск по похожим словам. Какое расширение подключить?

Подсказка: Подумайте о расширении для работы с триграммами.

**Ответ:**

- **pg\_trgm**
- PostGIS
- pg\_stat\_statements
- uuid-osp

## 🔮 Шаг 04:

### Миссия 4

Команда строит географические запросы для карты города. Что подключить?

Подсказка: Расширение для работы с геометрией и географией.

**Ответ:**

- **PostGIS**
- `pg_trgm`
- `pg_stat_statements`
- `citext`

## 🔮 Шаг 05:

### Миссия 5

Вы установили `pg_stat_statements` и хотите включить сбор статистики. Что сделать?

Подсказка: Настройка через конфигурацию PostgreSQL.

**Ответ:**

- Установить `shared_preload_libraries = 'pg_stat_statements'` и перезапустить сервер
- Просто вызвать `SELECT * FROM pg_stat_statements`
- Создать таблицу `log_statements`
- Выполнить `VACUUM ANALYZE`

## 🔮 Шаг 06:

### Миссия 6

Сопоставьте расширение PostgreSQL с его основным назначением.

Подсказка: Подумайте, где используется каждая технология.

**Ответ:**

- **PostGIS** — Работа с географическими данными и геометрией
- **`pg_stat_statements`** — Сбор статистики по выполненным SQL-запросам
- **`pg_trgm`** — Ускорение поиска по текстовым данным с помощью триграмм
- **`uuid-oss`** — Генерация уникальных идентификаторов

## 🔮 Шаг 07:

### Миссия 7

Сопоставьте функцию расширения с её назначением.

Подсказка: Каждая функция решает специфическую задачу.

**Ответ:**

- **`ST_Distance (PostGIS)`** — Вычисление расстояния между двумя геометриями
- **`similarity (pg_trgm)`** — Вычисление степени похожести строк
- **`pg_stat_statements_reset()`** — Сброс статистики по SQL-запросам
- **`uuid_generate_v4()`** — Создание случайного уникального идентификатора

## 🔮 Шаг 08:

### Миссия 8

Сопоставьте тип индекса с расширением, которое его использует для ускорения поиска.

Подсказка: Подумайте, где нужны текстовые и геометрические индексы.

**Ответ:**

- **GIST** — PostGIS
- **GIN** — pg\_trgm
- **B-Tree** — uuid-ossdp
- **HASH** — Общие текстовые поля

## 🔮 Шаг 09:

### Миссия 9

Сопоставьте SQL-конструкцию с её применением.

Подсказка: Подумайте, какие функции дают результат сразу.

**Ответ:**

- **SELECT \* FROM pg\_stat\_statements** — Просмотр статистики запросов
- **SELECT similarity('cat', 'car')** — Определение схожести строк
- **SELECT ST\_Within(geom1, geom2)** — Проверка, входит ли геометрия внутрь другой
- **SELECT uuid\_generate\_v4()** — Генерация уникального идентификатора

## 🔮 Шаг 10:

### Миссия 10

Сопоставьте проблему с расширением PostgreSQL, которое её решает.

Подсказка: Каждое расширение заточено под конкретную задачу.

**Ответ:**

- **Нужен поиск похожих слов** — pg\_trgm
- **Нужно хранить географические объекты** — PostGIS
- **Хотим собирать статистику SQL-запросов** — pg\_stat\_statements
- **Нужен генератор уникальных идентификаторов** — uuid-ossdp

## 🔮 Шаг 11:

### Миссия 11

Сопоставьте расширение PostgreSQL с примером использования в компании.

Подсказка: Где каждая технология может быть реально применена?

**Ответ:**

- **PostGIS** — Геоинформационная система для доставки товаров
- **pg\_stat\_statements** — Мониторинг медленных запросов в онлайн-магазине
- **pg\_trgm** — Поиск похожих названий товаров
- **uuid-ossdp** — Назначение уникальных ID пользователям

## 🔮 Шаг 12:

### Миссия 12

Расположите шаги подключения расширения PostGIS в правильном порядке:

1. Установка пакета PostGIS через систему пакетов
2. Включение расширения в базе с CREATE EXTENSION
3. Перезапуск PostgreSQL (если требуется)
4. Проверка функций PostGIS с помощью SELECT

Подсказка: Сначала устанавливаем пакет, потом подключаем расширение, проверяем работу, перезапускаем при необходимости.

**Ответ:**

- Установка пакета PostGIS через систему пакетов
- Включение расширения в базе с CREATE EXTENSION
- Перезапуск PostgreSQL (если требуется)
- Проверка функций PostGIS с помощью SELECT

### Шаг 13:

#### **Миссия 13**

Расположите шаги подключения pg\_stat\_statements:

1. Добавление pg\_stat\_statements в shared\_preload\_libraries
2. Включение расширения с CREATE EXTENSION
3. Перезапуск сервера после изменения конфигурации
4. Проверка статистики через SELECT \* FROM pg\_stat\_statements

Подсказка: Настройка → подключение → перезапуск → проверка.

**Ответ:**

- Добавление pg\_stat\_statements в shared\_preload\_libraries
- Включение расширения с CREATE EXTENSION
- Перезапуск сервера после изменения конфигурации
- Проверка статистики через SELECT \* FROM pg\_stat\_statements

### Шаг 14:

#### **Миссия 14**

Расположите шаги использования pg\_trgm для ускорения поиска по тексту:

1. Подключение расширения pg\_trgm
2. Создание индекса GIN на текстовое поле
3. Использование функции similarity() для поиска похожих строк
4. Проверка работы через SELECT

Подсказка: Подключаем расширение → создаем индекс → используем функции → проверяем.

**Ответ:**

- Подключение расширения pg\_trgm
- Создание индекса GIN на текстовое поле
- Использование функции similarity() для поиска похожих строк
- Проверка работы через SELECT

### Шаг 15:

#### **Миссия 15**

Расположите шаги мониторинга производительности с помощью pg\_stat\_statements:

1. Включить расширение и перезапустить сервер
2. Собрать статистику выполнения запросов
3. Сбросить статистику при необходимости
4. Анализировать результаты SELECT

Подсказка: Подключаем → собираем → при необходимости сбрасываем → анализируем.

**Ответ:**

- Включить расширение и перезапустить сервер
- Собрать статистику выполнения запросов
- Сбросить статистику при необходимости
- Анализировать результаты SELECT

## **Модуль 8 - 🎓 Финальный модуль Проект под нагрузкой**

### **Урок 1 - Архитектура БД для реального продукта**

#### **Шаг 01:**

##### **Миссия 1**

Вы проектируете базу данных для нового стартапа и хотите выбрать оптимальную структуру таблиц. Что важно учитывать в первую очередь?

Подсказка: Подумайте, как структура данных повлияет на скорость и масштабируемость.

**Ответ:**

- **Нормализацию и зависимости между таблицами**
- Использовать только TEXT поля во всех таблицах
- Создать одну огромную таблицу для всех данных
- Не учитывать связи между таблицами

#### **Шаг 02:**

##### **Миссия 2**

Ваш продукт быстро растет, и нагрузка на базу данных увеличивается. Какой подход поможет распределить нагрузку?

Подсказка: Подумайте о горизонтальном масштабировании и разделении данных.

**Ответ:**

- **Шардирование и репликация данных**
- Добавление больше индексов без анализа
- Использовать один сервер с огромной памятью
- Игнорировать нагрузку и масштабировать позже

#### **Шаг 03:**

##### **Миссия 3**

Вы проектируете продукт с большим количеством аналитики. Что следует учесть при проектировании базы?

Подсказка: Подумайте о быстром выполнении сложных запросов.

**Ответ:**

- **Использовать индексирование и материализованные представления**
- Не использовать индексы, чтобы ускорить вставку
- Хранить данные только в JSON без схемы
- Делать все поля nullable без ограничений

## 🔮 Шаг 04:

### Миссия 4

Сопоставьте тип архитектуры с её особенностью.

Подсказка: Подумайте, где применяются горизонтальное и вертикальное масштабирование.

**Ответ:**

- **Монолитная БД** — Все данные на одном сервере, простота разработки, но сложно масштабировать
- **Многоуровневая архитектура** — Деление нагрузки между серверами, легко масштабировать
- **Шардированная БД** — Разделение данных на части по ключам для горизонтального масштабирования
- **Реплицированная БД** — Копирование данных на несколько серверов для отказоустойчивости

## 🔮 Шаг 05:

### Миссия 5

Сопоставьте понятие с его ролью при проектировании базы под нагрузку.

Подсказка: Каждый компонент системы решает конкретную задачу.

**Ответ:**

- **Индексы** — Ускорение выборки данных
- **Материализованные представления** — Предвычисление сложных запросов для ускорения отчетности
- **Кэширование** — Снижение нагрузки на базу путем хранения часто используемых данных
- **Шардирование** — Горизонтальное распределение данных по нескольким серверам

## 🔮 Шаг 06:

### Миссия 6

Сопоставьте тип индекса с его применением.

Подсказка: Подумайте, какие индексы помогают при выборке больших объемов данных.

**Ответ:**

- **B-Tree** — Стандартный индекс для точного поиска и диапазонов
- **GIN** — Для полнотекстового поиска и массивов
- **BRIN** — Для очень больших таблиц с последовательными значениями
- **Hash** — Для быстрого поиска точного значения

## 🔮 Шаг 07:

### Миссия 7

Сопоставьте подход к резервному копированию с его характеристикой.

Подсказка: Подумайте о скорости восстановления и объеме резервных данных.

**Ответ:**

- **Полное резервное копирование** — Сохраняет всю базу, долго создается, быстро восстанавливается
- **Инкрементное копирование** — Сохраняет только изменения с последнего бэкапа, быстро создается

- **Дифференциальное копирование** — Сохраняет изменения с последнего полного бэкапа
- **Репликация** — Постоянное копирование данных на другой сервер для отказоустойчивости

#### 🔮 Шаг 08:

##### Миссия 8

Сопоставьте тип нагрузки с рекомендуемым подходом к архитектуре.

Подсказка: Чтение и запись требуют разных стратегий.

**Ответ:**

- **Чтение доминирует** — Репликация на несколько серверов для масштабирования чтения
- **Запись доминирует** — Шардирование по ключу для горизонтальной нагрузки
- **Смешанная нагрузка** — Гибрид шардирования и репликации
- **Аналитическая нагрузка** — Использование отдельной аналитической базы или материализованных представлений

#### 🔮 Шаг 09:

##### Миссия 9

Ваша команда решает, стоит ли использовать репликацию для базы данных. Какое преимущество это даст?

Подсказка: Подумайте о доступности и отказоустойчивости.

**Ответ:**

- **Повышение отказоустойчивости и распределение нагрузки на чтение**
- Автоматическая нормализация данных
- Снижение объема дискового пространства
- Уменьшение времени разработки приложения

#### 🔮 Шаг 10:

##### Миссия 10

Вы проектируете БД для интернет-магазина с миллионами пользователей. Что поможет ускорить поиск по товарам?

Подсказка: Подумайте, какие индексы и расширения PostgreSQL поддерживают полнотекстовый поиск.

**Ответ:**

- **Использовать GIN-индексы и pg\_trgm для поиска по тексту**
- Все текстовые поля хранить как BLOB
- Отключить индексы, чтобы ускорить вставку
- Хранить данные только в JSON без индексов

#### 🔮 Шаг 11:

##### Миссия 11

Сопоставьте расширение PostgreSQL с его назначением.

Подсказка: Каждое расширение решает специфическую задачу.

**Ответ:**

- **PostGIS** — Поддержка географических данных и геопространственных запросов

- **pg\_stat\_statements** — Сбор статистики по выполнению SQL-запросов для оптимизации производительности
- **pg\_trgm** — Ускорение поиска по тексту с помощью триграммных индексов
- **pgcrypto** — Шифрование данных и создание хешей

## Шаг 12:

### Миссия 12

Расположите шаги подготовки БД к высоконагруженному проекту в правильном порядке:

1. Проанализировать требования и нагрузку
2. Спроектировать нормализованную схему
3. Добавить необходимые индексы и расширения
4. Настроить репликацию и шардирование
5. Запустить нагрузочные тесты и оптимизировать

Подсказка: Сначала проектирование, потом оптимизация и масштабирование.

#### Ответ:

- Проанализировать требования и нагрузку
- Спроектировать нормализованную схему
- Добавить необходимые индексы и расширения
- Настроить репликацию и шардирование
- Запустить нагрузочные тесты и оптимизировать

## Шаг 13:

### Миссия 13

Расположите шаги анализа производительности запросов в правильном порядке:

1. Собрать статистику выполнения запросов с помощью pg\_stat\_statements
2. Определить медленные или часто выполняемые запросы
3. Оптимизировать структуру таблиц и индексы
4. Повторно проверить скорость после изменений

Подсказка: Сначала собираем данные, потом анализируем и оптимизируем.

#### Ответ:

- Собрать статистику выполнения запросов с помощью pg\_stat\_statements
- Определить медленные или часто выполняемые запросы
- Оптимизировать структуру таблиц и индексы
- Повторно проверить скорость после изменений

## Урок 2 - Оптимизация ключевых запросов

## Шаг 01:

### Миссия 1

Ваш коллега замечает, что SELECT с JOIN выполняется медленно. Что лучше всего поможет ускорить запрос?

Подсказка: Подумайте о правильной организации индексов и использовании статистики.

#### Ответ:

- **Создать индексы на столбцах, участвующих в JOIN и WHERE**

- Добавить больше ORDER BY
- Увеличить количество строк в таблице
- Удалить все индексы, чтобы ускорить вставку

## 🔧 Шаг 02:

### Миссия 2

Вы разрабатываете API для высоконагруженного сервиса. Что поможет снизить нагрузку на базу данных?

Подсказка: Подумайте о кэшировании и сокращении количества запросов.

**Ответ:**

- **Использовать кэширование результатов часто выполняемых запросов**
- Увеличить количество транзакций
- Уменьшить количество индексов
- Удалить ограничения FOREIGN KEY

## 🔧 Шаг 03:

### Миссия 3

Сопоставьте инструмент PostgreSQL с его назначением для оптимизации запросов.

Подсказка: Каждый инструмент помогает решать определённую проблему производительности.

**Ответ:**

- **EXPLAIN** — Показывает план выполнения SQL-запроса
- **VACUUM** — Освобождает место после удалённых строк и обновляет статистику
- **ANALYZE** — Собирает статистику для оптимизатора
- **pg\_stat\_statements** — Сбор данных о частоте и времени выполнения запросов

## 🔧 Шаг 04:

### Миссия 4

Сопоставьте тип индекса PostgreSQL с его оптимизацией.

Подсказка: Индексы ускоряют поиск в разных ситуациях.

**Ответ:**

- **B-Tree** — Ускоряет точные совпадения и диапазонные запросы
- **GIN** — Оптимизация полнотекстового поиска и array-типов
- **GiST** — Поддержка геопространственных данных и сложных операторов
- **BRIN** — Эффективен для больших таблиц с последовательными значениями

## 🔧 Шаг 05:

### Миссия 5

Расположите шаги оптимизации ключевого запроса в правильном порядке:

1. Проанализировать EXPLAIN-план для выявления узких мест
2. Добавить индексы на часто используемые столбцы
3. Переписать запрос с учётом JOIN и фильтров
4. Повторно проверить план выполнения и измерить время

Подсказка: Сначала выявляем проблему, потом исправляем и проверяем результат.

#### Ответ:

- Проанализировать EXPLAIN-план для выявления узких мест
- Добавить индексы на часто используемые столбцы
- Переписать запрос с учётом JOIN и фильтров
- Повторно проверить план выполнения и измерить время

#### 🔮 Шаг 06:

##### Миссия 6

Сопоставьте методы оптимизации запросов с их назначением в PostgreSQL.

Подсказка: Подумайте, как каждый метод влияет на скорость и нагрузку на базу данных.

#### Ответ:

- **Использование индексов** — Сокращает количество сканируемых строк при поиске
- **Партиционирование таблиц** — Позволяет обрабатывать только нужную часть данных
- **Материализованные представления** — Хранение результата сложного запроса для быстрого доступа
- **Подготовленные запросы (Prepared Statements)** — Повышает скорость повторных вызовов одного и того же запроса

#### 🔮 Шаг 07:

##### Миссия 7

Какая стратегия поможет ускорить поиск по текстовым данным в большом объёме таблицы?

Подсказка: Подумайте о индексах, которые оптимизированы для текстового поиска.

#### Ответ:

- **Создать GIN-индекс для полнотекстового поиска**
- Создать B-Tree индекс на весь текст
- Удалить все индексы
- Сортировать таблицу по текстовому полю

#### 🔮 Шаг 08:

##### Миссия 8

Сопоставьте показатели pg\_stat\_statements с их значением для анализа запросов.

Подсказка: Эти показатели помогают понять, где узкие места в производительности.

#### Ответ:

- **calls** — Количество выполнений запроса
- **total\_time** — Общее время выполнения всех вызовов
- **rows** — Количество возвращённых строк
- **shared\_blks\_hit** — Число попаданий в shared buffer cache

#### 🔮 Шаг 09:

##### Миссия 9

Расположите шаги оптимизации SELECT-запроса с JOIN в правильном порядке:

1. Анализ EXPLAIN-плана
2. Создание необходимых индексов
3. Рефакторинг запроса для минимизации JOIN

#### 4. Проверка времени выполнения и повторная корректировка

Подсказка: Сначала ищем узкие места, потом исправляем и проверяем.

**Ответ:**

- Анализ EXPLAIN-плана
- Создание необходимых индексов
- Рефакторинг запроса для минимизации JOIN
- Проверка времени выполнения и повторная корректировка

#### Шаг 10:

##### Миссия 10

Вы заметили, что INSERT в большую таблицу замедляется. Как лучше оптимизировать процесс?

Подсказка: Подумайте о пакетной вставке и влиянии индексов на скорость.

**Ответ:**

- **Использовать batch INSERT и временно отключить индексы**
- Увеличить количество индексов
- Добавлять строки по одной
- Использовать SELECT вместо INSERT

#### Шаг 11:

##### Миссия 11

Сопоставьте типы индексов с их лучшим применением:

Подсказка: Каждый индекс подходит для конкретной задачи поиска или фильтрации.

**Ответ:**

- **B-Tree** — Для равенства и диапазонных запросов
- **GIN** — Для полнотекстового поиска и массивов
- **BRIN** — Для очень больших таблиц с последовательными значениями
- **GiST** — Для геопространственных данных

#### Шаг 12:

##### Миссия 12

Расположите шаги по оптимизации агрегатных функций в правильном порядке:

1. Использовать индексы для фильтрации
2. Переписать запрос с использованием CTE или подзапросов
3. Выполнить EXPLAIN ANALYZE для оценки плана
4. Проверить время выполнения и оптимизировать далее

Подсказка: Сначала выявляем проблемные места, затем переписываем и проверяем.

**Ответ:**

- Использовать индексы для фильтрации
- Переписать запрос с использованием CTE или подзапросов
- Выполнить EXPLAIN ANALYZE для оценки плана
- Проверить время выполнения и оптимизировать далее

## 🔮 Шаг 13:

### Миссия 13

Какой подход уменьшит нагрузку при частых SELECT с фильтром по нескольким столбцам?

Подсказка: Подумайте о составных индексах.

**Ответ:**

- **Создать составной индекс по этим столбцам**
- Создать отдельные индексы на каждый столбец
- Удалить все индексы
- Использовать ORDER BY вместо индексов

## 🔮 Шаг 14:

### Миссия 14

Сопоставьте инструменты PostgreSQL с их ролью в оптимизации нагрузки:

Подсказка: Каждый инструмент помогает выявить или уменьшить узкие места.

**Ответ:**

- **EXPLAIN** — Показывает план выполнения запроса
- **VACUUM** — Освобождает место и обновляет статистику
- **ANALYZE** — Обновляет статистику для оптимизатора
- **pg\_stat\_activity** — Отслеживает активные соединения и запросы

## 🔮 Шаг 15:

### Миссия 15

Расположите действия DBA по оптимизации крупной таблицы с миллионами строк:

1. Проанализировать EXPLAIN-план для выявления медленных запросов
2. Создать необходимые индексы и партиционировать таблицу
3. Очистить устаревшие данные и выполнить VACUUM
4. Переписать тяжелые запросы с использованием подзапросов и CTE

Подсказка: Действия идут от анализа к исправлению и проверке.

**Ответ:**

- Проанализировать EXPLAIN-план для выявления медленных запросов
- Создать необходимые индексы и партиционировать таблицу
- Очистить устаревшие данные и выполнить VACUUM
- Переписать тяжелые запросы с использованием подзапросов и CTE

## 🔮 Шаг 16:

### Миссия 16

Какая стратегия поможет ускорить выборку из таблицы с JSONB-полями?

Подсказка: Подумайте о специализированных индексах для JSONB.

**Ответ:**

- **Создать GIN-индекс на JSONB-поле**
- Создать B-Tree индекс на весь JSONB
- Сериализовать JSONB в текст и индексировать
- Удалить JSONB и хранить как текст

## Урок 3 - Функции + безопасный доступ

### Шаг 01:

#### Миссия 1

Какой тип функции в PostgreSQL позволяет ограничить доступ к данным и управлять безопасностью?

Подсказка: Подумайте о функциях, которые могут выполняться с привилегиями владельца.

**Ответ:**

- **SECURITY DEFINER функции**
- Функции с VOLATILE
- IMMUTABLE функции
- Функции без параметров

### Шаг 02:

#### Миссия 2

Какая функция поможет преобразовать строку в верхний регистр безопасно внутри функции?

Подсказка: Подумайте о стандартных строковых функциях PostgreSQL.

**Ответ:**

- **UPPER(text)**
- LOWER(text)
- INITCAP(text)
- LENGTH(text)

### Шаг 03:

#### Миссия 3

Какой уровень привилегий используется для функции с SECURITY DEFINER?

Подсказка: Она выполняется не от имени вызывающего пользователя.

**Ответ:**

- **От имени владельца функции**
- От имени суперпользователя
- От имени текущего пользователя
- Без привилегий

### Шаг 04:

#### Миссия 4

Сопоставьте тип функции PostgreSQL с её описанием.

Подсказка: Обратите внимание, какие функции могут изменять данные и какие нет.

**Ответ:**

- **IMMUTABLE** — Не изменяет базу данных, всегда возвращает одинаковый результат
- **STABLE** — Не изменяет данные, может зависеть от текущей сессии
- **VOLATILE** — Может изменять данные и иметь побочные эффекты
- **SECURITY DEFINER** — Выполняется с привилегиями владельца функции

## Шаг 05:

### Миссия 5

Сопоставьте функции PostgreSQL с их типичными применениями.

Подсказка: Подумайте о задачах, которые решаются с помощью встроенных функций.

**Ответ:**

- **NOW()** — Возвращает текущее время и дату
- **CURRENT\_USER** — Показывает имя текущего пользователя
- **SESSION\_USER** — Показывает имя пользователя сессии
- **SET ROLE** — Меняет роль текущего пользователя для безопасности

## Шаг 06:

### Миссия 6

Сопоставьте тип данных функции с примерами использования.

Подсказка: Данные функций часто определяют, какие операции можно выполнять внутри.

**Ответ:**

- **text** — Хранение строк, обработка сообщений и логов
- **integer** — Счётчики, идентификаторы и арифметика
- **boolean** — Флаги, условия и проверки
- **timestamp** — Отслеживание времени создания или изменения

## Шаг 07:

### Миссия 7

Расположите шаги безопасного создания функции в PostgreSQL:

1. Определить входные параметры и тип возвращаемого значения
2. Выбрать SECURITY DEFINER или SECURITY INVOKER
3. Написать тело функции с проверкой прав доступа
4. Протестировать функцию и выдать минимально необходимые права

Подсказка: Сначала проектируем, потом пишем и защищаем.

**Ответ:**

- Определить входные параметры и тип возвращаемого значения
- Выбрать SECURITY DEFINER или SECURITY INVOKER
- Написать тело функции с проверкой прав доступа
- Протестировать функцию и выдать минимально необходимые права

## Шаг 08:

### Миссия 8

Расположите шаги по обеспечению безопасного доступа через функции:

1. Создать роль с минимальными правами
2. Создать функцию с SECURITY DEFINER
3. Тестировать функцию от имени другой роли
4. Разрешить доступ только к функции, а не к таблицам

Подсказка: Сначала создаём роли, потом функции и проверяем доступ.

**Ответ:**

- Создать роль с минимальными правами
- Создать функцию с SECURITY DEFINER
- Тестировать функцию от имени другой роли
- Разрешить доступ только к функции, а не к таблицам

#### Шаг 09:

##### **Миссия 9**

Какая практика повышает безопасность функций в PostgreSQL?

Подсказка: Подумайте о проверке прав и минимизации привилегий.

**Ответ:**

- **Выдавать минимальные права и использовать SECURITY DEFINER**
- Давать всем пользователям полный доступ к таблицам
- Использовать VOLATILE без ограничений
- Хранить пароли в функции без шифрования

#### Шаг 10:

##### **Миссия 10**

Какой тип функции следует использовать для вычислений без изменения данных?

Подсказка: Подумайте, какие функции безопасны для вызова многократно.

**Ответ:**

- **IMMUTABLE**
- VOLATILE
- STABLE с побочными эффектами
- SECURITY INVOKER

#### Шаг 11:

##### **Миссия 11**

Сопоставьте встроенные функции PostgreSQL с их назначением:

Подсказка: Обратите внимание, какие функции помогают управлять безопасностью, а какие просто вычисляют значения.

**Ответ:**

- **CURRENT\_USER** — Возвращает имя пользователя, вызывающего функцию
- **SESSION\_USER** — Возвращает имя пользователя сессии
- **has\_table\_privilege** — Проверяет права доступа к таблице
- **pg\_has\_role** — Проверяет, есть ли роль у текущего пользователя

#### Шаг 12:

##### **Миссия 12**

Расположите шаги по созданию безопасной функции для обработки финансовых данных:

1. Определить минимальные права доступа к таблице
2. Создать функцию с SECURITY DEFINER
3. Включить проверки входных данных и обработку ошибок
4. Протестировать функцию от имени пользователя с ограниченными правами

Подсказка: Сначала права, затем функция, потом тестирование.

**Ответ:**

- Определить минимальные права доступа к таблице
- Создать функцию с SECURITY DEFINER
- Включить проверки входных данных и обработку ошибок
- Протестировать функцию от имени пользователя с ограниченными правами

### Шаг 13:

#### **Миссия 13**

Расположите шаги безопасной работы с логами через функции:

1. Создать таблицу логов с минимальными правами
2. Создать функцию для вставки логов с SECURITY DEFINER
3. Тестировать функцию от имени обычного пользователя
4. Ограничить прямой доступ к таблице логов

Подсказка: Защита таблицы и функции идёт рука об руку.

**Ответ:**

- Создать таблицу логов с минимальными правами
- Создать функцию для вставки логов с SECURITY DEFINER
- Тестировать функцию от имени обычного пользователя
- Ограничить прямой доступ к таблице логов

### Шаг 14:

#### **Миссия 14**

Расположите шаги по безопасной обработке пользовательских данных через функцию:

1. Проверка входных параметров на допустимые значения
2. Создание функции с SECURITY DEFINER
3. Запись данных в таблицу с ограниченным доступом
4. Тестирование функции с разными ролями пользователей

Подсказка: Сначала проверяем данные, потом пишем функцию и тестируем доступ.

**Ответ:**

- Проверка входных параметров на допустимые значения
- Создание функции с SECURITY DEFINER
- Запись данных в таблицу с ограниченным доступом
- Тестирование функции с разными ролями пользователей

### Шаг 15:

#### **Миссия 15**

Какой подход поможет избежать утечки привилегий при использовании функций?

Подсказка: Подумайте, кто выполняет функцию и с какими правами.

**Ответ:**

- **Использовать SECURITY INVOKER там, где данные должны оставаться защищёнными**
- Давать функции права суперпользователя по умолчанию

- Игнорировать проверку входных данных
- Создавать функции без ролей

#### 🔮 Шаг 16:

##### Миссия 16

Сопоставьте элементы безопасности PostgreSQL с их назначением:

Подсказка: Подумайте, что обеспечивает контроль доступа, а что аудит действий.

**Ответ:**

- **ROLE** — Определяет права пользователя и его группу
- **SECURITY DEFINER** — Функция выполняется с правами владельца
- **GRANT** — Выдаёт доступ к объектам базы данных
- **REVOKE** — Отбирает ранее выданные права

#### 📖 Урок 4 - Индексация, backup, анализ

#### 🔮 Шаг 01:

##### Миссия 1

Какой тип индекса чаще всего используют для ускорения поиска по точному совпадению?

Подсказка: Подумайте о быстром доступе по уникальному значению.

**Ответ:**

- **B-Tree**
- HashMap
- GIN
- Bitmap

#### 🔮 Шаг 02:

##### Миссия 2

Что важно учитывать при создании индекса на большой таблице с миллионами строк?

Подсказка: Индекс ускоряет чтение, но может замедлить запись.

**Ответ:**

- **Баланс между чтением и записью**
- Индекс всегда ускоряет все операции
- Индексы не занимают место
- Не влияет на производительность JOIN

#### 🔮 Шаг 03:

##### Миссия 3

Какой метод резервного копирования в PostgreSQL позволяет минимизировать время простоя базы?

Подсказка: Подумайте о способе, который не блокирует таблицы на запись.

**Ответ:**

- **pg\_dump с параллельной опцией**
- Полное выключение сервера
- Копирование файлов вручную без WAL
- Резервное копирование только схемы

## 🔮 Шаг 04:

### Миссия 4

Сопоставьте тип индекса с его применением:

Подсказка: Вспомните, какие индексы подходят для строк, массивов или JSON.

**Ответ:**

- **B-Tree** — Для быстрого поиска по точным значениям и диапазонам
- **GIN** — Для поиска по массивам, JSON и полнотекстовым полям
- **Hash** — Для поиска по точным хэш-значениям
- **BRIN** — Для больших таблиц с последовательными значениями

## 🔮 Шаг 05:

### Миссия 5

Сопоставьте команды PostgreSQL с их назначением:

Подсказка: Обратите внимание, что backup и restore имеют разные команды.

**Ответ:**

- **pg\_dump** — Создание резервной копии базы данных
- **pg\_restore** — Восстановление базы данных из дампа
- **VACUUM** — Очистка удалённых записей и оптимизация таблиц
- **ANALYZE** — Сбор статистики для оптимизатора запросов

## 🔮 Шаг 06:

### Миссия 6

Сопоставьте понятия анализа производительности с их назначением:

Подсказка: Подумайте, какие инструменты помогают выявить узкие места.

**Ответ:**

- **EXPLAIN** — Показ плана выполнения запроса
- **EXPLAIN ANALYZE** — Показ плана с реальным временем выполнения
- **pg\_stat\_activity** — Мониторинг текущих подключений и запросов
- **pg\_stat\_user\_tables** — Статистика использования таблиц пользователями

## 🔮 Шаг 07:

### Миссия 7

Что из ниже перечисленного ускоряет выполнение SELECT запросов на больших таблицах?

Подсказка: Подумайте про индексирование и порядок данных.

**Ответ:**

- **Создание индексов на часто используемых колонках**
- **Сбор статистики с ANALYZE**
- Удаление индексов
- Добавление лишних триггеров на SELECT

## 🔮 Шаг 08:

### Миссия 8

При каком типе резервного копирования база данных может оставаться доступной для записи?

Подсказка: Подумайте про горячее backup и онлайн доступ.

**Ответ:**

- **Горячее резервное копирование (online backup)**
- Cold backup при остановленном сервере
- Копирование данных без WAL
- Резервное копирование только индексов

## Шаг 09:

### Миссия 9

Сопоставьте команды резервного копирования с их свойствами:

Подсказка: Разные инструменты создают дампы в разных форматах.

**Ответ:**

- **pg\_dump** — Создаёт дамп в текстовом или custom формате
- **pg\_dumpall** — Создаёт дамп всех баз вместе с ролями и настройками
- **pg\_restore** — Восстанавливает базу из custom или tar дампа
- **rsync** — Синхронизация файлов, не создаёт логическую копию

## Шаг 10:

### Миссия 10

Какая практика помогает поддерживать высокую производительность больших баз с миллионами строк?

Подсказка: Подумайте про индексы и очистку старых данных.

**Ответ:**

- **Регулярный VACUUM и ANALYZE**
- **Оптимизация запросов и индексов**
- Отключение индексов навсегда
- Игнорирование статистики таблиц

## Шаг 11:

### Миссия 11

Расположите шаги для анализа производительности запроса:

1. EXPLAIN запрос
2. Сбор статистики через ANALYZE
3. Оптимизация индексов и условий WHERE
4. EXPLAIN ANALYZE и проверка реального времени выполнения

Подсказка: Сначала смотрим план, потом собираем статистику и оптимизируем.

**Ответ:**

- EXPLAIN запрос
- Сбор статистики через ANALYZE
- Оптимизация индексов и условий WHERE
- EXPLAIN ANALYZE и проверка реального времени выполнения

## Шаг 12:

### Миссия 12

Расположите шаги резервного копирования для минимизации потерь:

1. Создание дампа через `pg_dump` или `pg_dumpall`
2. Проверка целостности дампа
3. Хранение дампа в защищённом месте
4. Тестовое восстановление на отдельной базе

Подсказка: Сначала делаем копию, затем проверяем и тестируем восстановление.

**Ответ:**

- Создание дампа через `pg_dump` или `pg_dumpall`
- Проверка целостности дампа
- Хранение дампа в защищённом месте
- Тестовое восстановление на отдельной базе

### Шаг 13:

#### Миссия 13

Расположите этапы создания и использования индекса:

1. Выбор колонки для индексирования
2. Создание индекса с подходящим типом
3. Тестирование производительности `SELECT` запросов
4. Поддержание индекса актуальным через `VACUUM/REINDEX`

Подсказка: Сначала выбираем колонку, потом создаём индекс и проверяем его эффективность.

**Ответ:**

- Выбор колонки для индексирования
- Создание индекса с подходящим типом
- Тестирование производительности `SELECT` запросов
- Поддержание индекса актуальным через `VACUUM/REINDEX`

### Шаг 14:

#### Миссия 14

Какой тип индекса лучше использовать для быстрого поиска по частичному совпадению текста?

Подсказка: Подумайте про индексы для текстовых полей и полнотекстовый поиск.

**Ответ:**

- **GIN**
- B-Tree
- BRIN
- Hash

### Шаг 15:

#### Миссия 15

Сопоставьте виды резервного копирования с их особенностями:

Подсказка: Подумайте, какие методы требуют остановки сервера, а какие нет.

**Ответ:**

- **Cold backup** — Требуется остановки сервера, полная копия файлов
- **Hot backup** — Можно делать на работающей базе, использует WAL для согласованности

- **Logical backup** — Дамп таблиц и схем в SQL или custom формат
- **Physical backup** — Копирование всех файлов базы данных, включая WAL

#### 🔔 Шаг 16:

##### Миссия 16

Расположите шаги восстановления базы из резервной копии:

1. Подготовка пустой базы для восстановления
2. Запуск `pg_restore` или восстановление файлов физического бэкапа
3. Применение WAL для актуализации данных (если hot backup)
4. Проверка целостности и корректности данных

Подсказка: Сначала готовим среду, затем восстанавливаем, применяем журнал и проверяем данные.

##### Ответ:

- Подготовка пустой базы для восстановления
- Запуск `pg_restore` или восстановление файлов физического бэкапа
- Применение WAL для актуализации данных (если hot backup)
- Проверка целостности и корректности данных

## 📖 Урок 5 - Создать MVP-схему + документацию и план поддержки

#### 🔔 Шаг 01:

##### Миссия 1

Какой тип индекса чаще всего используется для ускорения поиска по ключам?

Подсказка: Подумайте про стандартный способ индексирования числовых и строковых ключей.

##### Ответ:

- **B-Tree**
- GIN
- Hash
- BRIN

#### 🔔 Шаг 02:

##### Миссия 2

Что ускоряет поиск по текстовым данным в больших таблицах?

Подсказка: Рассмотрите индексы, которые эффективны для полнотекстового поиска.

##### Ответ:

- **GIN индекс**
- B-Tree индекс
- Hash индекс
- BRIN индекс

#### 🔔 Шаг 03:

##### Миссия 3

Какой тип индекса эффективен для колонок с монотонными значениями, например, временными метками?

Подсказка: Рассмотрите индексы, которые экономят память и подходят для больших таблиц.

**Ответ:**

- **BRIN**
- B-Tree
- GIN
- Hash

#### 🔮 Шаг 04:

##### Миссия 4

Сопоставьте типы индексов с их особенностями:

Подсказка: Подумайте про подходящие сценарии использования каждого типа.

**Ответ:**

- **B-Tree** — Хорош для сортировки и поиска по ключам
- **GIN** — Эффективен для полнотекстового поиска
- **BRIN** — Идеален для больших таблиц с монотонными данными
- **Hash** — Быстро ищет точные совпадения

#### 🔮 Шаг 05:

##### Миссия 5

Сопоставьте виды резервного копирования с их особенностями:

Подсказка: Подумайте, какие методы требуют остановки сервера, а какие нет.

**Ответ:**

- **Cold backup** — Требуется остановки сервера, полная копия файлов
- **Hot backup** — Можно делать на работающей базе, использует WAL для согласованности
- **Logical backup** — Дамп таблиц и схем в SQL или custom формат
- **Physical backup** — Копирование всех файлов базы данных, включая WAL

#### 🔮 Шаг 06:

##### Миссия 6

Какой инструмент PostgreSQL используют для логического резервного копирования?

Подсказка: Это инструмент, который создаёт дампы таблиц и схем.

**Ответ:**

- **pg\_dump**
- pg\_restore
- pg\_basebackup
- psql

#### 🔮 Шаг 07:

##### Миссия 7

Что обеспечивает согласованность данных при восстановлении из hot backup?

Подсказка: Подумайте про механизм, который записывает все изменения после создания копии.

**Ответ:**

- **WAL (Write-Ahead Log)**
- B-Tree
- GIN

- `pg_dump`

## 🔮 Шаг 08:

### Миссия 8

Сопоставьте этапы анализа запросов с их описанием:

Подсказка: Используйте EXPLAIN для понимания планов выполнения.

**Ответ:**

- **EXPLAIN** — Показывает план выполнения запроса
- **ANALYZE** — Выполняет запрос и собирает статистику по фактическим затратам
- **VACUUM** — Очищает удалённые строки и обновляет статистику
- **REINDEX** — Перестраивает индекс для оптимизации поиска

## 🔮 Шаг 09:

### Миссия 9

Сопоставьте команды резервного копирования с их результатом:

Подсказка: Подумайте, какие команды создают дампы, а какие физические копии.

**Ответ:**

- **pg\_dump** — Создаёт логический дамп базы
- **pg\_restore** — Восстанавливает из дампа
- **pg\_basebackup** — Создаёт физическую копию всей базы
- **rsync файлов** — Копирует файлы базы вручную

## 🔮 Шаг 10:

### Миссия 10

Какой индекс следует использовать для ускорения агрегатных запросов по диапазону дат?

Подсказка: Подумайте о компромиссе между размером индекса и быстродействием.

**Ответ:**

- **BRIN**
- B-Tree
- GIN
- Hash

## 🔮 Шаг 11:

### Миссия 11

Что ускоряет выборку данных по нескольким колонкам одновременно?

Подсказка: Подумайте о составных индексах.

**Ответ:**

- **Составной B-Tree индекс**
- GIN индекс
- Hash индекс
- BRIN индекс

## 🔮 Шаг 12:

### Миссия 12

Сопоставьте термины по индексации с их определением:

Подсказка: Рассмотрите, какой индекс лучше подходит для каждого сценария.

**Ответ:**

- **Составной индекс** — Индекс, включающий несколько колонок
- **Уникальный индекс** — Обеспечивает уникальность значений
- **Partial индекс** — Индексирует только часть строк, соответствующих условию
- **Functional индекс** — Индексирует результат функции от колонок

### 🔮 Шаг 13:

#### Миссия 13

Сопоставьте этапы восстановления базы из резервной копии:

Подсказка: Сначала подготовка, затем восстановление, потом применение WAL и проверка.

**Ответ:**

- **Подготовка пустой базы** — Создание базы для восстановления
- **Восстановление дампа или файлов** — Использование `pg_restore` или копирование файлов
- **Применение WAL** — Актуализация данных после hot backup
- **Проверка целостности** — Убедиться, что данные корректны

### 🔮 Шаг 14:

#### Миссия 14

Какой инструмент PostgreSQL используют для физического резервного копирования?

Подсказка: Этот инструмент копирует файлы базы целиком.

**Ответ:**

- **pg\_basebackup**
- `pg_dump`
- `pg_restore`
- `VACUUM`